```
# 文件: app/main.py
</final>
# 文件: app/config.py
import json
import os
from typing import Any, Dict, Optional
from app.core.logger import get_logger
logger = get_logger(__name__)
APP_NAME = "折痕图可视化编辑与标准化导出系统"
APP_VERSION = "1.0.0"
CONFIG_DIR_NAME = "CreasePatternStudio"
CONFIG_FILE_NAME = "settings.json"
class AppConfig:
    def __init__(self, config_dir: Optional[str] = None) -> None:
        self._config_dir = config_dir or self._default_config_dir()
        self._config_path = os.path.join(self._config_dir, CONFIG_FILE_NAME)
        self._data: Dict[str, Any] = {}
        self._defaults: Dict[str, Any] = self._build_defaults()
        self._data = dict(self._defaults)
        self.load()
    @staticmethod
    def _default_config_dir() -> str:
        base = os.environ.get("APPDATA") or os.path.expanduser("~")
        return os.path.join(base, CONFIG_DIR_NAME)
    @staticmethod
    def _build_defaults() -> Dict[str, Any]:
        return {
            "app_name": APP_NAME,
            "app_version": APP_VERSION,
            "window": {
                "width": 1360,
                "height": 860,
                "maximized": False,
                "remember_geometry": True,
            },
            "canvas": {
                "background_color": "
                "grid_visible": True,
                "grid_size": 20,
                "grid_color": "
                "snap_enabled": True,
                "snap_threshold": 8,
                "show_vertices": True,
                "vertex_radius": 3,
                "zoom_min": 0.1,
                "zoom_max": 20.0,
            },
            "crease_style": {
                "mountain_color": "
                "valley_color": "
                "flat_color": "
                "cut_color": "
                "boundary_color": "
                "auxiliary_color": "
                "line_width": 2.0,
                "dash_pattern": [6, 4],
            },
            "validation": {
                "auto_validate": True,
                "angle_tolerance": 0.5,
                "min_vertex_angle": 5.0,
                "detect_overlap": True,
                "detect_intersection": True,
                "warn_flat_foldable": True,
            },
            "preview": {
                "enabled": True,
                "background_color": "
                "show_axes": True,
                "fold_angle": 90.0,
                "render_quality": "medium",
            },
            "export": {
                "default_directory": "",
                "svg_unit": "mm",
                "dxf_unit": "mm",
                "fold_spec_version": "1.1",
                "include_metadata": True,
                "precision": 4,
            },
            "recent_files": [],
            "max_recent_files": 10,
            "language": "zh_CN",
```

```
            "theme": "light",
            "log_level": "INFO",
        }
    @property
    def config_path(self) -> str:
        return self._config_path
    @property
    def config_dir(self) -> str:
        return self._config_dir
    def load(self) -> None:
        if not os.path.exists(self._config_path):
            logger.info("配置文件不存在，使用默认配置: %s", self._config_path)
            return
        try:
            with open(self._config_path, "r", encoding="utf-8") as fp:
                loaded = json.load(fp)
            if isinstance(loaded, dict):
                self._data = self._merge(self._defaults, loaded)
                logger.info("配置加载成功: %s", self._config_path)
            else:
                logger.warning("配置文件内容非法，回退默认配置")
        except (OSError, json.JSONDecodeError) as exc:
            logger.error("配置加载失败: %s", exc)
    def save(self) -> bool:
        try:
            os.makedirs(self._config_dir, exist_ok=True)
            tmp_path = self._config_path + ".tmp"
            with open(tmp_path, "w", encoding="utf-8") as fp:
                json.dump(self._data, fp, ensure_ascii=False, indent=2)
            os.replace(tmp_path, self._config_path)
            logger.info("配置保存成功: %s", self._config_path)
            return True
        except OSError as exc:
            logger.error("配置保存失败: %s", exc)
            return False
    def reset(self) -> None:
        self._data = dict(self._defaults)
        logger.info("配置已重置为默认值")
    def get(self, key: str, default: Any = None) -> Any:
        node: Any = self._data
        for part in key.split("."):
            if isinstance(node, dict) and part in node:
                node = node[part]
            else:
                return default
        return node
    def set(self, key: str, value: Any) -> None:
        parts = key.split(".")
        node = self._data
        for part in parts[:-1]:
            if part not in node or not isinstance(node[part], dict):
                node[part] = {}
            node = node[part]
        node[parts[-1]] = value
    def update(self, values: Dict[str, Any]) -> None:
        for key, value in values.items():
            self.set(key, value)
    def as_dict(self) -> Dict[str, Any]:
        return json.loads(json.dumps(self._data))
    def add_recent_file(self, file_path: str) -> None:
        if not file_path:
            return
        normalized = os.path.abspath(file_path)
        recent = list(self._data.get("recent_files", []))
        if normalized in recent:
            recent.remove(normalized)
        recent.insert(0, normalized)
        limit = int(self._data.get("max_recent_files", 10))
        self._data["recent_files"] = recent[:limit]
    def recent_files(self) -> list:
        return list(self._data.get("recent_files", []))
    def clear_recent_files(self) -> None:
        self._data["recent_files"] = []
    @staticmethod
    def _merge(base: Dict[str, Any], override: Dict[str, Any]) -> Dict[str, Any]:
        result = dict(base)
        for key, value in override.items():
            if key in result and isinstance(result[key], dict) and isinstance(value, dict):
                result[key] = AppConfig._merge(result[key], value)
            else:
                result[key] = value
        return result
```

```
_config_instance: Optional[AppConfig] = None
def get_config() -> AppConfig:
    global _config_instance
    if _config_instance is None:
        _config_instance = AppConfig()
    return _config_instance
</final>
# 文件: app/__init__.py
from app.config import APP_NAME, APP_VERSION, ORG_NAME
from app.core.logger import get_logger
__all__ = ["APP_NAME", "APP_VERSION", "ORG_NAME", "get_logger", "init_application"]
_logger = get_logger(__name__)
def init_application() -> bool:
    try:
        from app.database import init_database
        init_database()
    except Exception as exc:
        _logger.error("应用初始化失败: %s", exc)
        return False
    _logger.info("%s v%s 初始化完成", APP_NAME, APP_VERSION)
    return True
</□□DSML□□ parameter>
</invoke>
# 文件: app/database.py
import os
from contextlib import contextmanager
from typing import Iterator, Optional
from sqlalchemy import create_engine, event, text
from sqlalchemy.engine import Engine
from sqlalchemy.exc import SQLAlchemyError
from sqlalchemy.orm import Session, scoped_session, sessionmaker
from app.config import config
from app.core.logger import get_logger
logger = get_logger(__name__)
_engine: Optional[Engine] = None
_session_factory: Optional[sessionmaker] = None
_scoped_session: Optional[scoped_session] = None
def _build_sqlite_url(db_path: str) -> str:
    normalized = os.path.abspath(db_path).replace("\\", "/")
    return f"sqlite:///{normalized}"
def _ensure_directory(db_path: str) -> None:
    directory = os.path.dirname(os.path.abspath(db_path))
    if directory and not os.path.isdir(directory):
        os.makedirs(directory, exist_ok=True)
@event.listens_for(Engine, "connect")
def _configure_sqlite_connection(dbapi_connection, connection_record) -> None:
    cursor = dbapi_connection.cursor()
    try:
        cursor.execute("PRAGMA foreign_keys=ON")
        cursor.execute("PRAGMA journal_mode=WAL")
        cursor.execute("PRAGMA synchronous=NORMAL")
        cursor.execute("PRAGMA busy_timeout=5000")
    finally:
        cursor.close()
def init_database(db_path: Optional[str] = None, echo: bool = False) -> Engine:
    global _engine, _session_factory, _scoped_session
    if _engine is not None:
        return _engine
    target_path = db_path or config.database_path
    _ensure_directory(target_path)
    url = _build_sqlite_url(target_path)
    try:
        _engine = create_engine(
            url,
            echo=echo,
            future=True,
            connect_args={"check_same_thread": False, "timeout": 15},
        )
        _session_factory = sessionmaker(
            bind=_engine,
            autoflush=False,
            autocommit=False,
            expire_on_commit=False,
            future=True,
        )
        _scoped_session = scoped_session(_session_factory)
        logger.info("数据库初始化完成: %s", target_path)
    except SQLAlchemyError as exc:
        logger.error("数据库初始化失败: %s", exc)
        raise
    return _engine
def get_engine() -> Engine:
    if _engine is None:
```

```
        return init_database()
    return _engine
def get_session_factory() -> sessionmaker:
    if _session_factory is None:
        init_database()
    assert _session_factory is not None
    return _session_factory
def get_scoped_session() -> scoped_session:
    if _scoped_session is None:
        init_database()
    assert _scoped_session is not None
    return _scoped_session
def create_session() -> Session:
    return get_session_factory()()
@contextmanager
def session_scope() -> Iterator[Session]:
    session = create_session()
    try:
        yield session
        session.commit()
    except SQLAlchemyError as exc:
        session.rollback()
        logger.error("数据库事务回滚: %s", exc)
        raise
    except Exception as exc:
        session.rollback()
        logger.error("会话异常回滚: %s", exc)
        raise
    finally:
        session.close()
def create_all_tables() -> None:
    from app.models import Base
    engine = get_engine()
    try:
        Base.metadata.create_all(engine)
        logger.info("数据表创建/校验完成，共 %d 张表", len(Base.metadata.tables))
    except SQLAlchemyError as exc:
        logger.error("数据表创建失败: %s", exc)
        raise
def drop_all_tables() -> None:
    from app.models import Base
    engine = get_engine()
    try:
        Base.metadata.drop_all(engine)
        logger.warning("已删除全部数据表")
    except SQLAlchemyError as exc:
        logger.error("数据表删除失败: %s", exc)
        raise
def check_connection() -> bool:
    engine = get_engine()
    try:
        with engine.connect() as conn:
            conn.execute(text("SELECT 1"))
        return True
    except SQLAlchemyError as exc:
        logger.error("数据库连接检测失败: %s", exc)
        return False
def vacuum_database() -> None:
    engine = get_engine()
    try:
        with engine.connect() as conn:
            conn.execute(text("VACUUM"))
        logger.info("数据库整理完成")
    except SQLAlchemyError as exc:
        logger.error("数据库整理失败: %s", exc)
        raise
def dispose_database() -> None:
    global _engine, _session_factory, _scoped_session
    if _scoped_session is not None:
        try:
            _scoped_session.remove()
        except SQLAlchemyError as exc:
            logger.warning("清理会话失败: %s", exc)
        _scoped_session = None
    if _engine is not None:
        try:
            _engine.dispose()
        except SQLAlchemyError as exc:
            logger.warning("释放引擎失败: %s", exc)
        _engine = None
    _session_factory = None
    logger.info("数据库资源已释放")
```

```
def get_database_info() -> dict:
    engine = get_engine()
    info = {
        "url": str(engine.url),
        "driver": engine.dialect.name,
        "path": config.database_path,
        "connected": check_connection(),
    }
    try:
        with engine.connect() as conn:
            result = conn.execute(text("PRAGMA page_count")).scalar()
            info["page_count"] = int(result) if result is not None else 0
    except SQLAlchemyError as exc:
        logger.warning("读取数据库信息失败: %s", exc)
        info["page_count"] = 0
    return info
# 文件: app/exceptions.py
from typing import Any, Dict, Optional
class CreaseSystemError(Exception):
    default_message = "折痕图系统发生未知错误"
    def __init__(self, message: Optional[str] = None, *, detail: Optional[Dict[str, Any]] = None) -> None:
        self.message = message or self.default_message
        self.detail: Dict[str, Any] = detail or {}
        super().__init__(self.message)
    def to_dict(self) -> Dict[str, Any]:
        return {
            "error_type": self.__class__.__name__,
            "message": self.message,
            "detail": self.detail,
        }
    def __str__(self) -> str:
        if self.detail:
            return f"{self.message} ({self.detail})"
        return self.message
class ProjectError(CreaseSystemError):
    default_message = "工程文件操作失败"
class ProjectNotFoundError(ProjectError):
    default_message = "指定的工程不存在"
    def __init__(self, project_id: Any, *, detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"project_id": project_id}
        if detail:
            merged.update(detail)
        super().__init__(f"工程 {project_id} 不存在", detail=merged)
class ProjectSaveError(ProjectError):
    default_message = "工程保存失败"
class ProjectLoadError(ProjectError):
    default_message = "工程加载失败"
class CreaseError(CreaseSystemError):
    default_message = "折痕操作失败"
class CreaseNotFoundError(CreaseError):
    default_message = "指定的折痕不存在"
    def __init__(self, crease_id: Any, *, detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"crease_id": crease_id}
        if detail:
            merged.update(detail)
        super().__init__(f"折痕 {crease_id} 不存在", detail=merged)
class InvalidCreaseTypeError(CreaseError):
    default_message = "非法的折痕类型"
    def __init__(self, crease_type: Any, allowed: Optional[Any] = None, *,
                 detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"crease_type": crease_type, "allowed": allowed}
        if detail:
            merged.update(detail)
        super().__init__(f"折痕类型 {crease_type!r} 不被支持", detail=merged)
class DegenerateSegmentError(CreaseError):
    default_message = "折痕线段退化，起止点重合"
    def __init__(self, start: Any, end: Any, *, detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"start": start, "end": end}
        if detail:
            merged.update(detail)
        super().__init__("折痕线段起止点重合，无法构成有效折痕", detail=merged)
class LayerError(CreaseSystemError):
    default_message = "图层操作失败"
class LayerNotFoundError(LayerError):
    default_message = "指定的图层不存在"
    def __init__(self, layer_id: Any, *, detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"layer_id": layer_id}
        if detail:
            merged.update(detail)
        super().__init__(f"图层 {layer_id} 不存在", detail=merged)
class LayerLockedError(LayerError):
    default_message = "图层已锁定，禁止修改"
```

```
    def __init__(self, layer_id: Any, *, detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"layer_id": layer_id}
        if detail:
            merged.update(detail)
        super().__init__(f"图层 {layer_id} 已锁定，无法执行修改操作", detail=merged)
class ValidationError(CreaseSystemError):
    default_message = "折痕图校验失败"
class GeometryError(ValidationError):
    default_message = "折痕图几何关系非法"
class TopologyError(ValidationError):
    default_message = "折痕图拓扑关系非法"
class FoldabilityError(ValidationError):
    default_message = "折痕图不满足可折叠条件"
class ExportError(CreaseSystemError):
    default_message = "折痕图导出失败"
class UnsupportedFormatError(ExportError):
    default_message = "不支持的导出格式"
    def __init__(self, fmt: Any, supported: Optional[Any] = None, *,
                 detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"format": fmt, "supported": supported}
        if detail:
            merged.update(detail)
        super().__init__(f"导出格式 {fmt!r} 不受支持", detail=merged)
class ExportWriteError(ExportError):
    default_message = "导出文件写入失败"
    def __init__(self, path: Any, reason: Optional[str] = None, *,
                 detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"path": str(path), "reason": reason}
        if detail:
            merged.update(detail)
        super().__init__(f"写入导出文件失败□{path}", detail=merged)
class ImportError_(CreaseSystemError):
    default_message = "折痕图导入失败"
class UnsupportedImportFormatError(ImportError_):
    default_message = "不支持的导入格式"
    def __init__(self, fmt: Any, *, detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"format": fmt}
        if detail:
            merged.update(detail)
        super().__init__(f"导入格式 {fmt!r} 不受支持", detail=merged)
class ImportParseError(ImportError_):
    default_message = "导入文件解析失败"
    def __init__(self, path: Any, reason: Optional[str] = None, *,
                 detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"path": str(path), "reason": reason}
        if detail:
            merged.update(detail)
        super().__init__(f"解析导入文件失败□{path}", detail=merged)
class DatabaseError(CreaseSystemError):
    default_message = "数据库操作失败"
class RecordNotFoundError(DatabaseError):
    default_message = "数据库记录不存在"
    def __init__(self, table: str, key: Any, *, detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"table": table, "key": key}
        if detail:
            merged.update(detail)
        super().__init__(f"表 {table} 中不存在键为 {key!r} 的记录", detail=merged)
class ConfigError(CreaseSystemError):
    default_message = "配置操作失败"
class ConfigKeyError(ConfigError):
    default_message = "配置项不存在"
    def __init__(self, key: str, *, detail: Optional[Dict[str, Any]] = None) -> None:
        merged = {"key": key}
        if detail:
            merged.update(detail)
        super().__init__(f"配置项 {key!r} 不存在", detail=merged)
class UndoStackError(CreaseSystemError):
    default_message = "撤销重做操作失败"
class EmptyUndoStackError(UndoStackError):
    default_message = "撤销栈为空，无可撤销操作"
class EmptyRedoStackError(UndoStackError):
    default_message = "重做栈为空，无可重做操作"
def wrap_exception(exc: Exception, *, context: Optional[str] = None) -> CreaseSystemError:
    if isinstance(exc, CreaseSystemError):
        if context:
            exc.detail.setdefault("context", context)
        return exc
    detail: Dict[str, Any] = {"origin_type": exc.__class__.__name__}
    if context:
        detail["context"] = context
    return CreaseSystemError(str(exc) or None, detail=detail)
</□□DSML□□ parameter>
```

```
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/models/__init__.py
from app.models.crease import Crease, CreaseType
from app.models.export_record import ExportRecord, ExportFormat
from app.models.layer import Layer
from app.models.project import Project
from app.models.validation import ValidationIssue, IssueSeverity, IssueCategory
__all__ = [
    "Crease",
    "CreaseType",
    "ExportRecord",
    "ExportFormat",
    "Layer",
    "Project",
    "ValidationIssue",
    "IssueSeverity",
    "IssueCategory",
]
# 文件: app/models/crease.py
from datetime import datetime
from typing import List, Optional
from sqlalchemy import ForeignKey, Index, String, Float, Integer, Boolean, DateTime, UniqueConstraint
from sqlalchemy.orm import Mapped, mapped_column, relationship
from app.common import BaseModel, logger
CREASE_TYPE_VALLEY = "valley"
CREASE_TYPE_MOUNTAIN = "mountain"
CREASE_TYPE_FLAT = "flat"
CREASE_TYPE_BOUNDARY = "boundary"
CREASE_TYPE_CUT = "cut"
CREASE_TYPE_AUXILIARY = "auxiliary"
VALID_CREASE_TYPES = (
    CREASE_TYPE_VALLEY,
    CREASE_TYPE_MOUNTAIN,
    CREASE_TYPE_FLAT,
    CREASE_TYPE_BOUNDARY,
    CREASE_TYPE_CUT,
    CREASE_TYPE_AUXILIARY,
)
CREASE_TYPE_LABELS = {
    CREASE_TYPE_VALLEY: "谷折线",
    CREASE_TYPE_MOUNTAIN: "峰折线",
    CREASE_TYPE_FLAT: "段折线",
    CREASE_TYPE_BOUNDARY: "边界线",
    CREASE_TYPE_CUT: "剪裁线",
    CREASE_TYPE_AUXILIARY: "辅助参考线",
}
class Vertex(BaseModel):
    __tablename__ = "vertex"
    __table_args__ = (
        Index("ix_vertex_project", "project_id"),
        Index("ix_vertex_position", "x", "y"),
    )
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    project_id: Mapped[int] = mapped_column(ForeignKey("project.id", ondelete="CASCADE"), nullable=False)
    layer_id: Mapped[Optional[int]] = mapped_column(ForeignKey("layer.id", ondelete="SET NULL"), nullable=True)
    name: Mapped[Optional[str]] = mapped_column(String(64), nullable=True)
    x: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
    y: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
    z: Mapped[float] = mapped_column(Float, nullable=False, default=0.0)
    is_locked: Mapped[bool] = mapped_column(Boolean, nullable=False, default=False)
    is_boundary: Mapped[bool] = mapped_column(Boolean, nullable=False, default=False)
    degree: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    created_at: Mapped[datetime] = mapped_column(DateTime, nullable=False, default=datetime.utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime, nullable=False, default=datetime.utcnow,
onupdate=datetime.utcnow)
    creases_as_start: Mapped[List["Crease"]] = relationship(
        "Crease",
        foreign_keys="Crease.start_vertex_id",
        back_populates="start_vertex",
        cascade="all, delete-orphan",
    )
    creases_as_end: Mapped[List["Crease"]] = relationship(
        "Crease",
        foreign_keys="Crease.end_vertex_id",
        back_populates="end_vertex",
        cascade="all, delete-orphan",
    )
    def move_to(self, x: float, y: float, z: Optional[float] = None) -> None:
        if self.is_locked:
            logger.warning("顶点 %s 已锁定，忽略移动操作", self.id)
            return
```

```
        self.x = float(x)
        self.y = float(y)
        if z is not None:
            self.z = float(z)
        self.updated_at = datetime.utcnow()
    def translate(self, dx: float, dy: float, dz: float = 0.0) -> None:
        if self.is_locked:
            return
        self.x += float(dx)
        self.y += float(dy)
        self.z += float(dz)
        self.updated_at = datetime.utcnow()
    def coordinates(self) -> tuple:
        return self.x, self.y, self.z
    def refresh_degree(self) -> int:
        self.degree = len(self.creases_as_start) + len(self.creases_as_end)
        return self.degree
    def __repr__(self) -> str:
        return f"<Vertex id={self.id} x={self.x:.3f} y={self.y:.3f}>"
class Crease(BaseModel):
    __tablename__ = "crease"
    __table_args__ = (
        Index("ix_crease_project", "project_id"),
        Index("ix_crease_type", "crease_type"),
        Index("ix_crease_layer", "layer_id"),
        UniqueConstraint("start_vertex_id", "end_vertex_id", "crease_type", name="uq_crease_edge_type"),
    )
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    project_id: Mapped[int] = mapped_column(ForeignKey("project.id", ondelete="CASCADE"), nullable=False)
    layer_id: Mapped[Optional[int]] = mapped_column(ForeignKey("layer.id", ondelete="SET NULL"), nullable=True)
    start_vertex_id: Mapped[int] = mapped_column(ForeignKey("vertex.id", ondelete="CASCADE"), nullable=False)
    end_vertex_id: Mapped[int] = mapped_column(ForeignKey("vertex.id", ondelete="CASCADE"), nullable=False)
    crease_type: Mapped[str] = mapped_column(String(16), nullable=False, default=CREASE_TYPE_VALLEY)
    label: Mapped[Optional[str]] = mapped_column(String(128), nullable=True)
    color: Mapped[str] = mapped_column(String(16), nullable=False, default="
    line_width: Mapped[float] = mapped_column(Float, nullable=False, default=1.0)
    line_style: Mapped[str] = mapped_column(String(16), nullable=False, default="solid")
    fold_angle: Mapped[float] = mapped_column(Float, nullable=False, default=180.0)
    is_selected: Mapped[bool] = mapped_column(Boolean, nullable=False, default=False)
    is_visible: Mapped[bool] = mapped_column(Boolean, nullable=False, default=True)
    order_index: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    created_at: Mapped[datetime] = mapped_column(DateTime, nullable=False, default=datetime.utcnow)
    updated_at: Mapped[datetime] = mapped_column(DateTime, nullable=False, default=datetime.utcnow,
onupdate=datetime.utcnow)
    start_vertex: Mapped["Vertex"] = relationship(
        "Vertex",
        foreign_keys=[start_vertex_id],
        back_populates="creases_as_start",
    )
    end_vertex: Mapped["Vertex"] = relationship(
        "Vertex",
        foreign_keys=[end_vertex_id],
        back_populates="creases_as_end",
    )
    def set_crease_type(self, crease_type: str) -> bool:
        if crease_type not in VALID_CREASE_TYPES:
            logger.error("非法的折痕类型: %s", crease_type)
            return False
        self.crease_type = crease_type
        self.updated_at = datetime.utcnow()
        return True
    def set_style(self, color: Optional[str] = None, line_width: Optional[float] = None,
                  line_style: Optional[str] = None) -> None:
        if color is not None:
            self.color = color
        if line_width is not None:
            self.line_width = float(line_width)
        if line_style is not None:
            self.line_style = line_style
        self.updated_at = datetime.utcnow()
    def set_fold_angle(self, angle: float) -> None:
        self.fold_angle = max(0.0, min(360.0, float(angle)))
        self.updated_at = datetime.utcnow()
    def length(self) -> float:
        if self.start_vertex is None or self.end_vertex is None:
            return 0.0
        dx = self.end_vertex.x - self.start_vertex.x
        dy = self.end_vertex.y - self.start_vertex.y
        return (dx * dx + dy * dy) ** 0.5
    def direction(self) -> tuple:
        if self.start_vertex is None or self.end_vertex is None:
            return 0.0, 0.0
        return self.end_vertex.x - self.start_vertex.x, self.end_vertex.y - self.start_vertex.y
```

```
    def midpoint(self) -> tuple:
        if self.start_vertex is None or self.end_vertex is None:
            return 0.0, 0.0
        return (self.start_vertex.x + self.end_vertex.x) / 2.0, (self.start_vertex.y + self.end_vertex.y) / 2.0
    def type_label(self) -> str:
        return CREASE_TYPE_LABELS.get(self.crease_type, self.crease_type)
    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "project_id": self.project_id,
            "layer_id": self.layer_id,
            "start_vertex_id": self.start_vertex_id,
            "end_vertex_id": self.end_vertex_id,
            "crease_type": self.crease_type,
            "label": self.label,
            "color": self.color,
            "line_width": self.line_width,
            "line_style": self.line_style,
            "fold_angle": self.fold_angle,
            "is_visible": self.is_visible,
            "order_index": self.order_index,
        }
    def __repr__(self) -> str:
        return f"<Crease id={self.id} type={self.crease_type} {self.start_vertex_id}->{self.end_vertex_id}>"
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/models/export_record.py
from datetime import datetime
from typing import Optional
from sqlalchemy import String, Integer, Float, DateTime, ForeignKey, Index, Text
from sqlalchemy.orm import Mapped, mapped_column, relationship
from app.common import BaseModel, logger
EXPORT_FORMAT_SVG = "SVG"
EXPORT_FORMAT_DXF = "DXF"
EXPORT_FORMAT_FOLD = "FOLD"
EXPORT_FORMAT_PDF = "PDF"
EXPORT_STATUS_PENDING = "pending"
EXPORT_STATUS_SUCCESS = "success"
EXPORT_STATUS_FAILED = "failed"
SUPPORTED_EXPORT_FORMATS = (
    EXPORT_FORMAT_SVG,
    EXPORT_FORMAT_DXF,
    EXPORT_FORMAT_FOLD,
    EXPORT_FORMAT_PDF,
)
class ExportRecord(BaseModel):
    __tablename__ = "export_record"
    __table_args__ = (
        Index("ix_export_record_project_id", "project_id"),
        Index("ix_export_record_format", "export_format"),
        Index("ix_export_record_created_at", "created_at"),
        Index("ix_export_record_status", "status"),
    )
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    project_id: Mapped[int] = mapped_column(
        Integer, ForeignKey("project.id", ondelete="CASCADE"), nullable=False
    )
    export_format: Mapped[str] = mapped_column(String(16), nullable=False)
    file_path: Mapped[str] = mapped_column(String(512), nullable=False)
    file_name: Mapped[str] = mapped_column(String(255), nullable=False)
    file_size: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    status: Mapped[str] = mapped_column(
        String(16), default=EXPORT_STATUS_PENDING, nullable=False
    )
    crease_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    vertex_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    layer_count: Mapped[int] = mapped_column(Integer, default=0, nullable=False)
    scale_factor: Mapped[float] = mapped_column(Float, default=1.0, nullable=False)
    unit: Mapped[str] = mapped_column(String(16), default="mm", nullable=False)
    duration_ms: Mapped[float] = mapped_column(Float, default=0.0, nullable=False)
    error_message: Mapped[Optional[str]] = mapped_column(Text, nullable=True)
    remark: Mapped[Optional[str]] = mapped_column(String(255), nullable=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime, default=datetime.now, nullable=False
    )
    finished_at: Mapped[Optional[datetime]] = mapped_column(DateTime, nullable=True)
    project: Mapped["Project"] = relationship(
        "Project", back_populates="export_records", lazy="joined"
    )
    def mark_success(self, file_size: int, duration_ms: float) -> None:
        self.status = EXPORT_STATUS_SUCCESS
        self.file_size = max(int(file_size), 0)
```

```
        self.duration_ms = max(float(duration_ms), 0.0)
        self.finished_at = datetime.now()
        self.error_message = None
        logger.info(
          "导出成功: project=%s format=%s file=%s",
            self.project_id,
            self.export_format,
            self.file_name,
        )
    def mark_failed(self, error_message: str, duration_ms: float = 0.0) -> None:
        self.status = EXPORT_STATUS_FAILED
        self.error_message = (error_message or "未知错误")[:2000]
        self.duration_ms = max(float(duration_ms), 0.0)
        self.finished_at = datetime.now()
        logger.warning(
          "导出失败: project=%s format=%s reason=%s",
            self.project_id,
            self.export_format,
            self.error_message,
        )
    def is_success(self) -> bool:
        return self.status == EXPORT_STATUS_SUCCESS
    def is_failed(self) -> bool:
        return self.status == EXPORT_STATUS_FAILED
    def is_pending(self) -> bool:
        return self.status == EXPORT_STATUS_PENDING
    def size_in_kb(self) -> float:
        return round(self.file_size / 1024.0, 2)
    def duration_in_seconds(self) -> float:
        return round(self.duration_ms / 1000.0, 3)
    def summary(self) -> str:
        return (
            f"[{self.export_format}] {self.file_name} "
            f"({self.size_in_kb()} KB, {self.duration_in_seconds()} s, {self.status})"
        )
    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "project_id": self.project_id,
            "export_format": self.export_format,
            "file_path": self.file_path,
            "file_name": self.file_name,
            "file_size": self.file_size,
            "status": self.status,
            "crease_count": self.crease_count,
            "vertex_count": self.vertex_count,
            "layer_count": self.layer_count,
            "scale_factor": self.scale_factor,
            "unit": self.unit,
            "duration_ms": self.duration_ms,
            "error_message": self.error_message,
            "remark": self.remark,
            "created_at": self.created_at.isoformat() if self.created_at else None,
            "finished_at": self.finished_at.isoformat() if self.finished_at else None,
        }
    @classmethod
    def build(
        cls,
        project_id: int,
        export_format: str,
        file_path: str,
        file_name: str,
        crease_count: int = 0,
        vertex_count: int = 0,
        layer_count: int = 0,
        scale_factor: float = 1.0,
        unit: str = "mm",
        remark: Optional[str] = None,
    ) -> "ExportRecord":
        normalized_format = (export_format or "").strip().upper()
        if normalized_format not in SUPPORTED_EXPORT_FORMATS:
            raise ValueError(f"不支持的导出格式: {export_format}")
        if not file_path:
            raise ValueError("导出文件路径不能为空")
        if not file_name:
            raise ValueError("导出文件名不能为空")
        record = cls(
            project_id=project_id,
            export_format=normalized_format,
            file_path=file_path,
            file_name=file_name,
            crease_count=max(int(crease_count), 0),
            vertex_count=max(int(vertex_count), 0),
```

```
            layer_count=max(int(layer_count), 0),
            scale_factor=float(scale_factor) if scale_factor else 1.0,
            unit=unit or "mm",
            remark=remark,
            status=EXPORT_STATUS_PENDING,
        )
        return record
    def __repr__(self) -> str:
        return (
            f"<ExportRecord id={self.id} project={self.project_id} "
            f"format={self.export_format} status={self.status}>"
        )
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/core/__init__.py
from app.core.constraints import (
    ConstraintResult,
    ConstraintSeverity,
    ConstraintType,
    CreaseConstraintChecker,
)
from app.core.geometry import (
    BoundingBox,
    GeometryUtils,
    Point2D,
    Segment2D,
    Vector2D,
)
from app.core.logger import get_logger, setup_logging
from app.core.settings import (
    AppSettings,
    CanvasSettings,
    ExportSettings,
    RenderSettings,
    ValidationSettings,
    get_settings,
    reload_settings,
)
from app.core.undo_stack import (
    Command,
    CommandType,
    UndoStack,
)
__all__ = [
    "ConstraintResult",
    "ConstraintSeverity",
    "ConstraintType",
    "CreaseConstraintChecker",
    "BoundingBox",
    "GeometryUtils",
    "Point2D",
    "Segment2D",
    "Vector2D",
    "get_logger",
    "setup_logging",
    "AppSettings",
    "CanvasSettings",
    "ExportSettings",
    "RenderSettings",
    "ValidationSettings",
    "get_settings",
    "reload_settings",
    "Command",
    "CommandType",
    "UndoStack",
]
def init_core() -> AppSettings:
    settings = get_settings()
    setup_logging(settings.log_level, settings.log_file)
    logger = get_logger(__name__)
    logger.info("核心模块初始化完成，日志级别=%s", settings.log_level)
    return settings
def describe_core() -> dict:
    return {
        "geometry": ["Point2D", "Vector2D", "Segment2D", "BoundingBox", "GeometryUtils"],
        "constraints": ["CreaseConstraintChecker", "ConstraintResult", "ConstraintType"],
        "undo": ["UndoStack", "Command", "CommandType"],
        "settings": ["AppSettings", "CanvasSettings", "RenderSettings", "ExportSettings"],
        "logging": ["setup_logging", "get_logger"],
    }
def get_core_version() -> str:
    return "1.0.0"
```

```
def get_core_author() -> str:
    return "折痕图可视化编辑与标准化导出系统"
def get_core_license() -> str:
    return "© 2026 折痕图可视化编辑与标准化导出系统. All rights reserved."
def get_core_info() -> dict:
    return {
        "version": get_core_version(),
        "author": get_core_author(),
        "license": get_core_license(),
        "capabilities": describe_core(),
    }
# 文件: app/core/constraints.py
import math
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Sequence, Tuple
from app.common import logger
from app.exceptions import GeometryError
EPSILON = 1e-7
ANGLE_TOLERANCE = 1e-4
KAWASAKI_TOLERANCE = 1e-3
@dataclass
class ConstraintIssue:
    code: str
    level: str
    message: str
    vertex_index: Optional[int] = None
    edge_index: Optional[int] = None
    detail: Dict[str, float] = field(default_factory=dict)
@dataclass
class VertexData:
    index: int
    x: float
    y: float
    incident_edges: List[int] = field(default_factory=list)
@dataclass
class EdgeData:
    index: int
    start: int
    end: int
    crease_type: str = "valley"
def _distance(ax: float, ay: float, bx: float, by: float) -> float:
    return math.hypot(bx - ax, by - ay)
def _angle_between(ax: float, ay: float, bx: float, by: float) -> float:
    dot = ax * bx + ay * by
    na = math.hypot(ax, ay)
    nb = math.hypot(bx, by)
    if na < EPSILON or nb < EPSILON:
        return 0.0
    cos_value = max(-1.0, min(1.0, dot / (na * nb)))
    return math.acos(cos_value)
def _orientation(ax: float, ay: float, bx: float, by: float, cx: float, cy: float) -> float:
    return (bx - ax) * (cy - ay) - (by - ay) * (cx - ax)
def _segments_intersect(
    p1: Tuple[float, float],
    p2: Tuple[float, float],
    p3: Tuple[float, float],
    p4: Tuple[float, float],
) -> bool:
    d1 = _orientation(p3[0], p3[1], p4[0], p4[1], p1[0], p1[1])
    d2 = _orientation(p3[0], p3[1], p4[0], p4[1], p2[0], p2[1])
    d3 = _orientation(p1[0], p1[1], p2[0], p2[1], p3[0], p3[1])
    d4 = _orientation(p1[0], p1[1], p2[0], p2[1], p4[0], p4[1])
    if ((d1 > EPSILON and d2 < -EPSILON) or (d1 < -EPSILON and d2 > EPSILON)) and (
        (d3 > EPSILON and d4 < -EPSILON) or (d3 < -EPSILON and d4 > EPSILON)
    ):
        return True
    return False
class CreaseConstraintChecker:
    def __init__(self, vertices: Sequence[Tuple[float, float]], edges: Sequence[EdgeData]):
        self.vertices: List[VertexData] = []
        for idx, (vx, vy) in enumerate(vertices):
            self.vertices.append(VertexData(index=idx, x=float(vx), y=float(vy)))
        self.edges: List[EdgeData] = list(edges)
        self._adjacency: Dict[int, List[int]] = {v.index: [] for v in self.vertices}
        for edge in self.edges:
            if edge.start in self._adjacency:
                self._adjacency[edge.start].append(edge.index)
            if edge.end in self._adjacency:
                self._adjacency[edge.end].append(edge.index)
    def _edge_points(self, edge: EdgeData) -> Tuple[Tuple[float, float], Tuple[float, float]]:
        a = self.vertices[edge.start]
        b = self.vertices[edge.end]
        return (a.x, a.y), (b.x, b.y)
```

```
    def check_duplicate_vertices(self) -> List[ConstraintIssue]:
        issues: List[ConstraintIssue] = []
        count = len(self.vertices)
        for i in range(count):
            for j in range(i + 1, count):
                vi = self.vertices[i]
                vj = self.vertices[j]
                if _distance(vi.x, vi.y, vj.x, vj.y) < EPSILON:
                    issues.append(
                        ConstraintIssue(
                            code="DUPLICATE_VERTEX",
                            level="warning",
                            message=f"顶点 {i} 与顶点 {j} 坐标重合",
                            vertex_index=j,
                            detail={"source": float(i)},
                        )
                    )
        return issues
    def check_degenerate_edges(self) -> List[ConstraintIssue]:
        issues: List[ConstraintIssue] = []
        for edge in self.edges:
            a = self.vertices[edge.start]
            b = self.vertices[edge.end]
            if _distance(a.x, a.y, b.x, b.y) < EPSILON:
                issues.append(
                    ConstraintIssue(
                        code="DEGENERATE_EDGE",
                        level="error",
                        message=f"折痕 {edge.index} 长度为零",
                        edge_index=edge.index,
                    )
                )
        return issues
    def check_edge_crossings(self) -> List[ConstraintIssue]:
        issues: List[ConstraintIssue] = []
        count = len(self.edges)
        for i in range(count):
            for j in range(i + 1, count):
                ei = self.edges[i]
                ej = self.edges[j]
                if len({ei.start, ei.end} & {ej.start, ej.end}) > 0:
                    continue
                p1, p2 = self._edge_points(ei)
                p3, p4 = self._edge_points(ej)
                if _segments_intersect(p1, p2, p3, p4):
                    issues.append(
                        ConstraintIssue(
                            code="EDGE_CROSSING",
                            level="error",
                            message=f"折痕 {ei.index} 与折痕 {ej.index} 交叉",
                            edge_index=ej.index,
                            detail={"other": float(ei.index)},
                        )
                    )
        return issues
    def _vertex_angles(self, vertex_index: int) -> List[float]:
        vertex = self.vertices[vertex_index]
        directions: List[Tuple[float, float]] = []
        for edge_index in self._adjacency.get(vertex_index, []):
            edge = self.edges[edge_index]
            other_index = edge.end if edge.start == vertex_index else edge.start
            other = self.vertices[other_index]
            dx = other.x - vertex.x
            dy = other.y - vertex.y
            length = math.hypot(dx, dy)
            if length < EPSILON:
                continue
            directions.append((dx / length, dy / length))
        if len(directions) < 2:
            return []
        angles = [math.atan2(dy, dx) for dx, dy in directions]
        angles.sort()
        sectors: List[float] = []
        for k in range(len(angles)):
            nxt = angles[(k + 1) % len(angles)]
            delta = nxt - angles[k]
            if delta <= 0:
                delta += 2 * math.pi
            sectors.append(delta)
        return sectors
    def check_vertex_degree(self) -> List[ConstraintIssue]:
        issues: List[ConstraintIssue] = []
        for vertex in self.vertices:
```

```
            degree = len(self._adjacency.get(vertex.index, []))
            if degree == 1:
                issues.append(
                    ConstraintIssue(
                        code="DANGLING_VERTEX",
                        level="warning",
                        message=f"顶点 {vertex.index} 仅连接一条折痕",
                        vertex_index=vertex.index,
                    )
                )
            elif degree == 2:
                sectors = self._vertex_angles(vertex.index)
                if sectors and abs(sectors[0] - math.pi) > ANGLE_TOLERANCE:
                    issues.append(
                        ConstraintIssue(
                            code="BENT_VERTEX",
                            level="warning",
                            message=f"顶点 {vertex.index} 处折痕不共线",
                            vertex_index=vertex.index,
                        )
                    )
        return issues
    def check_kawasaki(self) -> List[ConstraintIssue]:
        issues: List[ConstraintIssue] = []
        for vertex in self.vertices:
            degree = len(self._adjacency.get(vertex.index, []))
            if degree < 4 or degree % 2 != 0:
                continue
            sectors = self._vertex_angles(vertex.index)
            if len(sectors) < 4:
                continue
            alternating = 0.0
            for k in range(0, len(sectors), 2):
                alternating += sectors[k]
            for k in range(1, len(sectors), 2):
                alternating -= sectors[k]
            if abs(alternating) > KAWASAKI_TOLERANCE:
                issues.append(
                    ConstraintIssue(
                        code="KAWASAKI_VIOLATION",
                        level="error",
                        message=f"顶点 {vertex.index} 不满足川崎定理",
                        vertex_index=vertex.index,
                        detail={"deviation": alternating},
                    )
                )
        return issues
    def check_maekawa(self) -> List[ConstraintIssue]:
        issues: List[ConstraintIssue] = []
        for vertex in self.vertices:
            incident = self._adjacency.get(vertex.index, [])
            if len(incident) < 4:
                continue
            mountain = 0
            valley = 0
            for edge_index in incident:
                crease_type = self.edges[edge_index].crease_type
                if crease_type == "mountain":
                    mountain += 1
                elif crease_type == "valley":
                    valley += 1
            if mountain + valley < 4:
                continue
            if abs(mountain - valley) != 2:
                issues.append(
                    ConstraintIssue(
                        code="MAEKAWA_VIOLATION",
                        level="error",
                        message=f"顶点 {vertex.index} 峰谷数量差不为 2",
                        vertex_index=vertex.index,
                        detail={"mountain": float(mountain), "valley": float(valley)},
                    )
                )
        return issues
    def check_flat_foldable(self) -> List[ConstraintIssue]:
        issues: List[ConstraintIssue] = []
        for vertex in self.vertices:
            degree = len(self._adjacency.get(vertex.index, []))
            if degree == 0:
                continue
            if degree % 2 != 0:
                issues.append(
                    ConstraintIssue(
```

```
                        code="ODD_DEGREE",
                        level="error",
                        message=f"顶点 {vertex.index} 度数为奇数，无法平面折叠",
                        vertex_index=vertex.index,
                        detail={"degree": float(degree)},
                    )
                )
        return issues
    def check_boundary_closure(self) -> List[ConstraintIssue]:
        issues: List[ConstraintIssue] = []
        for vertex in self.vertices:
            degree = len(self._adjacency.get(vertex.index, []))
            if degree == 0:
                issues.append(
                    ConstraintIssue(
                        code="ISOLATED_VERTEX",
                        level="warning",
                        message=f"顶点 {vertex.index} 未连接任何折痕",
                        vertex_index=vertex.index,
                    )
                )
        return issues
    def run_all(self) -> List[ConstraintIssue]:
        issues: List[ConstraintIssue] = []
        try:
            issues.extend(self.check_duplicate_vertices())
            issues.extend(self.check_degenerate_edges())
            issues.extend(self.check_edge_crossings())
            issues.extend(self.check_vertex_degree())
            issues.extend(self.check_kawasaki())
            issues.extend(self.check_maekawa())
            issues.extend(self.check_flat_foldable())
            issues.extend(self.check_boundary_closure())
        except GeometryError as exc:
            logger.error("几何约束校验失败: %s", exc)
            raise
        except Exception as exc:
            logger.exception("约束校验出现未预期异常")
            raise GeometryError(f"约束校验异常: {exc}") from exc
        logger.info("折痕图约束校验完成，共发现 %d 项问题", len(issues))
        return issues
    def summary(self) -> Dict[str, int]:
        issues = self.run_all()
        result = {"error": 0, "warning": 0, "info": 0}
        for issue in issues:
            if issue.level in result:
                result[issue.level] += 1
        return result
def build_checker_from_creases(
    vertices: Sequence[Tuple[float, float]],
    creases: Sequence[Dict[str, object]],
) -> CreaseConstraintChecker:
    edges: List[EdgeData] = []
    for idx, crease in enumerate(creases):
        start = int(crease.get("start", 0))
        end = int(crease.get("end", 0))
        crease_type = str(crease.get("crease_type", "valley"))
        edges.append(EdgeData(index=idx, start=start, end=end, crease_type=crease_type))
    return CreaseConstraintChecker(vertices, edges)
def validate_crease_pattern(
    vertices: Sequence[Tuple[float, float]],
    creases: Sequence[Dict[str, object]],
) -> List[ConstraintIssue]:
    checker = build_checker_from_creases(vertices, creases)
    return checker.run_all()
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/core/geometry.py
import math
from typing import Iterable, List, Optional, Sequence, Tuple
from app.common import logger
from app.exceptions import GeometryError
EPSILON = 1e-9
ANGLE_EPSILON = 1e-6
class Point:
    __slots__ = ("x", "y")
    def __init__(self, x: float, y: float):
        self.x = float(x)
        self.y = float(y)
    def as_tuple(self) -> Tuple[float, float]:
        return (self.x, self.y)
```

```
    def distance_to(self, other: "Point") -> float:
        return math.hypot(self.x - other.x, self.y - other.y)
    def __eq__(self, other: object) -> bool:
        if not isinstance(other, Point):
            return NotImplemented
        return abs(self.x - other.x) < EPSILON and abs(self.y - other.y) < EPSILON
    def __hash__(self) -> int:
        return hash((round(self.x, 6), round(self.y, 6)))
    def __repr__(self) -> str:
        return f"Point({self.x:.4f}, {self.y:.4f})"
class Segment:
    __slots__ = ("start", "end")
    def __init__(self, start: Point, end: Point):
        self.start = start
        self.end = end
    @property
    def length(self) -> float:
        return self.start.distance_to(self.end)
    @property
    def direction(self) -> Tuple[float, float]:
        dx = self.end.x - self.start.x
        dy = self.end.y - self.start.y
        norm = math.hypot(dx, dy)
        if norm < EPSILON:
            return (0.0, 0.0)
        return (dx / norm, dy / norm)
    def midpoint(self) -> Point:
        return Point((self.start.x + self.end.x) / 2.0, (self.start.y + self.end.y) / 2.0)
    def angle(self) -> float:
        return math.atan2(self.end.y - self.start.y, self.end.x - self.start.x)
    def __repr__(self) -> str:
        return f"Segment({self.start!r} -> {self.end!r})"
def cross_product(ax: float, ay: float, bx: float, by: float) -> float:
    return ax * by - ay * bx
def orientation(p: Point, q: Point, r: Point) -> float:
    return cross_product(q.x - p.x, q.y - p.y, r.x - p.x, r.y - p.y)
def point_on_segment(point: Point, seg: Segment, tolerance: float = EPSILON) -> bool:
    if abs(orientation(seg.start, seg.end, point)) > tolerance:
        return False
    min_x = min(seg.start.x, seg.end.x) - tolerance
    max_x = max(seg.start.x, seg.end.x) + tolerance
    min_y = min(seg.start.y, seg.end.y) - tolerance
    max_y = max(seg.start.y, seg.end.y) + tolerance
    return min_x <= point.x <= max_x and min_y <= point.y <= max_y
def segments_intersect(a: Segment, b: Segment, tolerance: float = EPSILON) -> Optional[Point]:
    d1 = orientation(b.start, b.end, a.start)
    d2 = orientation(b.start, b.end, a.end)
    d3 = orientation(a.start, a.end, b.start)
    d4 = orientation(a.start, a.end, b.end)
    if ((d1 > tolerance and d2 < -tolerance) or (d1 < -tolerance and d2 > tolerance)) and \
       ((d3 > tolerance and d4 < -tolerance) or (d3 < -tolerance and d4 > tolerance)):
        return _line_intersection(a, b)
    if abs(d1) <= tolerance and point_on_segment(a.start, b, tolerance):
        return Point(a.start.x, a.start.y)
    if abs(d2) <= tolerance and point_on_segment(a.end, b, tolerance):
        return Point(a.end.x, a.end.y)
    if abs(d3) <= tolerance and point_on_segment(b.start, a, tolerance):
        return Point(b.start.x, b.start.y)
    if abs(d4) <= tolerance and point_on_segment(b.end, a, tolerance):
        return Point(b.end.x, b.end.y)
    return None
def _line_intersection(a: Segment, b: Segment) -> Optional[Point]:
    x1, y1 = a.start.x, a.start.y
    x2, y2 = a.end.x, a.end.y
    x3, y3 = b.start.x, b.start.y
    x4, y4 = b.end.x, b.end.y
    denom = (x1 - x2) * (y3 - y4) - (y1 - y2) * (x3 - x4)
    if abs(denom) < EPSILON:
        return None
    px = ((x1 * y2 - y1 * x2) * (x3 - x4) - (x1 - x2) * (x3 * y4 - y3 * x4)) / denom
    py = ((x1 * y2 - y1 * x2) * (y3 - y4) - (y1 - y2) * (x3 * y4 - y3 * x4)) / denom
    return Point(px, py)
def angle_between(v1: Tuple[float, float], v2: Tuple[float, float]) -> float:
    n1 = math.hypot(v1[0], v1[1])
    n2 = math.hypot(v2[0], v2[1])
    if n1 < EPSILON or n2 < EPSILON:
        return 0.0
    cos_value = (v1[0] * v2[0] + v1[1] * v2[1]) / (n1 * n2)
    cos_value = max(-1.0, min(1.0, cos_value))
    return math.degrees(math.acos(cos_value))
def vertex_angles(center: Point, neighbors: Sequence[Point]) -> List[float]:
    if len(neighbors) < 2:
        return []
```

```
    vectors = [(p.x - center.x, p.y - center.y) for p in neighbors]
    vectors.sort(key=lambda v: math.atan2(v[1], v[0]))
    angles: List[float] = []
    count = len(vectors)
    for i in range(count):
        v1 = vectors[i]
        v2 = vectors[(i + 1) % count]
        angles.append(angle_between(v1, v2))
    return angles
def polygon_area(points: Sequence[Point]) -> float:
    if len(points) < 3:
        return 0.0
    total = 0.0
    count = len(points)
    for i in range(count):
        p1 = points[i]
        p2 = points[(i + 1) % count]
        total += p1.x * p2.y - p2.x * p1.y
    return total / 2.0
def polygon_centroid(points: Sequence[Point]) -> Point:
    area = polygon_area(points)
    if abs(area) < EPSILON:
        if not points:
            return Point(0.0, 0.0)
        sx = sum(p.x for p in points) / len(points)
        sy = sum(p.y for p in points) / len(points)
        return Point(sx, sy)
    cx = 0.0
    cy = 0.0
    count = len(points)
    for i in range(count):
        p1 = points[i]
        p2 = points[(i + 1) % count]
        factor = p1.x * p2.y - p2.x * p1.y
        cx += (p1.x + p2.x) * factor
        cy += (p1.y + p2.y) * factor
    return Point(cx / (6.0 * area), cy / (6.0 * area))
def point_in_polygon(point: Point, polygon: Sequence[Point]) -> bool:
    if len(polygon) < 3:
        return False
    inside = False
    count = len(polygon)
    j = count - 1
    for i in range(count):
        pi = polygon[i]
        pj = polygon[j]
        if ((pi.y > point.y) != (pj.y > point.y)):
            x_intersect = (pj.x - pi.x) * (point.y - pi.y) / (pj.y - pi.y + EPSILON) + pi.x
            if point.x < x_intersect:
                inside = not inside
        j = i
    return inside
def bounding_box(points: Iterable[Point]) -> Tuple[float, float, float, float]:
    xs: List[float] = []
    ys: List[float] = []
    for p in points:
        xs.append(p.x)
        ys.append(p.y)
    if not xs:
        return (0.0, 0.0, 0.0, 0.0)
    return (min(xs), min(ys), max(xs), max(ys))
def rotate_point(point: Point, origin: Point, angle_deg: float) -> Point:
    rad = math.radians(angle_deg)
    cos_a = math.cos(rad)
    sin_a = math.sin(rad)
    dx = point.x - origin.x
    dy = point.y - origin.y
    return Point(origin.x + dx * cos_a - dy * sin_a, origin.y + dx * sin_a + dy * cos_a)
def scale_point(point: Point, origin: Point, factor: float) -> Point:
    if factor <= 0:
        raise GeometryError("缩放因子必须为正数")
    return Point(origin.x + (point.x - origin.x) * factor, origin.y + (point.y - origin.y) * factor)
def snap_point(point: Point, grid_size: float) -> Point:
    if grid_size <= 0:
        return Point(point.x, point.y)
    return Point(round(point.x / grid_size) * grid_size, round(point.y / grid_size) * grid_size)
def merge_close_points(points: Sequence[Point], tolerance: float = 1e-4) -> List[Point]:
    merged: List[Point] = []
    for p in points:
        found = False
        for existing in merged:
            if existing.distance_to(p) <= tolerance:
                found = True
```

```
                break
        if not found:
            merged.append(Point(p.x, p.y))
    return merged
def is_flat_foldable(angles: Sequence[float], tolerance: float = 1.0) -> bool:
    if not angles:
        return True
    total = sum(angles)
    if abs(total - 360.0) > tolerance:
        return False
    even_sum = sum(angles[i] for i in range(0, len(angles), 2))
    odd_sum = sum(angles[i] for i in range(1, len(angles), 2))
    return abs(even_sum - odd_sum) <= tolerance
def kawasaki_condition(angles: Sequence[float], tolerance: float = 1.0) -> bool:
    if len(angles) < 4 or len(angles) % 2 != 0:
        return False
    even_sum = sum(angles[i] for i in range(0, len(angles), 2))
    odd_sum = sum(angles[i] for i in range(1, len(angles), 2))
    return abs(even_sum - odd_sum) <= tolerance
def maekawa_condition(peak_count: int, valley_count: int) -> bool:
    return abs((peak_count - valley_count)) == 2
def line_intersection_with_segment(line_start: Point, line_end: Point, seg: Segment) -> Optional[Point]:
    return segments_intersect(Segment(line_start, line_end), seg)
def project_point_on_segment(point: Point, seg: Segment) -> Point:
    dx = seg.end.x - seg.start.x
    dy = seg.end.y - seg.start.y
    length_sq = dx * dx + dy * dy
    if length_sq < EPSILON:
        return Point(seg.start.x, seg.start.y)
    t = ((point.x - seg.start.x) * dx + (point.y - seg.start.y) * dy) / length_sq
    t = max(0.0, min(1.0, t))
    return Point(seg.start.x + t * dx, seg.start.y + t * dy)
def distance_point_to_segment(point: Point, seg: Segment) -> float:
    projected = project_point_on_segment(point, seg)
    return point.distance_to(projected)
def normalize_vector(vector: Tuple[float, float]) -> Tuple[float, float]:
    norm = math.hypot(vector[0], vector[1])
    if norm < EPSILON:
        return (0.0, 0.0)
    return (vector[0] / norm, vector[1] / norm)
def perpendicular_vector(vector: Tuple[float, float]) -> Tuple[float, float]:
    return (-vector[1], vector[0])
def reflect_point(point: Point, axis_start: Point, axis_end: Point) -> Point:
    axis = Segment(axis_start, axis_end)
    projected = project_point_on_segment(point, axis)
    return Point(2.0 * projected.x - point.x, 2.0 * projected.y - point.y)
def validate_segment(seg: Segment) -> None:
    if seg.length < EPSILON:
        logger.warning("检测到零长度线段: %r", seg)
        raise GeometryError("线段长度为零，无法进行几何运算")
# 文件: app/core/logger.py
import logging
import logging.handlers
import os
import sys
from datetime import datetime
from typing import Optional
from app.config import config
LOG_FORMAT = "%(asctime)s [%(levelname)s] %(name)s - %(message)s"
DATE_FORMAT = "%Y-%m-%d %H:%M:%S"
MAX_BYTES = 5 * 1024 * 1024
BACKUP_COUNT = 5
_LOGGER_NAME = "crease_system"
_initialized = False
class _ContextFilter(logging.Filter):
    def __init__(self, app_name: str) -> None:
        super().__init__()
        self._app_name = app_name
    def filter(self, record: logging.LogRecord) -> bool:
        record.app_name = self._app_name
        record.pid = os.getpid()
        return True
def _ensure_log_dir(log_dir: str) -> str:
    try:
        os.makedirs(log_dir, exist_ok=True)
    except OSError as exc:
        fallback = os.path.join(os.path.expanduser("~"), "crease_system_logs")
        os.makedirs(fallback, exist_ok=True)
        sys.stderr.write(f"日志目录创建失败({exc})，回退至 {fallback}\n")
        return fallback
    return log_dir
def _build_file_handler(log_path: str, level: int) -> logging.Handler:
    handler = logging.handlers.RotatingFileHandler(
```

```
        log_path,
        maxBytes=MAX_BYTES,
        backupCount=BACKUP_COUNT,
        encoding="utf-8",
    )
    handler.setLevel(level)
    handler.setFormatter(logging.Formatter(LOG_FORMAT, DATE_FORMAT))
    return handler
def _build_console_handler(level: int) -> logging.Handler:
    handler = logging.StreamHandler(sys.stdout)
    handler.setLevel(level)
    handler.setFormatter(logging.Formatter(LOG_FORMAT, DATE_FORMAT))
    return handler
def _build_error_handler(log_path: str) -> logging.Handler:
    handler = logging.handlers.RotatingFileHandler(
        log_path,
        maxBytes=MAX_BYTES,
        backupCount=BACKUP_COUNT,
        encoding="utf-8",
    )
    handler.setLevel(logging.ERROR)
    handler.setFormatter(logging.Formatter(LOG_FORMAT, DATE_FORMAT))
    return handler
def setup_logger(
    log_dir: Optional[str] = None,
    level: Optional[int] = None,
    console: bool = True,
) -> logging.Logger:
    global _initialized
    logger = logging.getLogger(_LOGGER_NAME)
    resolved_level = level if level is not None else _resolve_level(config.get("log_level", "INFO"))
    logger.setLevel(resolved_level)
    logger.propagate = False
    if _initialized:
        for handler in logger.handlers:
            handler.setLevel(resolved_level)
        return logger
    target_dir = log_dir or config.get("log_dir") or _default_log_dir()
    target_dir = _ensure_log_dir(target_dir)
    context_filter = _ContextFilter(config.get("app_name", "折痕图可视化编辑与标准化导出系统"))
    logger.addFilter(context_filter)
    main_log = os.path.join(target_dir, "crease_system.log")
    error_log = os.path.join(target_dir, "crease_system_error.log")
    logger.addHandler(_build_file_handler(main_log, resolved_level))
    logger.addHandler(_build_error_handler(error_log))
    if console:
        logger.addHandler(_build_console_handler(resolved_level))
    _initialized = True
    logger.info("日志系统初始化完成，输出目录：%s", target_dir)
    return logger
def _resolve_level(level_name: str) -> int:
    if isinstance(level_name, int):
        return level_name
    return logging.getLevelName(str(level_name).upper()) if str(level_name).upper() in logging._nameToLevel else
logging.INFO
def _default_log_dir() -> str:
    base = os.path.join(os.path.expanduser("~"), ".crease_system", "logs")
    return base
def get_logger(name: Optional[str] = None) -> logging.Logger:
    if not _initialized:
        setup_logger()
    if name:
        return logging.getLogger(f"{_LOGGER_NAME}.{name}")
    return logging.getLogger(_LOGGER_NAME)
def log_exception(logger: logging.Logger, message: str, exc: BaseException) -> None:
    logger.error("%s | %s: %s", message, type(exc).__name__, exc, exc_info=True)
def log_operation(logger: logging.Logger, action: str, detail: str = "") -> None:
    if detail:
        logger.info("操作[%s] %s", action, detail)
    else:
        logger.info("操作[%s]", action)
def shutdown_logger() -> None:
    global _initialized
    logger = logging.getLogger(_LOGGER_NAME)
    for handler in list(logger.handlers):
        try:
            handler.flush()
            handler.close()
        except Exception:
            continue
        logger.removeHandler(handler)
    _initialized = False
def current_log_file() -> str:
```

```
    logger = logging.getLogger(_LOGGER_NAME)
    for handler in logger.handlers:
        if isinstance(handler, logging.handlers.RotatingFileHandler):
            return handler.baseFilename
    return ""
def export_log_snapshot(target_path: str) -> bool:
    source = current_log_file()
    if not source or not os.path.exists(source):
        return False
    try:
        with open(source, "r", encoding="utf-8", errors="ignore") as src:
            content = src.read()
        with open(target_path, "w", encoding="utf-8") as dst:
            dst.write(f"
            dst.write(content)
        return True
    except OSError:
        return False
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/core/settings.py
import json
from pathlib import Path
from typing import Any, Dict, Optional
from app.common import config, logger
from app.exceptions import SettingsError
DEFAULT_SETTINGS: Dict[str, Any] = {
    "appearance": {
        "canvas_background": "
        "grid_visible": True,
        "grid_size": 20,
        "grid_color": "
        "snap_to_grid": True,
        "snap_tolerance": 8,
        "show_vertices": True,
        "vertex_radius": 3,
    },
    "crease_style": {
        "mountain_color": "
        "valley_color": "
        "flat_color": "
        "cut_color": "
        "boundary_color": "
        "auxiliary_color": "
        "mountain_width": 2,
        "valley_width": 2,
        "flat_width": 1,
        "cut_width": 2,
        "boundary_width": 3,
        "auxiliary_width": 1,
        "mountain_dash": "solid",
        "valley_dash": "dash",
    },
    "editor": {
        "auto_validate": True,
        "validate_interval_ms": 800,
        "undo_depth": 100,
        "default_layer": "默认图层″,
        "auto_save": True,
        "auto_save_interval_min": 5,
    },
    "preview": {
        "show_3d_preview": True,
        "fold_angle_deg": 90.0,
        "preview_background": "
        "render_quality": "medium",
        "animation_enabled": True,
        "animation_duration_ms": 400,
    },
    "export": {
        "default_format": "fold",
        "svg_units": "mm",
        "dxf_units": "mm",
        "include_metadata": True,
        "precision": 4,
        "export_directory": "",
    },
    "validation": {
        "angle_tolerance_deg": 1.0,
        "min_face_area": 0.01,
        "check_overlap": True,
        "check_flat_foldable": True,
```

```
        "check_degree_parity": True,
    },
    "system": {
        "language": "zh_CN",
        "recent_files_limit": 10,
        "log_level": "INFO",
        "theme": "light",
    },
}
_SETTINGS_SECTIONS = tuple(DEFAULT_SETTINGS.keys())
class SettingsManager:
    def __init__(self, settings_path: Optional[Path] = None) -> None:
        self._path = Path(settings_path) if settings_path else self._default_path()
        self._data: Dict[str, Any] = {}
        self.load()
    @staticmethod
    def _default_path() -> Path:
        base = Path(config.get("data_dir", str(Path.home() / ".crease_editor")))
        return base / "settings.json"
    @property
    def path(self) -> Path:
        return self._path
    def load(self) -> None:
        self._data = self._deep_copy(DEFAULT_SETTINGS)
        if not self._path.exists():
            logger.info("设置文件不存在，使用默认设置: %s", self._path)
            return
        try:
            raw = self._path.read_text(encoding="utf-8")
            stored = json.loads(raw)
        except (OSError, json.JSONDecodeError) as exc:
            logger.warning("读取设置文件失败，回退默认设置: %s", exc)
            return
        if not isinstance(stored, dict):
            logger.warning("设置文件结构非法，回退默认设置")
            return
        self._merge(self._data, stored)
    def save(self) -> None:
        try:
            self._path.parent.mkdir(parents=True, exist_ok=True)
            self._path.write_text(
                json.dumps(self._data, ensure_ascii=False, indent=2),
                encoding="utf-8",
            )
            logger.info("设置已保存: %s", self._path)
        except OSError as exc:
            raise SettingsError(f"保存设置失败: {exc}") from exc
    def reset(self) -> None:
        self._data = self._deep_copy(DEFAULT_SETTINGS)
        logger.info("设置已重置为默认值")
    def get(self, section: str, key: str, default: Any = None) -> Any:
        if section not in self._data:
            return default
        return self._data[section].get(key, default)
    def set(self, section: str, key: str, value: Any) -> None:
        if section not in _SETTINGS_SECTIONS:
            raise SettingsError(f"未知设置分区: {section}")
        self._data.setdefault(section, {})[key] = value
    def section(self, section: str) -> Dict[str, Any]:
        return dict(self._data.get(section, {}))
    def update_section(self, section: str, values: Dict[str, Any]) -> None:
        if section not in _SETTINGS_SECTIONS:
            raise SettingsError(f"未知设置分区: {section}")
        self._data.setdefault(section, {}).update(values)
    def as_dict(self) -> Dict[str, Any]:
        return self._deep_copy(self._data)
    def apply_dict(self, values: Dict[str, Any]) -> None:
        if not isinstance(values, dict):
            raise SettingsError("设置数据必须为字典")
        self._merge(self._data, values)
    @staticmethod
    def _deep_copy(source: Dict[str, Any]) -> Dict[str, Any]:
        return json.loads(json.dumps(source))
    @classmethod
    def _merge(cls, target: Dict[str, Any], incoming: Dict[str, Any]) -> None:
        for key, value in incoming.items():
            if isinstance(value, dict) and isinstance(target.get(key), dict):
                cls._merge(target[key], value)
            else:
                target[key] = value
_settings_manager: Optional[SettingsManager] = None
def get_settings_manager() -> SettingsManager:
```

```
    global _settings_manager
    if _settings_manager is None:
        _settings_manager = SettingsManager()
    return _settings_manager
def get_setting(section: str, key: str, default: Any = None) -> Any:
    return get_settings_manager().get(section, key, default)
def set_setting(section: str, key: str, value: Any) -> None:
    get_settings_manager().set(section, key, value)
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/core/undo_stack.py
def __len__(self) -> int:
        return len(self._undo)
    def __bool__(self) -> bool:
        return bool(self._undo)
class CommandBuilder:
    def __init__(self, name: str, merge_key: Optional[str] = None) -> None:
        if not name:
            raise UndoStackError("命令名称不能为空")
        self._name = name
        self._merge_key = merge_key
        self._do_actions: List[Callable[[], Any]] = []
        self._undo_actions: List[Callable[[], Any]] = []
        self._payload: Dict[str, Any] = {}
    def add(
        self,
        do: Callable[[], Any],
        undo: Callable[[], Any],
        **payload: Any,
    ) -> "CommandBuilder":
        if not callable(do) or not callable(undo):
            raise UndoStackError("do/undo 必须为可调用对象")
        self._do_actions.append(do)
        self._undo_actions.append(undo)
        self._payload.update(payload)
        return self
    def set_payload(self, **payload: Any) -> "CommandBuilder":
        self._payload.update(payload)
        return self
    def build(self) -> Command:
        if not self._do_actions:
            raise UndoStackError("命令至少需要一个操作")
        do_actions = list(self._do_actions)
        undo_actions = list(self._undo_actions)
        def _do() -> Any:
            result: Any = None
            for action in do_actions:
                result = action()
            return result
        def _undo() -> Any:
            result: Any = None
            for action in reversed(undo_actions):
                result = action()
            return result
        return Command(
            name=self._name,
            do=_do,
            undo=_undo,
            payload=dict(self._payload),
            merge_key=self._merge_key,
        )
class TransactionContext:
    def __init__(self, stack: UndoStack, name: str, merge_key: Optional[str] = None) -> None:
        self._stack = stack
        self._builder = CommandBuilder(name, merge_key=merge_key)
        self._entered = False
    def __enter__(self) -> CommandBuilder:
        if self._entered:
            raise UndoStackError("事务上下文不可重复进入")
        self._entered = True
        return self._builder
    def __exit__(self, exc_type: Any, exc: Any, tb: Any) -> bool:
        if exc_type is not None:
            return False
        if self._builder._do_actions:
            self._stack.execute(self._builder.build())
        return False
def make_command(
    name: str,
    do: Callable[[], Any],
    undo: Callable[[], Any],
    merge_key: Optional[str] = None,
```

```
    **payload: Any,
) -> Command:
    if not callable(do) or not callable(undo):
        raise UndoStackError("do/undo 必须为可调用对象")
    return Command(
        name=name,
        do=do,
        undo=undo,
        payload=dict(payload),
        merge_key=merge_key,
    )
# 文件: app/business/__init__.py
from app.business.module_zhetuwenzengshanjianmo import (
    CreaseNode,
    CreaseSegment,
    CreasePatternModel,
    CreasePatternBuilder,
    build_pattern_from_segments,
    summarize_pattern,
)
from app.business.module_zhetuleixingbiaozhu import (
    CreaseType,
    CreaseTypeStyle,
    CreaseTypeRegistry,
    default_crease_registry,
    classify_segment,
    apply_type_style,
)
from app.business.module_jihejiansuoxiaoyan import (
    ValidationIssue,
    ValidationResult,
    GeometryValidator,
    validate_pattern,
    quick_check,
)
from app.business.module_erweisanweilianDong import (
    FoldState,
    FoldPreviewEngine,
    compute_fold_state,
    preview_vertices,
)
from app.business.module_gongchengwenjianGuanli import (
    ProjectFileInfo,
    ProjectFileManager,
    save_project_file,
    load_project_file,
)
from app.business.module_biaozhunhuadaochu import (
    ExportFormat,
    ExportRequest,
    ExportOutcome,
    StandardExporter,
    export_pattern,
    supported_formats,
)
__all__ = [
    "CreaseNode",
    "CreaseSegment",
    "CreasePatternModel",
    "CreasePatternBuilder",
    "build_pattern_from_segments",
    "summarize_pattern",
    "CreaseType",
    "CreaseTypeStyle",
    "CreaseTypeRegistry",
    "default_crease_registry",
    "classify_segment",
    "apply_type_style",
    "ValidationIssue",
    "ValidationResult",
    "GeometryValidator",
    "validate_pattern",
    "quick_check",
    "FoldState",
    "FoldPreviewEngine",
    "compute_fold_state",
    "preview_vertices",
    "ProjectFileInfo",
    "ProjectFileManager",
    "save_project_file",
    "load_project_file",
    "ExportFormat",
    "ExportRequest",
```

```
    "ExportOutcome",
    "StandardExporter",
    "export_pattern",
    "supported_formats",
]
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/business/module_biaozhunhuadaochu.py
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple
from PySide6.QtCore import Qt
from PySide6.QtWidgets import (
    QCheckBox,
    QComboBox,
    QDialog,
    QDialogButtonBox,
    QDoubleSpinBox,
    QFileDialog,
    QFormLayout,
    QGroupBox,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QListWidget,
    QListWidgetItem,
    QMessageBox,
    QPushButton,
    QSpinBox,
    QVBoxLayout,
    QWidget,
)
from app.common import logger
from app.exceptions import ExportError, ValidationError
from app.io.dxf_exporter import DxfExporter
from app.io.fold_exporter import FoldExporter
from app.io.svg_exporter import SvgExporter
from app.models.crease import Crease, CreaseType
from app.services.crease_service import CreaseService
from app.services.export_service import ExportService
from app.utils.file_utils import ensure_dir, normalize_path
from app.utils.unit_utils import UnitConverter
EXPORT_FORMATS: Dict[str, str] = {
    "FOLD": "fold",
    "SVG": "svg",
    "DXF": "dxf",
}
FORMAT_DESCRIPTIONS: Dict[str, str] = {
    "FOLD": "折纸标准交换格式，保留折痕类型与拓扑结构",
    "SVG": "矢量图形格式，适合排版与教学演示",
    "DXF": "CAD 交换格式，可对接数控加工与结构分析",
}
@dataclass
class ExportOptions:
    fmt: str = "FOLD"
    output_path: str = ""
    include_auxiliary: bool = False
    include_boundary: bool = True
    scale: float = 1.0
    unit: str = "mm"
    precision: int = 3
    layer_ids: List[int] = field(default_factory=list)
    crease_types: List[str] = field(default_factory=list)
    def validate(self) -> None:
        if self.fmt not in EXPORT_FORMATS:
            raise ValidationError(f"不支持的导出格式□{self.fmt}")
        if not self.output_path:
            raise ValidationError("导出路径不能为空")
        if self.scale <= 0:
            raise ValidationError("缩放比例必须大于 0")
        if self.precision < 0 or self.precision > 8:
            raise ValidationError("精度取值范围为 0~8")
@dataclass
class ExportResult:
    fmt: str
    output_path: str
    crease_count: int
    file_size: int
    success: bool
    message: str = ""
class StandardExportService:
    def __init__(self, crease_service: CreaseService, export_service: ExportService) -> None:
```

```
        self._crease_service = crease_service
        self._export_service = export_service
        self._exporters = {
            "FOLD": FoldExporter(),
            "SVG": SvgExporter(),
            "DXF": DxfExporter(),
        }
    def collect_creases(self, project_id: int, options: ExportOptions) -> List[Crease]:
        creases = self._crease_service.list_by_project(project_id)
        result: List[Crease] = []
        for crease in creases:
            if not options.include_auxiliary and crease.crease_type == CreaseType.AUXILIARY:
                continue
            if not options.include_boundary and crease.crease_type == CreaseType.BOUNDARY:
                continue
            if options.layer_ids and crease.layer_id not in options.layer_ids:
                continue
            if options.crease_types and crease.crease_type.value not in options.crease_types:
                continue
            result.append(crease)
        return result
    def build_payload(self, creases: List[Crease], options: ExportOptions) -> Dict:
        converter = UnitConverter(options.unit)
        nodes: List[Dict] = []
        edges: List[Dict] = []
        node_index: Dict[Tuple[float, float], int] = {}
        def node_id(x: float, y: float) -> int:
            key = (round(x, options.precision), round(y, options.precision))
            if key not in node_index:
                node_index[key] = len(nodes)
                nodes.append({
                    "id": node_index[key],
                    "x": round(converter.to_millimeter(x) * options.scale, options.precision),
                    "y": round(converter.to_millimeter(y) * options.scale, options.precision),
                })
            return node_index[key]
        for crease in creases:
            start = node_id(crease.start_x, crease.start_y)
            end = node_id(crease.end_x, crease.end_y)
            edges.append({
                "start": start,
                "end": end,
                "type": crease.crease_type.value,
                "layer": crease.layer_id,
                "color": crease.color,
            })
        return {
            "format": EXPORT_FORMATS[options.fmt],
            "unit": options.unit,
            "scale": options.scale,
            "nodes": nodes,
            "edges": edges,
        }
    def export(self, project_id: int, options: ExportOptions) -> ExportResult:
        options.validate()
        creases = self.collect_creases(project_id, options)
        if not creases:
            raise ExportError("当前筛选条件下没有可导出的折痕数据")
        payload = self.build_payload(creases, options)
        exporter = self._exporters[options.fmt]
        target = normalize_path(options.output_path)
        ensure_dir(target.rsplit("\\", 1)[0] if "\\" in target else target.rsplit("/", 1)[0])
        try:
            exporter.export(payload, target)
        except Exception as exc:
            logger.error("导出失败 fmt=%s path=%s err=%s", options.fmt, target, exc)
            raise ExportError(f"导出失败：{exc}") from exc
        file_size = self._export_service.file_size(target)
        self._export_service.record_export(
            project_id=project_id,
            fmt=options.fmt,
            output_path=target,
            crease_count=len(creases),
            file_size=file_size,
        )
        logger.info("导出完成 fmt=%s creases=%d path=%s", options.fmt, len(creases), target)
        return ExportResult(
            fmt=options.fmt,
            output_path=target,
            crease_count=len(creases),
            file_size=file_size,
            success=True,
            message="导出成功",
```

```
        )
    def preview_summary(self, project_id: int, options: ExportOptions) -> Dict[str, int]:
        creases = self.collect_creases(project_id, options)
        summary: Dict[str, int] = {"total": len(creases)}
        for crease in creases:
            key = crease.crease_type.value
            summary[key] = summary.get(key, 0) + 1
        return summary
class ExportDialog(QDialog):
    def __init__(self, service: StandardExportService, project_id: int, parent: Optional[QWidget] = None) ->
None:
        super().__init__(parent)
        self._service = service
        self._project_id = project_id
        self._result: Optional[ExportResult] = None
        self.setWindowTitle("标准化导出")
        self.setMinimumWidth(520)
        self._build_ui()
        self._refresh_summary()
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        format_box = QGroupBox("导出格式")
        format_layout = QVBoxLayout(format_box)
        self._format_combo = QComboBox()
        for name in EXPORT_FORMATS:
            self._format_combo.addItem(name)
        self._format_combo.currentTextChanged.connect(self._on_format_changed)
        format_layout.addWidget(self._format_combo)
        self._format_hint = QLabel(FORMAT_DESCRIPTIONS["FOLD"])
        self._format_hint.setWordWrap(True)
        format_layout.addWidget(self._format_hint)
        layout.addWidget(format_box)
        path_box = QGroupBox("输出路径")
        path_layout = QHBoxLayout(path_box)
        self._path_edit = QLineEdit()
        self._path_edit.setPlaceholderText("请选择导出文件路径")
        browse_btn = QPushButton("浏览")
        browse_btn.clicked.connect(self._choose_path)
        path_layout.addWidget(self._path_edit)
        path_layout.addWidget(browse_btn)
        layout.addWidget(path_box)
        param_box = QGroupBox("导出参数")
        param_layout = QFormLayout(param_box)
        self._scale_spin = QDoubleSpinBox()
        self._scale_spin.setRange(0.01, 100.0)
        self._scale_spin.setValue(1.0)
        self._scale_spin.setSingleStep(0.1)
        param_layout.addRow("缩放比例", self._scale_spin)
        self._unit_combo = QComboBox()
        self._unit_combo.addItems(["mm", "cm", "in"])
        param_layout.addRow("单位", self._unit_combo)
        self._precision_spin = QSpinBox()
        self._precision_spin.setRange(0, 8)
        self._precision_spin.setValue(3)
        param_layout.addRow("坐标精度", self._precision_spin)
        self._aux_check = QCheckBox("包含辅助参考线")
        param_layout.addRow("", self._aux_check)
        self._boundary_check = QCheckBox("包含边界线")
        self._boundary_check.setChecked(True)
        param_layout.addRow("", self._boundary_check)
        layout.addWidget(param_box)
        summary_box = QGroupBox("数据概览")
        summary_layout = QVBoxLayout(summary_box)
        self._summary_list = QListWidget()
        summary_layout.addWidget(self._summary_list)
        layout.addWidget(summary_box)
        buttons = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
        buttons.button(QDialogButtonBox.Ok).setText("导出")
        buttons.button(QDialogButtonBox.Cancel).setText("取消")
        buttons.accepted.connect(self._on_accept)
        buttons.rejected.connect(self.reject)
        layout.addWidget(buttons)
    def _on_format_changed(self, name: str) -> None:
        self._format_hint.setText(FORMAT_DESCRIPTIONS.get(name, ""))
        self._refresh_summary()
    def _choose_path(self) -> None:
        fmt = self._format_combo.currentText().lower()
        path, _ = QFileDialog.getSaveFileName(
            self, "选择导出文件", "", f"{self._format_combo.currentText()} 文件 (*.{fmt})"
        )
        if path:
            self._path_edit.setText(path)
```

```
    def _current_options(self) -> ExportOptions:
        return ExportOptions(
            fmt=self._format_combo.currentText(),
            output_path=self._path_edit.text().strip(),
            include_auxiliary=self._aux_check.isChecked(),
            include_boundary=self._boundary_check.isChecked(),
            scale=self._scale_spin.value(),
            unit=self._unit_combo.currentText(),
            precision=self._precision_spin.value(),
        )
    def _refresh_summary(self) -> None:
        self._summary_list.clear()
        try:
            summary = self._service.preview_summary(self._project_id, self._current_options())
        except Exception as exc:
            logger.warning("导出概览统计失败□%s", exc)
            return
        for key, value in summary.items():
            item = QListWidgetItem(f"{key}: {value}")
            item.setTextAlignment(Qt.AlignLeft | Qt.AlignVCenter)
            self._summary_list.addWidget if False else self._summary_list.addItem(item)
    def _on_accept(self) -> None:
        options = self._current_options()
        try:
            self._result = self._service.export(self._project_id, options)
        except (ValidationError, ExportError) as exc:
            QMessageBox.warning(self, "导出失败", str(exc))
            return
        QMessageBox.information(self, "导出完成", f"已导出 {self._result.crease_count} 条折痕")
        self.accept()
    def export_result(self) -> Optional[ExportResult]:
        return self._result
class StandardExportPanel(QWidget):
    def __init__(self, service: StandardExportService, project_id: int, parent: Optional[QWidget] = None) ->
None:
        super().__init__(parent)
        self._service = service
        self._project_id = project_id
        self._build_ui()
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        layout.addWidget(QLabel("标准化导出"))
        self._format_combo = QComboBox()
        for name in EXPORT_FORMATS:
            self._format_combo.addItem(name)
        layout.addWidget(self._format_combo)
        self._summary_label = QLabel("尚未统计")
        layout.addWidget(self._summary_label)
        btn_row = QHBoxLayout()
        refresh_btn = QPushButton("刷新概览")
        refresh_btn.clicked.connect(self.refresh_summary)
        export_btn = QPushButton("打开导出对话框")
        export_btn.clicked.connect(self.open_export_dialog)
        btn_row.addWidget(refresh_btn)
        btn_row.addWidget(export_btn)
        layout.addLayout(btn_row)
        layout.addStretch(1)
    def refresh_summary(self) -> None:
        options = ExportOptions(fmt=self._format_combo.currentText())
        try:
            summary = self._service.preview_summary(self._project_id, options)
        except Exception as exc:
            logger.warning("刷新导出概览失败□%s", exc)
            self._summary_label.setText("统计失败")
            return
        parts = [f"{k}={v}" for k, v in summary.items()]
        self._summary_label.setText("□".join(parts) if parts else "无数据")
    def open_export_dialog(self) -> Optional[ExportResult]:
        dialog = ExportDialog(self._service, self._project_id, self)
        if dialog.exec() == QDialog.Accepted:
            return dialog.export_result()
        return None
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/business/module_erweisanweilianDong.py
import math
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple
from PySide6.QtCore import QObject, Signal, Qt
from PySide6.QtGui import QColor, QPainter, QPen, QBrush, QPolygonF, QFont
from PySide6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QLabel, QSlider, QGroupBox,
```

```
QFormLayout, QCheckBox, QSplitter
from app.common import logger
from app.core.geometry import Point2D, distance_2d, rotate_point_2d
from app.models.crease import CreaseType
@dataclass
class FoldVertex:
    vertex_id: int
    x: float
    y: float
    z: float = 0.0
    fixed: bool = False
@dataclass
class FoldFace:
    face_id: int
    vertex_ids: List[int] = field(default_factory=list)
    normal: Tuple[float, float, float] = (0.0, 0.0, 1.0)
    folded: bool = False
@dataclass
class FoldState:
    vertices: Dict[int, FoldVertex] = field(default_factory=dict)
    faces: Dict[int, FoldFace] = field(default_factory=dict)
    fold_angle: float = 0.0
    fold_progress: float = 0.0
class FoldModel:
    def __init__(self) -> None:
        self._state = FoldState()
        self._crease_links: Dict[int, Tuple[int, int, CreaseType]] = {}
        self._next_vertex_id = 1
        self._next_face_id = 1
    @property
    def state(self) -> FoldState:
        return self._state
    def clear(self) -> None:
        self._state = FoldState()
        self._crease_links.clear()
        self._next_vertex_id = 1
        self._next_face_id = 1
    def add_vertex(self, x: float, y: float, fixed: bool = False) -> int:
        vid = self._next_vertex_id
        self._next_vertex_id += 1
        self._state.vertices[vid] = FoldVertex(vertex_id=vid, x=x, y=y, fixed=fixed)
        return vid
    def add_face(self, vertex_ids: List[int]) -> int:
        if len(vertex_ids) < 3:
            raise ValueError("面片至少需要三个顶点")
        for vid in vertex_ids:
            if vid not in self._state.vertices:
                raise KeyError(f"顶点 {vid} 不存在")
        fid = self._next_face_id
        self._next_face_id += 1
        self._state.faces[fid] = FoldFace(face_id=fid, vertex_ids=list(vertex_ids))
        return fid
    def link_crease(self, crease_id: int, v_start: int, v_end: int, crease_type: CreaseType) -> None:
        if v_start not in self._state.vertices or v_end not in self._state.vertices:
            raise KeyError("折痕端点顶点不存在")
        self._crease_links[crease_id] = (v_start, v_end, crease_type)
    def unlink_crease(self, crease_id: int) -> None:
        self._crease_links.pop(crease_id, None)
    def crease_count(self) -> int:
        return len(self._crease_links)
    def face_count(self) -> int:
        return len(self._state.faces)
    def vertex_count(self) -> int:
        return len(self._state.vertices)
    def apply_fold(self, progress: float) -> None:
        progress = max(0.0, min(1.0, progress))
        self._state.fold_progress = progress
        self._state.fold_angle = progress * math.pi
        for vid, vertex in self._state.vertices.items():
            if vertex.fixed:
                vertex.z = 0.0
                continue
            depth = self._vertex_fold_depth(vid)
            vertex.z = depth * math.sin(self._state.fold_angle)
    def _vertex_fold_depth(self, vertex_id: int) -> float:
        depth = 0.0
        for _, (v_start, v_end, crease_type) in self._crease_links.items():
            if vertex_id not in (v_start, v_end):
                continue
            sign = 1.0 if crease_type == CreaseType.MOUNTAIN else -1.0
            if crease_type == CreaseType.FLAT:
                sign = 0.0
            depth += sign * 0.5
```

```
        return depth
    def project(self, yaw: float, pitch: float, scale: float) -> Dict[int, Point2D]:
        projected: Dict[int, Point2D] = {}
        cy, sy = math.cos(yaw), math.sin(yaw)
        cp, sp = math.cos(pitch), math.sin(pitch)
        for vid, vertex in self._state.vertices.items():
            x1 = vertex.x * cy - vertex.z * sy
            z1 = vertex.x * sy + vertex.z * cy
            y1 = vertex.y * cp - z1 * sp
            projected[vid] = Point2D(x1 * scale, y1 * scale)
        return projected
    def bounding_box(self) -> Tuple[float, float, float, float]:
        if not self._state.vertices:
            return (0.0, 0.0, 0.0, 0.0)
        xs = [v.x for v in self._state.vertices.values()]
        ys = [v.y for v in self._state.vertices.values()]
        return (min(xs), min(ys), max(xs), max(ys))
class LinkageController(QObject):
    fold_progress_changed = Signal(float)
    view_angle_changed = Signal(float, float)
    model_rebuilt = Signal(int, int)
    def __init__(self, model: Optional[FoldModel] = None) -> None:
        super().__init__()
        self._model = model or FoldModel()
        self._yaw = math.radians(35.0)
        self._pitch = math.radians(25.0)
        self._scale = 1.0
        self._auto_sync = True
    @property
    def model(self) -> FoldModel:
        return self._model
    @property
    def yaw(self) -> float:
        return self._yaw
    @property
    def pitch(self) -> float:
        return self._pitch
    @property
    def scale(self) -> float:
        return self._scale
    @property
    def auto_sync(self) -> bool:
        return self._auto_sync
    def set_auto_sync(self, enabled: bool) -> None:
        self._auto_sync = enabled
    def set_view_angle(self, yaw_deg: float, pitch_deg: float) -> None:
        self._yaw = math.radians(yaw_deg)
        self._pitch = math.radians(max(-89.0, min(89.0, pitch_deg)))
        self.view_angle_changed.emit(yaw_deg, pitch_deg)
    def set_scale(self, scale: float) -> None:
        self._scale = max(0.1, min(10.0, scale))
    def set_fold_progress(self, progress: float) -> None:
        try:
            self._model.apply_fold(progress)
        except Exception as exc:
            logger.error("折叠进度更新失败: %s", exc)
            return
        self.fold_progress_changed.emit(progress)
    def rebuild_from_creases(self, creases: List[dict]) -> None:
        self._model.clear()
        vertex_map: Dict[Tuple[float, float], int] = {}
        try:
            for crease in creases:
                start = (round(float(crease["x1"]), 4), round(float(crease["y1"]), 4))
                end = (round(float(crease["x2"]), 4), round(float(crease["y2"]), 4))
                v_start = self._ensure_vertex(vertex_map, start)
                v_end = self._ensure_vertex(vertex_map, end)
                crease_type = crease.get("crease_type", CreaseType.VALLEY)
                self._model.link_crease(int(crease["id"]), v_start, v_end, crease_type)
        except (KeyError, TypeError, ValueError) as exc:
            logger.error("折痕数据解析失败: %s", exc)
            return
        self.model_rebuilt.emit(self._model.vertex_count(), self._model.crease_count())
    def _ensure_vertex(self, vertex_map: Dict[Tuple[float, float], int], key: Tuple[float, float]) -> int:
        if key in vertex_map:
            return vertex_map[key]
        vid = self._model.add_vertex(key[0], key[1])
        vertex_map[key] = vid
        return vid
    def projected_points(self) -> Dict[int, Point2D]:
        return self._model.project(self._yaw, self._pitch, self._scale)
    def statistics(self) -> Dict[str, float]:
        state = self._model.state
```

```
        zs = [v.z for v in state.vertices.values()]
        return {
            "vertex_count": float(self._model.vertex_count()),
            "face_count": float(self._model.face_count()),
            "crease_count": float(self._model.crease_count()),
            "fold_angle_deg": math.degrees(state.fold_angle),
            "max_height": max(zs) if zs else 0.0,
            "min_height": min(zs) if zs else 0.0,
        }
class FoldPreviewWidget(QWidget):
    def __init__(self, controller: LinkageController, parent: Optional[QWidget] = None) -> None:
        super().__init__(parent)
        self._controller = controller
        self._show_faces = True
        self._show_vertices = True
        self.setMinimumSize(360, 300)
        self.setAutoFillBackground(True)
        self._controller.fold_progress_changed.connect(self.update)
        self._controller.view_angle_changed.connect(lambda *_: self.update())
        self._controller.model_rebuilt.connect(lambda *_: self.update())
    def set_show_faces(self, enabled: bool) -> None:
        self._show_faces = enabled
        self.update()
    def set_show_vertices(self, enabled: bool) -> None:
        self._show_vertices = enabled
        self.update()
    def paintEvent(self, event) -> None:
        painter = QPainter(self)
        painter.setRenderHint(QPainter.Antialiasing, True)
        painter.fillRect(self.rect(), QColor(248, 249, 251))
        points = self._controller.projected_points()
        if not points:
            painter.setPen(QPen(QColor(150, 150, 150)))
            painter.setFont(QFont("Microsoft YaHei", 10))
            painter.drawText(self.rect(), Qt.AlignCenter, "暂无折痕数据")
            return
        self._draw_ground_grid(painter)
        center = self._view_center(points)
        painter.translate(center.x(), center.y())
        if self._show_faces:
            self._draw_faces(painter, points)
        self._draw_creases(painter, points)
        if self._show_vertices:
            self._draw_vertices(painter, points)
    def _view_center(self, points: Dict[int, Point2D]) -> Point2D:
        xs = [p.x for p in points.values()]
        ys = [p.y for p in points.values()]
        return Point2D(self.width() / 2.0 - (min(xs) + max(xs)) / 2.0,
                       self.height() / 2.0 - (min(ys) + max(ys)) / 2.0)
    def _draw_ground_grid(self, painter: QPainter) -> None:
        painter.setPen(QPen(QColor(226, 230, 236), 1, Qt.DashLine))
        step = 40
        for x in range(0, self.width(), step):
            painter.drawLine(x, 0, x, self.height())
        for y in range(0, self.height(), step):
            painter.drawLine(0, y, self.width(), y)
    def _draw_faces(self, painter: QPainter, points: Dict[int, Point2D]) -> None:
        state = self._controller.model.state
        for face in state.faces.values():
            polygon = QPolygonF()
            for vid in face.vertex_ids:
                pt = points.get(vid)
                if pt is None:
                    break
                polygon.append(pt.to_qpoint())
            if polygon.count() < 3:
                continue
            painter.setBrush(QBrush(QColor(120, 170, 220, 60)))
            painter.setPen(QPen(QColor(90, 130, 180), 1))
            painter.drawPolygon(polygon)
    def _draw_creases(self, painter: QPainter, points: Dict[int, Point2D]) -> None:
        state = self._controller.model.state
        for _, (v_start, v_end, crease_type) in self._controller.model._crease_links.items():
            p1 = points.get(v_start)
            p2 = points.get(v_end)
            if p1 is None or p2 is None:
                continue
            color, style = self._crease_style(crease_type)
            pen = QPen(color, 2)
            pen.setStyle(style)
            painter.setPen(pen)
            painter.drawLine(p1.to_qpoint(), p2.to_qpoint())
    def _crease_style(self, crease_type: CreaseType) -> Tuple[QColor, Qt.PenStyle]:
```

```
        if crease_type == CreaseType.MOUNTAIN:
            return QColor(200, 60, 60), Qt.SolidLine
        if crease_type == CreaseType.VALLEY:
            return QColor(60, 110, 200), Qt.DashLine
        if crease_type == CreaseType.CUT:
            return QColor(40, 40, 40), Qt.DotLine
        return QColor(120, 120, 120), Qt.DashDotLine
    def _draw_vertices(self, painter: QPainter, points: Dict[int, Point2D]) -> None:
        painter.setPen(QPen(QColor(70, 70, 70), 1))
        painter.setBrush(QBrush(QColor(255, 255, 255)))
        for pt in points.values():
            painter.drawEllipse(pt.to_qpoint(), 3.0, 3.0)
class LinkagePanel(QWidget):
    def __init__(self, controller: LinkageController, parent: Optional[QWidget] = None) -> None:
        super().__init__(parent)
        self._controller = controller
        self._build_ui()
        self._connect_signals()
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        fold_group = QGroupBox("折叠控制")
        fold_form = QFormLayout(fold_group)
        self._fold_slider = QSlider(Qt.Horizontal)
        self._fold_slider.setRange(0, 100)
        self._fold_slider.setValue(0)
        self._fold_label = QLabel("0 %")
        fold_form.addRow("折叠进度", self._fold_slider)
        fold_form.addRow("当前进度", self._fold_label)
        layout.addWidget(fold_group)
        view_group = QGroupBox("视角控制")
        view_form = QFormLayout(view_group)
        self._yaw_slider = QSlider(Qt.Horizontal)
        self._yaw_slider.setRange(-180, 180)
        self._yaw_slider.setValue(35)
        self._pitch_slider = QSlider(Qt.Horizontal)
        self._pitch_slider.setRange(-89, 89)
        self._pitch_slider.setValue(25)
        self._scale_slider = QSlider(Qt.Horizontal)
        self._scale_slider.setRange(10, 300)
        self._scale_slider.setValue(100)
        view_form.addRow("偏航角", self._yaw_slider)
        view_form.addRow("俯仰角", self._pitch_slider)
        view_form.addRow("缩放", self._scale_slider)
        layout.addWidget(view_group)
        display_group = QGroupBox("显示选项")
        display_layout = QVBoxLayout(display_group)
        self._face_check = QCheckBox("显示面片")
        self._face_check.setChecked(True)
        self._vertex_check = QCheckBox("显示顶点")
        self._vertex_check.setChecked(True)
        self._sync_check = QCheckBox("二维编辑自动同步")
        self._sync_check.setChecked(True)
        display_layout.addWidget(self._face_check)
        display_layout.addWidget(self._vertex_check)
        display_layout.addWidget(self._sync_check)
        layout.addWidget(display_group)
        self._stat_label = QLabel("顶点 0 / 折痕 0 / 面片 0")
        self._stat_label.setWordWrap(True)
        layout.addWidget(self._stat_label)
        button_row = QHBoxLayout()
        self._reset_button = QPushButton("重置视角")
        self._apply_button = QPushButton("应用折叠")
        button_row.addWidget(self._reset_button)
        button_row.addWidget(self._apply_button)
        layout.addLayout(button_row)
        layout.addStretch(1)
    def _connect_signals(self) -> None:
        self._fold_slider.valueChanged.connect(self._on_fold_changed)
        self._yaw_slider.valueChanged.connect(self._on_view_changed)
        self._pitch_slider.valueChanged.connect(self._on_view_changed)
        self._scale_slider.valueChanged.connect(self._on_scale_changed)
        self._reset_button.clicked.connect(self._on_reset_view)
        self._apply_button.clicked.connect(self._on_apply_fold)
        self._sync_check.toggled.connect(self._controller.set_auto_sync)
        self._controller.model_rebuilt.connect(self._refresh_statistics)
    def bind_preview(self, preview: FoldPreviewWidget) -> None:
        self._face_check.toggled.connect(preview.set_show_faces)
        self._vertex_check.toggled.connect(preview.set_show_vertices)
    def _on_fold_changed(self, value: int) -> None:
        self._fold_label.setText(f"{value} %")
        if self._controller.auto_sync:
            self._controller.set_fold_progress(value / 100.0)
```

```
    def _on_apply_fold(self) -> None:
        self._controller.set_fold_progress(self._fold_slider.value() / 100.0)
    def _on_view_changed(self) -> None:
        self._controller.set_view_angle(self._yaw_slider.value(), self._pitch_slider.value())
    def _on_scale_changed(self, value: int) -> None:
        self._controller.set_scale(value / 100.0)
    def _on_reset_view(self) -> None:
        self._yaw_slider.setValue(35)
        self._pitch_slider.setValue(25)
        self._scale_slider.setValue(100)
    def _refresh_statistics(self, vertex_count: int, crease_count: int) -> None:
        stats = self._controller.statistics()
        self._stat_label.setText(
            f"顶点 {vertex_count} / 折痕 {crease_count} / 面片 {int(stats['face_count'])}"
        )
class LinkageWorkspace(QWidget):
    def __init__(self, parent: Optional[QWidget] = None) -> None:
        super().__init__(parent)
        self._controller = LinkageController()
        self._preview = FoldPreviewWidget(self._controller)
        self._panel = LinkagePanel(self._controller)
        self._panel.bind_preview(self._preview)
        self._build_layout()
    @property
    def controller(self) -> LinkageController:
        return self._controller
    def _build_layout(self) -> None:
        layout = QHBoxLayout(self)
        splitter = QSplitter(Qt.Horizontal)
        splitter.addWidget(self._preview)
        splitter.addWidget(self._panel)
        splitter.setStretchFactor(0, 3)
        splitter.setStretchFactor(1, 1)
        layout.addWidget(splitter)
    def load_creases(self, creases: List[dict]) -> None:
        self._controller.rebuild_from_creases(creases)
        self._preview.update()
    def refresh(self) -> None:
        self._preview.update()
    def export_snapshot(self) -> Dict[str, object]:
        stats = self._controller.statistics()
        return {
            "vertex_count": int(stats["vertex_count"]),
            "crease_count": int(stats["crease_count"]),
            "face_count": int(stats["face_count"]),
            "fold_angle_deg": round(stats["fold_angle_deg"], 3),
            "max_height": round(stats["max_height"], 4),
            "min_height": round(stats["min_height"], 4),
        }
def build_linkage_workspace() -> LinkageWorkspace:
    return LinkageWorkspace()
def compute_fold_angle(progress: float) -> float:
    return max(0.0, min(1.0, progress)) * 180.0
def rotate_view_point(point: Point2D, angle_deg: float) -> Point2D:
    return rotate_point_2d(point, angle_deg)
def measure_crease_length(x1: float, y1: float, x2: float, y2: float) -> float:
    return distance_2d(Point2D(x1, y1), Point2D(x2, y2))
def summarize_fold_state(controller: LinkageController) -> str:
    stats = controller.statistics()
    return (
        f"顶点 {int(stats['vertex_count'])}□折痕 {int(stats['crease_count'])}□"
        f"面片 {int(stats['face_count'])}□折叠角 {stats['fold_angle_deg']:.1f}°"
    )
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/business/module_gongchengwenjianGuanli.py
import os
import shutil
from datetime import datetime
from typing import Optional
from PySide6.QtCore import Qt
from PySide6.QtWidgets import (
    QDialog,
    QFileDialog,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QListWidget,
    QListWidgetItem,
    QMessageBox,
    QPushButton,
    QVBoxLayout,
```

```
)
from app.common import logger
from app.database import db
from app.exceptions import ProjectError
from app.models.project import Project
from app.models.crease import Crease
from app.models.layer import Layer
from app.models.export_record import ExportRecord
from app.models.validation import ValidationRecord
from app.utils.file_utils import ensure_dir, read_json, write_json, safe_remove
PROJECT_FILE_SUFFIX = ".cps"
PROJECT_FILE_FILTER = "折痕图工程文件 (*.cps);;所有文件 (*.*)"
class ProjectManager:
    def __init__(self, storage_dir: Optional[str] = None) -> None:
        self.storage_dir = storage_dir or self.default_storage_dir()
        ensure_dir(self.storage_dir)
    @staticmethod
    def default_storage_dir() -> str:
        base = os.path.join(os.path.expanduser("~"), "CreasePatternStudio")
        return os.path.join(base, "projects")
    def create_project(self, name: str, author: str = "", description: str = "") -> Project:
        if not name or not name.strip():
            raise ProjectError("工程名称不能为空")
        session = db.get_session()
        try:
            exists = session.query(Project).filter(Project.name == name.strip()).first()
            if exists is not None:
                raise ProjectError(f"已存在同名工程：{name}")
            project = Project(
                name=name.strip(),
                author=author.strip(),
                description=description.strip(),
                created_at=datetime.now(),
                updated_at=datetime.now(),
                file_path="",
                crease_count=0,
                layer_count=0,
                is_open=False,
            )
            session.add(project)
            session.commit()
            session.refresh(project)
            logger.info("创建工程 %s (id=%s)", project.name, project.id)
            return project
        except ProjectError:
            session.rollback()
            raise
        except Exception as exc:
            session.rollback()
            logger.exception("创建工程失败")
            raise ProjectError(f"创建工程失败：{exc}") from exc
        finally:
            session.close()
    def list_projects(self, keyword: str = "") -> list[Project]:
        session = db.get_session()
        try:
            query = session.query(Project)
            if keyword:
                like = f"%{keyword.strip()}%"
                query = query.filter(Project.name.like(like))
            return query.order_by(Project.updated_at.desc()).all()
        finally:
            session.close()
    def get_project(self, project_id: int) -> Optional[Project]:
        session = db.get_session()
        try:
            return session.query(Project).filter(Project.id == project_id).first()
        finally:
            session.close()
    def rename_project(self, project_id: int, new_name: str) -> Project:
        if not new_name or not new_name.strip():
            raise ProjectError("工程名称不能为空")
        session = db.get_session()
        try:
            project = session.query(Project).filter(Project.id == project_id).first()
            if project is None:
                raise ProjectError("工程不存在")
            duplicate = (
                session.query(Project)
                .filter(Project.name == new_name.strip(), Project.id != project_id)
                .first()
            )
```

```
            if duplicate is not None:
                raise ProjectError(f"已存在同名工程：{new_name}")
            project.name = new_name.strip()
            project.updated_at = datetime.now()
            session.commit()
            session.refresh(project)
            return project
        except ProjectError:
            session.rollback()
            raise
        except Exception as exc:
            session.rollback()
            logger.exception("重命名工程失败")
            raise ProjectError(f"重命名工程失败：{exc}") from exc
        finally:
            session.close()
    def delete_project(self, project_id: int, remove_file: bool = True) -> None:
        session = db.get_session()
        try:
            project = session.query(Project).filter(Project.id == project_id).first()
            if project is None:
                raise ProjectError("工程不存在")
            file_path = project.file_path
            session.query(Crease).filter(Crease.project_id == project_id).delete()
            session.query(Layer).filter(Layer.project_id == project_id).delete()
            session.query(ExportRecord).filter(ExportRecord.project_id == project_id).delete()
            session.query(ValidationRecord).filter(ValidationRecord.project_id == project_id).delete()
            session.delete(project)
            session.commit()
            if remove_file and file_path and os.path.exists(file_path):
                safe_remove(file_path)
            logger.info("删除工程 id=%s", project_id)
        except ProjectError:
            session.rollback()
            raise
        except Exception as exc:
            session.rollback()
            logger.exception("删除工程失败")
            raise ProjectError(f"删除工程失败：{exc}") from exc
        finally:
            session.close()
    def save_project(self, project_id: int, target_path: str = "") -> str:
        session = db.get_session()
        try:
            project = session.query(Project).filter(Project.id == project_id).first()
            if project is None:
                raise ProjectError("工程不存在")
            creases = session.query(Crease).filter(Crease.project_id == project_id).all()
            layers = session.query(Layer).filter(Layer.project_id == project_id).all()
            path = target_path or project.file_path
            if not path:
                path = os.path.join(self.storage_dir, f"{project.name}{PROJECT_FILE_SUFFIX}")
            ensure_dir(os.path.dirname(path))
            payload = {
                "format": "crease-pattern-studio",
                "version": 1,
                "meta": {
                    "name": project.name,
                    "author": project.author,
                    "description": project.description,
                    "created_at": project.created_at.isoformat() if project.created_at else "",
                    "updated_at": datetime.now().isoformat(),
                },
                "layers": [self._layer_to_dict(layer) for layer in layers],
                "creases": [self._crease_to_dict(crease) for crease in creases],
            }
            write_json(path, payload)
            project.file_path = path
            project.updated_at = datetime.now()
            project.crease_count = len(creases)
            project.layer_count = len(layers)
            session.commit()
            logger.info("保存工程 %s -> %s", project.name, path)
            return path
        except ProjectError:
            session.rollback()
            raise
        except Exception as exc:
            session.rollback()
            logger.exception("保存工程失败")
            raise ProjectError(f"保存工程失败：{exc}") from exc
        finally:
```

```
            session.close()
    def load_project(self, file_path: str) -> Project:
        if not file_path or not os.path.exists(file_path):
            raise ProjectError("工程文件不存在")
        try:
            payload = read_json(file_path)
        except Exception as exc:
            logger.exception("读取工程文件失败")
            raise ProjectError(f"读取工程文件失败：{exc}") from exc
        if not isinstance(payload, dict) or "meta" not in payload:
            raise ProjectError("工程文件格式不正确")
        meta = payload.get("meta", {})
        name = meta.get("name") or os.path.splitext(os.path.basename(file_path))[0]
        session = db.get_session()
        try:
            project = session.query(Project).filter(Project.file_path == file_path).first()
            if project is None:
                project = Project(
                    name=name,
                    author=meta.get("author", ""),
                    description=meta.get("description", ""),
                    created_at=datetime.now(),
                    updated_at=datetime.now(),
                    file_path=file_path,
                    crease_count=0,
                    layer_count=0,
                    is_open=True,
                )
                session.add(project)
                session.flush()
            else:
                project.updated_at = datetime.now()
                project.is_open = True
                session.query(Crease).filter(Crease.project_id == project.id).delete()
                session.query(Layer).filter(Layer.project_id == project.id).delete()
            layer_map: dict[str, int] = {}
            for layer_data in payload.get("layers", []):
                layer = Layer(
                    project_id=project.id,
                    name=layer_data.get("name", "默认图层"),
                    visible=bool(layer_data.get("visible", True)),
                    locked=bool(layer_data.get("locked", False)),
                    color=layer_data.get("color", "
                    z_order=int(layer_data.get("z_order", 0)),
                )
                session.add(layer)
                session.flush()
                layer_map[layer.name] = layer.id
            crease_total = 0
            for crease_data in payload.get("creases", []):
                layer_name = crease_data.get("layer_name", "")
                crease = Crease(
                    project_id=project.id,
                    layer_id=layer_map.get(layer_name, None),
                    crease_type=crease_data.get("crease_type", "valley"),
                    x1=float(crease_data.get("x1", 0.0)),
                    y1=float(crease_data.get("y1", 0.0)),
                    x2=float(crease_data.get("x2", 0.0)),
                    y2=float(crease_data.get("y2", 0.0)),
                    line_width=float(crease_data.get("line_width", 1.0)),
                    color=crease_data.get("color", "
                    label=crease_data.get("label", ""),
                )
                session.add(crease)
                crease_total += 1
            project.crease_count = crease_total
            project.layer_count = len(layer_map)
            session.commit()
            session.refresh(project)
            logger.info("加载工程 %s 折痕 %s 条", project.name, crease_total)
            return project
        except Exception as exc:
            session.rollback()
            logger.exception("加载工程失败")
            raise ProjectError(f"加载工程失败：{exc}") from exc
        finally:
            session.close()
    def export_project_copy(self, project_id: int, target_path: str) -> str:
        session = db.get_session()
        try:
            project = session.query(Project).filter(Project.id == project_id).first()
            if project is None:
```

```
                raise ProjectError("工程不存在")
            if not project.file_path or not os.path.exists(project.file_path):
                self.save_project(project_id)
                session.refresh(project)
            if not project.file_path or not os.path.exists(project.file_path):
                raise ProjectError("工程尚未保存，无法导出副本")
            ensure_dir(os.path.dirname(target_path))
            shutil.copyfile(project.file_path, target_path)
            logger.info("导出工程副本 %s -> %s", project.name, target_path)
            return target_path
        finally:
            session.close()
    def statistics(self) -> dict:
        session = db.get_session()
        try:
            total = session.query(Project).count()
            opened = session.query(Project).filter(Project.is_open.is_(True)).count()
            crease_total = session.query(Crease).count()
            layer_total = session.query(Layer).count()
            latest = session.query(Project).order_by(Project.updated_at.desc()).first()
            return {
                "project_total": total,
                "opened_total": opened,
                "crease_total": crease_total,
                "layer_total": layer_total,
                "latest_name": latest.name if latest else "",
                "latest_time": latest.updated_at.strftime("%Y-%m-%d %H:%M") if latest and latest.updated_at else
"",
            }
        finally:
            session.close()
    @staticmethod
    def _layer_to_dict(layer: Layer) -> dict:
        return {
            "name": layer.name,
            "visible": layer.visible,
            "locked": layer.locked,
            "color": layer.color,
            "z_order": layer.z_order,
        }
    @staticmethod
    def _crease_to_dict(crease: Crease) -> dict:
        return {
            "crease_type": crease.crease_type,
            "x1": crease.x1,
            "y1": crease.y1,
            "x2": crease.x2,
            "y2": crease.y2,
            "line_width": crease.line_width,
            "color": crease.color,
            "label": crease.label,
            "layer_name": crease.layer.name if crease.layer else "",
        }
class ProjectFileDialog(QDialog):
    def __init__(self, manager: ProjectManager, parent=None) -> None:
        super().__init__(parent)
        self.manager = manager
        self.current_project_id: Optional[int] = None
        self.setWindowTitle("折痕图可视化编辑与标准化导出系统 - 工程文件管理")
        self.resize(640, 460)
        self._build_ui()
        self.refresh_list()
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        search_row = QHBoxLayout()
        search_row.addWidget(QLabel("工程检索："))
        self.search_edit = QLineEdit()
        self.search_edit.setPlaceholderText("输入工程名称关键字")
        self.search_edit.textChanged.connect(self.refresh_list)
        search_row.addWidget(self.search_edit)
        layout.addLayout(search_row)
        self.project_list = QListWidget()
        self.project_list.itemSelectionChanged.connect(self._on_selection_changed)
        layout.addWidget(self.project_list)
        self.info_label = QLabel("未选择工程")
        self.info_label.setWordWrap(True)
        layout.addWidget(self.info_label)
        button_row = QHBoxLayout()
        self.new_button = QPushButton("新建工程")
        self.new_button.clicked.connect(self.create_project)
        self.save_button = QPushButton("保存工程")
        self.save_button.clicked.connect(self.save_project)
```

```
        self.load_button = QPushButton("加载工程")
        self.load_button.clicked.connect(self.load_project)
        self.delete_button = QPushButton("删除工程")
        self.delete_button.clicked.connect(self.delete_project)
        self.close_button = QPushButton("关闭")
        self.close_button.clicked.connect(self.accept)
        for button in (
            self.new_button,
            self.save_button,
            self.load_button,
            self.delete_button,
            self.close_button,
        ):
            button_row.addWidget(button)
        layout.addLayout(button_row)
    def refresh_list(self) -> None:
        keyword = self.search_edit.text().strip()
        self.project_list.clear()
        for project in self.manager.list_projects(keyword):
            updated = project.updated_at.strftime("%Y-%m-%d %H:%M") if project.updated_at else "-"
            text = f"{project.name}    折痕 {project.crease_count} 条    更新 {updated}"
            item = QListWidgetItem(text)
            item.setData(Qt.UserRole, project.id)
            self.project_list.addItem(item)
    def _on_selection_changed(self) -> None:
        items = self.project_list.selectedItems()
        if not items:
            self.current_project_id = None
            self.info_label.setText("未选择工程")
            return
        self.current_project_id = items[0].data(Qt.UserRole)
        project = self.manager.get_project(self.current_project_id)
        if project is None:
            self.info_label.setText("工程记录已失效")
            return
        self.info_label.setText(
            f"名称：{project.name}\n作者：{project.author or '-'}\n"
            f"折痕：{project.crease_count} 条    图层：{project.layer_count} 个\n"
            f"文件：{project.file_path or '尚未保存到磁盘'}"
        )
    def create_project(self) -> None:
        name, ok = self._prompt_text("新建工程", "请输入工程名称：")
        if not ok or not name.strip():
            return
        try:
            project = self.manager.create_project(name)
        except ProjectError as exc:
            QMessageBox.warning(self, "新建失败", str(exc))
            return
        self.refresh_list()
        self._select_project(project.id)
    def save_project(self) -> None:
        if self.current_project_id is None:
            QMessageBox.information(self, "提示", "请先选择要保存的工程")
            return
        project = self.manager.get_project(self.current_project_id)
        if project is None:
            QMessageBox.warning(self, "保存失败", "工程不存在")
            return
        default_path = project.file_path or os.path.join(
            self.manager.storage_dir, f"{project.name}{PROJECT_FILE_SUFFIX}"
        )
        path, _ = QFileDialog.getSaveFileName(self, "保存工程文件", default_path, PROJECT_FILE_FILTER)
        if not path:
            return
        try:
            saved = self.manager.save_project(self.current_project_id, path)
        except ProjectError as exc:
            QMessageBox.warning(self, "保存失败", str(exc))
            return
        self.refresh_list()
        self._select_project(self.current_project_id)
        QMessageBox.information(self, "保存成功", f"工程已保存至：\n{saved}")
    def load_project(self) -> None:
        path, _ = QFileDialog.getOpenFileName(
            self, "加载工程文件", self.manager.storage_dir, PROJECT_FILE_FILTER
        )
        if not path:
            return
        try:
            project = self.manager.load_project(path)
        except ProjectError as exc:
```

```
            QMessageBox.warning(self, "加载失败", str(exc))
            return
        self.refresh_list()
        self._select_project(project.id)
        QMessageBox.information(self, "加载成功", f"已加载工程：{project.name}")
    def delete_project(self) -> None:
        if self.current_project_id is None:
            QMessageBox.information(self, "提示", "请先选择要删除的工程")
            return
        answer = QMessageBox.question(
            self,
          "删除确认",
          "删除后工程记录与关联折痕数据将一并移除，是否继续？",
            QMessageBox.Yes | QMessageBox.No,
        )
        if answer != QMessageBox.Yes:
            return
        try:
            self.manager.delete_project(self.current_project_id)
        except ProjectError as exc:
            QMessageBox.warning(self, "删除失败", str(exc))
            return
        self.current_project_id = None
        self.refresh_list()
    def _select_project(self, project_id: int) -> None:
        for row in range(self.project_list.count()):
            item = self.project_list.item(row)
            if item.data(Qt.UserRole) == project_id:
                self.project_list.setCurrentItem(item)
                break
    def _prompt_text(self, title: str, label: str) -> tuple[str, bool]:
        dialog = QDialog(self)
        dialog.setWindowTitle(title)
        layout = QVBoxLayout(dialog)
        layout.addWidget(QLabel(label))
        edit = QLineEdit(dialog)
        layout.addWidget(edit)
        row = QHBoxLayout()
        ok_button = QPushButton("确定")
        cancel_button = QPushButton("取消")
        ok_button.clicked.connect(dialog.accept)
        cancel_button.clicked.connect(dialog.reject)
        row.addWidget(ok_button)
        row.addWidget(cancel_button)
        layout.addLayout(row)
        accepted = dialog.exec() == QDialog.Accepted
        return edit.text(), accepted
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/business/module_jihejiansuoxiaoyan.py
import math
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Sequence, Tuple
from PySide6.QtWidgets import (
    QAbstractItemView,
    QComboBox,
    QHBoxLayout,
    QHeaderView,
    QLabel,
    QMessageBox,
    QPushButton,
    QTableWidget,
    QTableWidgetItem,
    QVBoxLayout,
    QWidget,
)
from app.common import logger
from app.core.geometry import Point2D, Segment2D, polygon_area, segments_intersect
from app.exceptions import ValidationError
from app.models.crease import Crease, CreaseType
from app.models.validation import ValidationIssue, ValidationSeverity
@dataclass
class ConstraintRule:
    rule_id: str
    name: str
    severity: ValidationSeverity
    enabled: bool = True
    description: str = ""
@dataclass
class ValidationContext:
    creases: List[Crease] = field(default_factory=list)
```

```
    tolerance: float = 1e-6
    min_vertex_angle: float = 1.0
    max_vertex_angle: float = 359.0
    rules: Dict[str, ConstraintRule] = field(default_factory=dict)
@dataclass
class ValidationResult:
    issues: List[ValidationIssue] = field(default_factory=list)
    checked_rules: int = 0
    elapsed_ms: float = 0.0
    @property
    def error_count(self) -> int:
        return sum(1 for i in self.issues if i.severity == ValidationSeverity.ERROR)
    @property
    def warning_count(self) -> int:
        return sum(1 for i in self.issues if i.severity == ValidationSeverity.WARNING)
    @property
    def passed(self) -> bool:
        return self.error_count == 0
DEFAULT_RULES: Tuple[ConstraintRule, ...] = (
    ConstraintRule("R001", "顶点角度异常", ValidationSeverity.ERROR, True, "顶点相邻折痕夹角过小或过大"),
    ConstraintRule("R002", "折痕交叉冲突", ValidationSeverity.ERROR, True, "非共享端点的折痕线段相交"),
    ConstraintRule("R003", "顶点度数异常", ValidationSeverity.WARNING, True, "顶点连接折痕数量不符合折叠规律"),
    ConstraintRule("R004", "面片重叠", ValidationSeverity.ERROR, True, "相邻面片存在重叠区域"),
    ConstraintRule("R005", "零长度折痕", ValidationSeverity.WARNING, True, "折痕两端点重合"),
)
def build_default_context(creases: Sequence[Crease], tolerance: float = 1e-6) -> ValidationContext:
    rules = {rule.rule_id: rule for rule in DEFAULT_RULES}
    return ValidationContext(creases=list(creases), tolerance=tolerance, rules=rules)
def _crease_to_segment(crease: Crease) -> Segment2D:
    return Segment2D(
        Point2D(crease.start_x, crease.start_y),
        Point2D(crease.end_x, crease.end_y),
    )
def _shared_endpoint(a: Crease, b: Crease, tol: float) -> bool:
    pairs = (
        (a.start_x, a.start_y, b.start_x, b.start_y),
        (a.start_x, a.start_y, b.end_x, b.end_y),
        (a.end_x, a.end_y, b.start_x, b.start_y),
        (a.end_x, a.end_y, b.end_x, b.end_y),
    )
    for ax, ay, bx, by in pairs:
        if abs(ax - bx) <= tol and abs(ay - by) <= tol:
            return True
    return False
def _vertex_angle(center: Point2D, p1: Point2D, p2: Point2D) -> float:
    v1x, v1y = p1.x - center.x, p1.y - center.y
    v2x, v2y = p2.x - center.x, p2.y - center.y
    n1 = math.hypot(v1x, v1y)
    n2 = math.hypot(v2x, v2y)
    if n1 < 1e-12 or n2 < 1e-12:
        return 0.0
    cos_val = max(-1.0, min(1.0, (v1x * v2x + v1y * v2y) / (n1 * n2)))
    return math.degrees(math.acos(cos_val))
def collect_vertices(creases: Sequence[Crease], tol: float) -> Dict[Tuple[int, int], List[int]]:
    buckets: Dict[Tuple[int, int], List[int]] = {}
    for idx, crease in enumerate(creases):
        for x, y in ((crease.start_x, crease.start_y), (crease.end_x, crease.end_y)):
            key = (round(x / tol), round(y / tol))
            buckets.setdefault(key, []).append(idx)
    return buckets
def validate_angle_consistency(ctx: ValidationContext) -> List[ValidationIssue]:
    issues: List[ValidationIssue] = []
    rule = ctx.rules.get("R001")
    if rule is None or not rule.enabled:
        return issues
    vertices = collect_vertices(ctx.creases, ctx.tolerance)
    for key, indices in vertices.items():
        if len(indices) < 2:
            continue
        center = Point2D(key[0] * ctx.tolerance, key[1] * ctx.tolerance)
        endpoints: List[Point2D] = []
        for idx in indices:
            crease = ctx.creases[idx]
            far = Point2D(crease.end_x, crease.end_y)
            if abs(far.x - center.x) <= ctx.tolerance and abs(far.y - center.y) <= ctx.tolerance:
                far = Point2D(crease.start_x, crease.start_y)
            endpoints.append(far)
        for i in range(len(endpoints)):
            for j in range(i + 1, len(endpoints)):
                angle = _vertex_angle(center, endpoints[i], endpoints[j])
                if angle < ctx.min_vertex_angle or angle > ctx.max_vertex_angle:
                    issues.append(
                        ValidationIssue(
```

```
                            rule_id=rule.rule_id,
                            severity=rule.severity,
                            message=f"顶点({center.x:.3f},{center.y:.3f})夹角 {angle:.2f}° 超出允许范围",
                            location=(center.x, center.y),
                        )
                    )
    return issues
def validate_crossing(ctx: ValidationContext) -> List[ValidationIssue]:
    issues: List[ValidationIssue] = []
    rule = ctx.rules.get("R002")
    if rule is None or not rule.enabled:
        return issues
    segments = [_crease_to_segment(c) for c in ctx.creases]
    for i in range(len(segments)):
        for j in range(i + 1, len(segments)):
            if _shared_endpoint(ctx.creases[i], ctx.creases[j], ctx.tolerance):
                continue
            if segments_intersect(segments[i], segments[j], ctx.tolerance):
                mid = Point2D(
                    (segments[i].a.x + segments[j].a.x) / 2.0,
                    (segments[i].a.y + segments[j].a.y) / 2.0,
                )
                issues.append(
                    ValidationIssue(
                        rule_id=rule.rule_id,
                        severity=rule.severity,
                        message=f"折痕 {ctx.creases[i].name} 与 {ctx.creases[j].name} 交叉冲突",
                        location=(mid.x, mid.y),
                    )
                )
    return issues
def validate_vertex_degree(ctx: ValidationContext) -> List[ValidationIssue]:
    issues: List[ValidationIssue] = []
    rule = ctx.rules.get("R003")
    if rule is None or not rule.enabled:
        return issues
    vertices = collect_vertices(ctx.creases, ctx.tolerance)
    for key, indices in vertices.items():
        degree = len(indices)
        if degree == 1:
            issues.append(
                ValidationIssue(
                    rule_id=rule.rule_id,
                    severity=rule.severity,
                    message=f"顶点({key[0] * ctx.tolerance:.3f},{key[1] * ctx.tolerance:.3f})为悬空端点",
                    location=(key[0] * ctx.tolerance, key[1] * ctx.tolerance),
                )
            )
        elif degree % 2 == 1 and degree > 1:
            issues.append(
                ValidationIssue(
                    rule_id=rule.rule_id,
                    severity=rule.severity,
                    message=f"顶点({key[0] * ctx.tolerance:.3f},{key[1] * ctx.tolerance:.3f})度数为奇数 {degree}",
                    location=(key[0] * ctx.tolerance, key[1] * ctx.tolerance),
                )
            )
    return issues
def validate_face_overlap(ctx: ValidationContext) -> List[ValidationIssue]:
    issues: List[ValidationIssue] = []
    rule = ctx.rules.get("R004")
    if rule is None or not rule.enabled:
        return issues
    polygons = _extract_faces(ctx.creases, ctx.tolerance)
    for i in range(len(polygons)):
        for j in range(i + 1, len(polygons)):
            if _polygons_overlap(polygons[i], polygons[j], ctx.tolerance):
                cx = sum(p.x for p in polygons[i]) / len(polygons[i])
                cy = sum(p.y for p in polygons[i]) / len(polygons[i])
                issues.append(
                    ValidationIssue(
                        rule_id=rule.rule_id,
                        severity=rule.severity,
                        message=f"面片 {i + 1} 与面片 {j + 1} 存在重叠",
                        location=(cx, cy),
                    )
                )
    return issues
def validate_zero_length(ctx: ValidationContext) -> List[ValidationIssue]:
    issues: List[ValidationIssue] = []
    rule = ctx.rules.get("R005")
    if rule is None or not rule.enabled:
        return issues
```

```
    for crease in ctx.creases:
        length = math.hypot(crease.end_x - crease.start_x, crease.end_y - crease.start_y)
        if length <= ctx.tolerance:
            issues.append(
                ValidationIssue(
                    rule_id=rule.rule_id,
                    severity=rule.severity,
                    message=f"折痕 {crease.name} 长度为零",
                    location=(crease.start_x, crease.start_y),
                )
            )
    return issues
def _extract_faces(creases: Sequence[Crease], tol: float) -> List[List[Point2D]]:
    faces: List[List[Point2D]] = []
    for crease in creases:
        if crease.crease_type == CreaseType.BOUNDARY:
            faces.append(
                [
                    Point2D(crease.start_x, crease.start_y),
                    Point2D(crease.end_x, crease.end_y),
                ]
            )
    return faces
def _polygons_overlap(a: Sequence[Point2D], b: Sequence[Point2D], tol: float) -> bool:
    if len(a) < 3 or len(b) < 3:
        return False
    area_a = abs(polygon_area(a))
    area_b = abs(polygon_area(b))
    if area_a <= tol or area_b <= tol:
        return False
    for pa in a:
        for pb in b:
            if abs(pa.x - pb.x) <= tol and abs(pa.y - pb.y) <= tol:
                return True
    return False
def run_validation(ctx: ValidationContext) -> ValidationResult:
    import time
    start = time.perf_counter()
    issues: List[ValidationIssue] = []
    checked = 0
    for func in (
        validate_angle_consistency,
        validate_crossing,
        validate_vertex_degree,
        validate_face_overlap,
        validate_zero_length,
    ):
        try:
            issues.extend(func(ctx))
            checked += 1
        except ValidationError as exc:
            logger.warning("校验规则执行失败: %s", exc)
        except Exception as exc:
            logger.exception("校验规则异常: %s", exc)
    elapsed = (time.perf_counter() - start) * 1000.0
    result = ValidationResult(issues=issues, checked_rules=checked, elapsed_ms=elapsed)
    logger.info("几何约束校验完成: 问题 %d 项, 耗时 %.2f ms", len(issues), elapsed)
    return result
def summarize_result(result: ValidationResult) -> str:
    return (
        f"校验规则 {result.checked_rules} 项，错误 {result.error_count} 项，"
        f"警告 {result.warning_count} 项，耗时 {result.elapsed_ms:.2f} ms"
    )
class ConstraintValidationPanel(QWidget):
    def __init__(self, parent: Optional[QWidget] = None) -> None:
        super().__init__(parent)
        self._creases: List[Crease] = []
        self._result: Optional[ValidationResult] = None
        self._build_ui()
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        toolbar = QHBoxLayout()
        self.rule_combo = QComboBox()
        self.rule_combo.addItem("全部规则", "ALL")
        for rule in DEFAULT_RULES:
            self.rule_combo.addItem(rule.name, rule.rule_id)
        toolbar.addWidget(QLabel("校验规则"))
        toolbar.addWidget(self.rule_combo)
        self.run_button = QPushButton("执行校验")
        self.run_button.clicked.connect(self.run_check)
        toolbar.addWidget(self.run_button)
        self.clear_button = QPushButton("清空结果")
```

```
        self.clear_button.clicked.connect(self.clear_result)
        toolbar.addWidget(self.clear_button)
        toolbar.addStretch(1)
        layout.addLayout(toolbar)
        self.summary_label = QLabel("尚未执行校验")
        layout.addWidget(self.summary_label)
        self.table = QTableWidget(0, 4)
        self.table.setHorizontalHeaderLabels(["规则", "级别", "描述", "位置"])
        self.table.setSelectionBehavior(QAbstractItemView.SelectionBehavior.SelectRows)
        self.table.setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers)
        header = self.table.horizontalHeader()
        header.setSectionResizeMode(2, QHeaderView.ResizeMode.Stretch)
        layout.addWidget(self.table)
    def set_creases(self, creases: Sequence[Crease]) -> None:
        self._creases = list(creases)
    def run_check(self) -> ValidationResult:
        ctx = build_default_context(self._creases)
        selected = self.rule_combo.currentData()
        if selected and selected != "ALL":
            for rule_id, rule in ctx.rules.items():
                rule.enabled = rule_id == selected
        self._result = run_validation(ctx)
        self._refresh_table()
        self.summary_label.setText(summarize_result(self._result))
        return self._result
    def clear_result(self) -> None:
        self._result = None
        self.table.setRowCount(0)
        self.summary_label.setText("尚未执行校验")
    def _refresh_table(self) -> None:
        self.table.setRowCount(0)
        if self._result is None:
            return
        for issue in self._result.issues:
            row = self.table.rowCount()
            self.table.insertRow(row)
            self.table.setItem(row, 0, QTableWidgetItem(issue.rule_id))
            self.table.setItem(row, 1, QTableWidgetItem(issue.severity.value))
            self.table.setItem(row, 2, QTableWidgetItem(issue.message))
            loc = issue.location
            text = f"({loc[0]:.3f}, {loc[1]:.3f})" if loc else "-"
            self.table.setItem(row, 3, QTableWidgetItem(text))
    def export_issues(self) -> List[ValidationIssue]:
        if self._result is None:
            return []
        return list(self._result.issues)
    def notify_summary(self) -> None:
        if self._result is None:
            QMessageBox.information(self, "几何约束校验", "尚未执行校验")
            return
        QMessageBox.information(self, "几何约束校验", summarize_result(self._result))
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/business/module_zhetuleixingbiaozhu.py
def _build_ui(self) -> None:
        root_layout = QVBoxLayout(self)
        root_layout.setContentsMargins(12, 12, 12, 12)
        root_layout.setSpacing(10)
        splitter = QSplitter(Qt.Horizontal, self)
        left_panel = QWidget(self)
        left_layout = QVBoxLayout(left_panel)
        left_layout.setContentsMargins(0, 0, 0, 0)
        left_layout.setSpacing(8)
        form_group = QGroupBox("折痕标注", left_panel)
        form_layout = QFormLayout(form_group)
        self._crease_id_edit = QLineEdit(form_group)
        self._crease_id_edit.setPlaceholderText("例如：12 或 12,15,18")
        form_layout.addRow("折痕编号□", self._crease_id_edit)
        self._type_combo = QComboBox(form_group)
        for crease_type in CreaseType:
            self._type_combo.addItem(crease_type.label, crease_type)
        form_layout.addRow("目标类型□", self._type_combo)
        button_row = QHBoxLayout()
        self._apply_button = QPushButton("应用标注", form_group)
        self._apply_button.clicked.connect(self._on_apply_clicked)
        self._batch_button = QPushButton("批量标注", form_group)
        self._batch_button.clicked.connect(self._on_batch_clicked)
        button_row.addWidget(self._apply_button)
        button_row.addWidget(self._batch_button)
        form_layout.addRow(button_row)
        left_layout.addWidget(form_group)
```

```
        stat_group = QGroupBox("折痕统计", left_panel)
        stat_layout = QVBoxLayout(stat_group)
        self._stat_table = QTableWidget(0, 4, stat_group)
        self._stat_table.setHorizontalHeaderLabels(["类型", "数量", "总长度", "平均长度"])
        self._stat_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self._stat_table.verticalHeader().setVisible(False)
        self._stat_table.setEditTriggers(QAbstractItemView.NoEditTriggers)
        stat_layout.addWidget(self._stat_table)
        left_layout.addWidget(stat_group, 1)
        right_panel = QWidget(self)
        right_layout = QVBoxLayout(right_panel)
        right_layout.setContentsMargins(0, 0, 0, 0)
        right_layout.setSpacing(8)
        style_group = QGroupBox("样式配置", right_panel)
        style_layout = QVBoxLayout(style_group)
        self._style_table = QTableWidget(0, 6, style_group)
        self._style_table.setHorizontalHeaderLabels(
            ["类型", "颜色", "线宽", "可见", "锁定", "操作"]
        )
        self._style_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self._style_table.verticalHeader().setVisible(False)
        self._style_table.setEditTriggers(QAbstractItemView.NoEditTriggers)
        style_layout.addWidget(self._style_table)
        reset_row = QHBoxLayout()
        reset_row.addStretch(1)
        self._reset_button = QPushButton("恢复默认样式", style_group)
        self._reset_button.clicked.connect(self._on_reset_style)
        reset_row.addWidget(self._reset_button)
        style_layout.addLayout(reset_row)
        right_layout.addWidget(style_group, 1)
        splitter.addWidget(left_panel)
        splitter.addWidget(right_panel)
        splitter.setStretchFactor(0, 1)
        splitter.setStretchFactor(1, 1)
        root_layout.addWidget(splitter)
    def _connect_signals(self) -> None:
        self._annotator.registry.style_updated.connect(self._on_style_updated)
    def _on_style_updated(self, crease_type: CreaseType) -> None:
        self._refresh_style_table()
        self.style_changed.emit(crease_type)
class CreaseAnnotationPanel(QWidget):
    crease_type_changed = Signal(int, CreaseType)
    style_changed = Signal(object)
    def __init__(
        self,
        annotator: CreaseAnnotator,
        parent: Optional[QWidget] = None,
    ) -> None:
        super().__init__(parent)
        self._annotator = annotator
        self._current_project_id: Optional[int] = None
        self._build_ui()
        self._connect_signals()
        self._refresh_style_table()
        self.refresh_statistics()
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(10, 10, 10, 10)
        layout.setSpacing(8)
        toolbar = QHBoxLayout()
        self._project_label = QLabel("当前项目：未选择", self)
        toolbar.addWidget(self._project_label)
        toolbar.addStretch(1)
        self._refresh_button = QPushButton("刷新", self)
        self._refresh_button.clicked.connect(self.refresh_statistics)
        toolbar.addWidget(self._refresh_button)
        layout.addLayout(toolbar)
        self._tabs = QTabWidget(self)
        self._annotation_tab = CreaseAnnotationTab(self._annotator, self)
        self._annotation_tab.crease_type_changed.connect(self.crease_type_changed.emit)
        self._annotation_tab.style_changed.connect(self.style_changed.emit)
        self._tabs.addTab(self._annotation_tab, "标注")
        self._stat_tab = CreaseStatisticsTab(self._annotator, self)
        self._tabs.addTab(self._stat_tab, "统计")
        self._style_tab = CreaseStyleTab(self._annotator, self)
        self._style_tab.style_changed.connect(self.style_changed.emit)
        self._tabs.addTab(self._style_tab, "样式")
        layout.addWidget(self._tabs, 1)
    def _connect_signals(self) -> None:
        self._annotation_tab.crease_type_changed.connect(self._on_crease_changed)
    def _on_crease_changed(self, crease_id: int, crease_type: CreaseType) -> None:
        self._stat_tab.refresh_statistics()
```

```
        self.crease_type_changed.emit(crease_id, crease_type)
    def set_project(self, project_id: Optional[int]) -> None:
        self._current_project_id = project_id
        if project_id is None:
            self._project_label.setText("当前项目：未选择")
        else:
            self._project_label.setText(f"当前项目：
        self._annotation_tab.set_project(project_id)
        self._stat_tab.set_project(project_id)
    def refresh_statistics(self) -> None:
        self._stat_tab.refresh_statistics()
        self._annotation_tab.refresh_statistics()
class CreaseStatisticsTab(QWidget):
    def __init__(
        self,
        annotator: CreaseAnnotator,
        parent: Optional[QWidget] = None,
    ) -> None:
        super().__init__(parent)
        self._annotator = annotator
        self._current_project_id: Optional[int] = None
        self._build_ui()
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(8, 8, 8, 8)
        layout.setSpacing(8)
        self._stat_table = QTableWidget(0, 4, self)
        self._stat_table.setHorizontalHeaderLabels(["类型", "数量", "总长度", "平均长度"])
        self._stat_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self._stat_table.verticalHeader().setVisible(False)
        self._stat_table.setEditTriggers(QAbstractItemView.NoEditTriggers)
        layout.addWidget(self._stat_table, 1)
        self._summary_label = QLabel("暂无统计数据", self)
        layout.addWidget(self._summary_label)
    def set_project(self, project_id: Optional[int]) -> None:
        self._current_project_id = project_id
        self.refresh_statistics()
    def refresh_statistics(self) -> None:
        if self._current_project_id is None:
            self._stat_table.setRowCount(0)
            self._summary_label.setText("暂无统计数据")
            return
        stats = self._annotator.collect_statistics(self._current_project_id)
        self._stat_table.setRowCount(len(stats))
        total_count = 0
        total_length = 0.0
        for row, stat in enumerate(stats):
            self._stat_table.setItem(row, 0, QTableWidgetItem(stat.label))
            self._stat_table.setItem(row, 1, QTableWidgetItem(str(stat.count)))
            self._stat_table.setItem(row, 2, QTableWidgetItem(f"{stat.total_length:.2f}"))
            self._stat_table.setItem(row, 3, QTableWidgetItem(f"{stat.average_length:.2f}"))
            total_count += stat.count
            total_length += stat.total_length
        self._summary_label.setText(
            f"共 {total_count} 条折痕，总长度 {total_length:.2f}"
        )
class CreaseStyleTab(QWidget):
    style_changed = Signal(object)
    def __init__(
        self,
        annotator: CreaseAnnotator,
        parent: Optional[QWidget] = None,
    ) -> None:
        super().__init__(parent)
        self._annotator = annotator
        self._build_ui()
        self._refresh_style_table()
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        layout.setContentsMargins(8, 8, 8, 8)
        layout.setSpacing(8)
        self._style_table = QTableWidget(0, 6, self)
        self._style_table.setHorizontalHeaderLabels(
          ["类型", "颜色", "线宽", "可见", "锁定", "操作"]
        )
        self._style_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self._style_table.verticalHeader().setVisible(False)
        self._style_table.setEditTriggers(QAbstractItemView.NoEditTriggers)
        layout.addWidget(self._style_table, 1)
        button_row = QHBoxLayout()
        button_row.addStretch(1)
        self._reset_button = QPushButton("恢复默认样式", self)
```

```
        self._reset_button.clicked.connect(self._on_reset_style)
        button_row.addWidget(self._reset_button)
        layout.addLayout(button_row)
    def _refresh_style_table(self) -> None:
        styles = self._annotator.registry.all_styles()
        self._style_table.setRowCount(len(styles))
        for row, style in enumerate(styles):
            self._style_table.setItem(row, 0, QTableWidgetItem(style.label))
            color_item = QTableWidgetItem(style.color)
            color_item.setBackground(QColor(style.color))
            color_item.setForeground(QColor("
            self._style_table.setItem(row, 1, color_item)
            self._style_table.setItem(row, 2, QTableWidgetItem(f"{style.line_width:.2f}"))
            self._style_table.setItem(row, 3, QTableWidgetItem("是" if style.visible else "否"))
            self._style_table.setItem(row, 4, QTableWidgetItem("是" if style.locked else "否"))
            edit_button = QPushButton("编辑", self._style_table)
            edit_button.clicked.connect(
                lambda _=False, ct=style.crease_type: self._edit_style(ct)
            )
            self._style_table.setCellWidget(row, 5, edit_button)
    def _edit_style(self, crease_type: CreaseType) -> None:
        style = self._annotator.registry.get_style(crease_type)
        color = QColorDialog.getColor(QColor(style.color), self, f"选择 {style.label} 颜色")
        if color.isValid():
            self._annotator.registry.update_style(crease_type, color=color.name())
            self.style_changed.emit(crease_type)
        self._refresh_style_table()
    def _on_reset_style(self) -> None:
        self._annotator.registry.reset_defaults()
        self._refresh_style_table()
        self.style_changed.emit(None)
# 文件: app/business/module_zhetuwenzengshanjianmo.py
import math
from dataclasses import dataclass, field
from typing import Dict, List, Optional
from PySide6.QtCore import QPointF, QRectF, Qt
from PySide6.QtGui import QColor, QPen, QBrush, QPainterPath
from PySide6.QtWidgets import (
    QWidget,
    QVBoxLayout,
    QHBoxLayout,
    QPushButton,
    QComboBox,
    QLabel,
    QSpinBox,
    QDoubleSpinBox,
    QGroupBox,
    QFormLayout,
    QMessageBox,
    QListWidget,
    QListWidgetItem,
    QColorDialog,
    QToolButton,
)
from app.common import logger
from app.core.geometry import distance_point_to_segment, point_in_polygon
from app.core.undo_stack import UndoStack
from app.utils.color_utils import crease_type_color, normalize_hex_color
CREASE_TYPE_PEAK = "peak"
CREASE_TYPE_VALLEY = "valley"
CREASE_TYPE_FLAT = "flat"
CREASE_TYPE_BOUNDARY = "boundary"
CREASE_TYPE_CUT = "cut"
CREASE_TYPE_AUXILIARY = "auxiliary"
CREASE_TYPE_LABELS = {
    CREASE_TYPE_PEAK: "峰折线",
    CREASE_TYPE_VALLEY: "谷折线",
    CREASE_TYPE_FLAT: "段折线",
    CREASE_TYPE_BOUNDARY: "边界线",
    CREASE_TYPE_CUT: "剪裁线",
    CREASE_TYPE_AUXILIARY: "辅助参考线",
}
DEFAULT_LINE_WIDTH = 1.6
DEFAULT_SNAP_RADIUS = 8.0
MIN_SEGMENT_LENGTH = 1e-4
@dataclass
class CreaseVertex:
    vertex_id: int
    x: float
    y: float
    locked: bool = False
    def as_point(self) -> QPointF:
```

```
        return QPointF(self.x, self.y)
    def move_to(self, x: float, y: float) -> None:
        if self.locked:
            raise ValueError("顶点已锁定，无法移动")
        self.x = float(x)
        self.y = float(y)
@dataclass
class CreaseSegment:
    segment_id: int
    start_vertex: int
    end_vertex: int
    crease_type: str = CREASE_TYPE_VALLEY
    layer_id: int = 0
    color: str = ""
    width: float = DEFAULT_LINE_WIDTH
    visible: bool = True
    def length(self, vertices: Dict[int, CreaseVertex]) -> float:
        a = vertices.get(self.start_vertex)
        b = vertices.get(self.end_vertex)
        if a is None or b is None:
            return 0.0
        return math.hypot(b.x - a.x, b.y - a.y)
@dataclass
class CreasePattern:
    name: str = "未命名折痕图"
    vertices: Dict[int, CreaseVertex] = field(default_factory=dict)
    segments: Dict[int, CreaseSegment] = field(default_factory=dict)
    next_vertex_id: int = 1
    next_segment_id: int = 1
    def add_vertex(self, x: float, y: float, locked: bool = False) -> CreaseVertex:
        vertex = CreaseVertex(self.next_vertex_id, float(x), float(y), locked)
        self.vertices[vertex.vertex_id] = vertex
        self.next_vertex_id += 1
        return vertex
    def add_segment(
        self,
        start_vertex: int,
        end_vertex: int,
        crease_type: str = CREASE_TYPE_VALLEY,
        layer_id: int = 0,
    ) -> CreaseSegment:
        if start_vertex == end_vertex:
            raise ValueError("折痕线段的两个端点不能相同")
        if start_vertex not in self.vertices or end_vertex not in self.vertices:
            raise KeyError("折痕线段端点不存在")
        segment = CreaseSegment(
            segment_id=self.next_segment_id,
            start_vertex=start_vertex,
            end_vertex=end_vertex,
            crease_type=crease_type,
            layer_id=layer_id,
            color=crease_type_color(crease_type),
        )
        self.segments[segment.segment_id] = segment
        self.next_segment_id += 1
        return segment
    def remove_segment(self, segment_id: int) -> bool:
        return self.segments.pop(segment_id, None) is not None
    def remove_vertex(self, vertex_id: int) -> int:
        removed = 0
        for seg_id in list(self.segments.keys()):
            seg = self.segments[seg_id]
            if seg.start_vertex == vertex_id or seg.end_vertex == vertex_id:
                del self.segments[seg_id]
                removed += 1
        self.vertices.pop(vertex_id, None)
        return removed
    def find_vertex_near(self, x: float, y: float, radius: float) -> Optional[CreaseVertex]:
        best: Optional[CreaseVertex] = None
        best_dist = radius
        for vertex in self.vertices.values():
            dist = math.hypot(vertex.x - x, vertex.y - y)
            if dist <= best_dist:
                best = vertex
                best_dist = dist
        return best
    def find_segment_near(self, x: float, y: float, radius: float) -> Optional[CreaseSegment]:
        best: Optional[CreaseSegment] = None
        best_dist = radius
        for segment in self.segments.values():
            if not segment.visible:
                continue
            a = self.vertices.get(segment.start_vertex)
```

```
            b = self.vertices.get(segment.end_vertex)
            if a is None or b is None:
                continue
            dist = distance_point_to_segment(x, y, a.x, a.y, b.x, b.y)
            if dist <= best_dist:
                best = segment
                best_dist = dist
        return best
    def bounding_rect(self) -> QRectF:
        if not self.vertices:
            return QRectF(0.0, 0.0, 1.0, 1.0)
        xs = [v.x for v in self.vertices.values()]
        ys = [v.y for v in self.vertices.values()]
        return QRectF(min(xs), min(ys), max(xs) - min(xs), max(ys) - min(ys))
    def count_by_type(self) -> Dict[str, int]:
        result = {key: 0 for key in CREASE_TYPE_LABELS}
        for segment in self.segments.values():
            if segment.crease_type in result:
                result[segment.crease_type] += 1
        return result
    def total_length(self) -> float:
        return sum(seg.length(self.vertices) for seg in self.segments.values())
    def to_dict(self) -> dict:
        return {
            "name": self.name,
            "next_vertex_id": self.next_vertex_id,
            "next_segment_id": self.next_segment_id,
            "vertices": [
                {"id": v.vertex_id, "x": v.x, "y": v.y, "locked": v.locked}
                for v in self.vertices.values()
            ],
            "segments": [
                {
                    "id": s.segment_id,
                    "start": s.start_vertex,
                    "end": s.end_vertex,
                    "type": s.crease_type,
                    "layer": s.layer_id,
                    "color": s.color,
                    "width": s.width,
                    "visible": s.visible,
                }
                for s in self.segments.values()
            ],
        }
    @classmethod
    def from_dict(cls, data: dict) -> "CreasePattern":
        pattern = cls(name=data.get("name", "未命名折痕图"))
        for item in data.get("vertices", []):
            vertex = CreaseVertex(
                int(item["id"]), float(item["x"]), float(item["y"]), bool(item.get("locked", False))
            )
            pattern.vertices[vertex.vertex_id] = vertex
        for item in data.get("segments", []):
            segment = CreaseSegment(
                segment_id=int(item["id"]),
                start_vertex=int(item["start"]),
                end_vertex=int(item["end"]),
                crease_type=item.get("type", CREASE_TYPE_VALLEY),
                layer_id=int(item.get("layer", 0)),
                color=item.get("color", ""),
                width=float(item.get("width", DEFAULT_LINE_WIDTH)),
                visible=bool(item.get("visible", True)),
            )
            pattern.segments[segment.segment_id] = segment
        pattern.next_vertex_id = int(data.get("next_vertex_id", pattern._max_vertex_id() + 1))
        pattern.next_segment_id = int(data.get("next_segment_id", pattern._max_segment_id() + 1))
        return pattern
    def _max_vertex_id(self) -> int:
        return max(self.vertices.keys()) if self.vertices else 0
    def _max_segment_id(self) -> int:
        return max(self.segments.keys()) if self.segments else 0
class CreaseModelService:
    def __init__(self) -> None:
        self.pattern = CreasePattern()
        self.undo_stack = UndoStack()
    def reset(self, name: str = "未命名折痕图") -> None:
        self.pattern = CreasePattern(name=name)
        self.undo_stack.clear()
        logger.info("折痕图已重置: %s", name)
    def snapshot(self) -> dict:
        return self.pattern.to_dict()
    def push_undo(self, label: str) -> None:
```

```
        self.undo_stack.push(label, self.snapshot())
    def undo(self) -> bool:
        state = self.undo_stack.undo(self.snapshot())
        if state is None:
            return False
        self.pattern = CreasePattern.from_dict(state)
        return True
    def redo(self) -> bool:
        state = self.undo_stack.redo(self.snapshot())
        if state is None:
            return False
        self.pattern = CreasePattern.from_dict(state)
        return True
    def add_vertex(self, x: float, y: float, locked: bool = False) -> CreaseVertex:
        self.push_undo("添加顶点")
        return self.pattern.add_vertex(x, y, locked)
    def add_segment(
        self,
        start_vertex: int,
        end_vertex: int,
        crease_type: str = CREASE_TYPE_VALLEY,
        layer_id: int = 0,
    ) -> CreaseSegment:
        self.push_undo("添加折痕线段")
        return self.pattern.add_segment(start_vertex, end_vertex, crease_type, layer_id)
    def delete_segment(self, segment_id: int) -> bool:
        self.push_undo("删除折痕线段")
        return self.pattern.remove_segment(segment_id)
    def delete_vertex(self, vertex_id: int) -> int:
        self.push_undo("删除顶点")
        return self.pattern.remove_vertex(vertex_id)
    def move_vertex(self, vertex_id: int, x: float, y: float) -> bool:
        vertex = self.pattern.vertices.get(vertex_id)
        if vertex is None:
            return False
        self.push_undo("移动顶点")
        vertex.move_to(x, y)
        return True
    def set_segment_type(self, segment_id: int, crease_type: str) -> bool:
        segment = self.pattern.segments.get(segment_id)
        if segment is None or crease_type not in CREASE_TYPE_LABELS:
            return False
        self.push_undo("切换折痕类型")
        segment.crease_type = crease_type
        segment.color = crease_type_color(crease_type)
        return True
    def set_segment_style(
        self,
        segment_id: int,
        color: Optional[str] = None,
        width: Optional[float] = None,
        visible: Optional[bool] = None,
    ) -> bool:
        segment = self.pattern.segments.get(segment_id)
        if segment is None:
            return False
        self.push_undo("修改线条样式")
        if color is not None:
            segment.color = normalize_hex_color(color)
        if width is not None:
            segment.width = max(0.2, float(width))
        if visible is not None:
            segment.visible = bool(visible)
        return True
    def set_segment_layer(self, segment_id: int, layer_id: int) -> bool:
        segment = self.pattern.segments.get(segment_id)
        if segment is None:
            return False
        self.push_undo("调整图层归属")
        segment.layer_id = int(layer_id)
        return True
    def split_segment(self, segment_id: int, x: float, y: float) -> Optional[int]:
        segment = self.pattern.segments.get(segment_id)
        if segment is None:
            return None
        self.push_undo("拆分折痕线段")
        new_vertex = self.pattern.add_vertex(x, y)
        original_end = segment.end_vertex
        segment.end_vertex = new_vertex.vertex_id
        new_segment = self.pattern.add_segment(
            new_vertex.vertex_id, original_end, segment.crease_type, segment.layer_id
        )
```

```
        new_segment.color = segment.color
        new_segment.width = segment.width
        return new_segment.segment_id
    def statistics(self) -> Dict[str, float]:
        counts = self.pattern.count_by_type()
        stats: Dict[str, float] = {
          "顶点数": float(len(self.pattern.vertices)),
          "线段数": float(len(self.pattern.segments)),
          "总长度": round(self.pattern.total_length(), 3),
        }
        for key, label in CREASE_TYPE_LABELS.items():
            stats[label] = float(counts.get(key, 0))
        return stats
    def merge_vertices(self, source_id: int, target_id: int, tolerance: float = 1e-3) -> bool:
        source = self.pattern.vertices.get(source_id)
        target = self.pattern.vertices.get(target_id)
        if source is None or target is None or source_id == target_id:
            return False
        if math.hypot(source.x - target.x, source.y - target.y) > tolerance:
            return False
        self.push_undo("合并顶点")
        for segment in self.pattern.segments.values():
            if segment.start_vertex == source_id:
                segment.start_vertex = target_id
            if segment.end_vertex == source_id:
                segment.end_vertex = target_id
        self.pattern.vertices.pop(source_id, None)
        self._drop_degenerate_segments()
        return True
    def _drop_degenerate_segments(self) -> None:
        for seg_id in list(self.pattern.segments.keys()):
            seg = self.pattern.segments[seg_id]
            if seg.start_vertex == seg.end_vertex:
                del self.pattern.segments[seg_id]
    def auto_snap_vertex(self, x: float, y: float, radius: float = DEFAULT_SNAP_RADIUS) ->
Optional[CreaseVertex]:
        return self.pattern.find_vertex_near(x, y, radius)
    def segment_hit_test(self, x: float, y: float, radius: float = DEFAULT_SNAP_RADIUS) ->
Optional[CreaseSegment]:
        return self.pattern.find_segment_near(x, y, radius)
class CreaseEditorWidget(QWidget):
    def __init__(self, service: CreaseModelService, parent: Optional[QWidget] = None) -> None:
        super().__init__(parent)
        self.service = service
        self._current_type = CREASE_TYPE_VALLEY
        self._current_layer = 0
        self._snap_radius = DEFAULT_SNAP_RADIUS
        self._build_ui()
        self.refresh_segment_list()
    def _build_ui(self) -> None:
        layout = QVBoxLayout(self)
        tool_group = QGroupBox("绘制工具")
        tool_layout = QFormLayout(tool_group)
        self.type_combo = QComboBox()
        for key, label in CREASE_TYPE_LABELS.items():
            self.type_combo.addItem(label, key)
        self.type_combo.currentIndexChanged.connect(self._on_type_changed)
        tool_layout.addRow("折痕类型", self.type_combo)
        self.layer_spin = QSpinBox()
        self.layer_spin.setRange(0, 99)
        self.layer_spin.valueChanged.connect(self._on_layer_changed)
        tool_layout.addRow("图层编号", self.layer_spin)
        self.width_spin = QDoubleSpinBox()
        self.width_spin.setRange(0.2, 12.0)
        self.width_spin.setSingleStep(0.2)
        self.width_spin.setValue(DEFAULT_LINE_WIDTH)
        tool_layout.addRow("线宽", self.width_spin)
        self.snap_spin = QDoubleSpinBox()
        self.snap_spin.setRange(1.0, 40.0)
        self.snap_spin.setValue(DEFAULT_SNAP_RADIUS)
        self.snap_spin.valueChanged.connect(self._on_snap_changed)
        tool_layout.addRow("吸附半径", self.snap_spin)
        self.color_button = QToolButton()
        self.color_button.setText("线条颜色")
        self.color_button.clicked.connect(self._pick_color)
        self._custom_color = crease_type_color(CREASE_TYPE_VALLEY)
        tool_layout.addRow("自定义颜色", self.color_button)
        layout.addWidget(tool_group)
        action_group = QGroupBox("编辑操作")
        action_layout = QHBoxLayout(action_group)
        self.undo_button = QPushButton("撤销")
        self.undo_button.clicked.connect(self._on_undo)
```

```
        self.redo_button = QPushButton("重做")
        self.redo_button.clicked.connect(self._on_redo)
        self.delete_button = QPushButton("删除选中")
        self.delete_button.clicked.connect(self._on_delete_selected)
        self.merge_button = QPushButton("合并顶点")
        self.merge_button.clicked.connect(self._on_merge_vertices)
        action_layout.addWidget(self.undo_button)
        action_layout.addWidget(self.redo_button)
        action_layout.addWidget(self.delete_button)
        action_layout.addWidget(self.merge_button)
        layout.addWidget(action_group)
        list_group = QGroupBox("折痕线段列表")
        list_layout = QVBoxLayout(list_group)
        self.segment_list = QListWidget()
        self.segment_list.itemSelectionChanged.connect(self._on_selection_changed)
        list_layout.addWidget(self.segment_list)
        layout.addWidget(list_group)
        self.stats_label = QLabel("统计信息：暂无数据")
        self.stats_label.setWordWrap(True)
        layout.addWidget(self.stats_label)
        layout.addStretch(1)
    def _on_type_changed(self, index: int) -> None:
        self._current_type = self.type_combo.itemData(index)
        self._custom_color = crease_type_color(self._current_type)
        self._apply_type_to_selection()
    def _on_layer_changed(self, value: int) -> None:
        self._current_layer = value
        self._apply_layer_to_selection()
    def _on_snap_changed(self, value: float) -> None:
        self._snap_radius = float(value)
    def _pick_color(self) -> None:
        color = QColorDialog.getColor(QColor(self._custom_color), self, "选择折痕线条颜色")
        if color.isValid():
            self._custom_color = color.name()
            self._apply_style_to_selection()
    def _apply_type_to_selection(self) -> None:
        for segment_id in self.selected_segment_ids():
            self.service.set_segment_type(segment_id, self._current_type)
        self.refresh_segment_list()
    def _apply_layer_to_selection(self) -> None:
        for segment_id in self.selected_segment_ids():
            self.service.set_segment_layer(segment_id, self._current_layer)
        self.refresh_segment_list()
    def _apply_style_to_selection(self) -> None:
        for segment_id in self.selected_segment_ids():
            self.service.set_segment_style(
                segment_id, color=self._custom_color, width=self.width_spin.value()
            )
        self.refresh_segment_list()
    def selected_segment_ids(self) -> List[int]:
        ids: List[int] = []
        for item in self.segment_list.selectedItems():
            ids.append(int(item.data(Qt.UserRole)))
        return ids
    def _on_selection_changed(self) -> None:
        ids = self.selected_segment_ids()
        if len(ids) == 1:
            segment = self.service.pattern.segments.get(ids[0])
            if segment is not None:
                self.width_spin.setValue(segment.width)
                self._custom_color = segment.color or crease_type_color(segment.crease_type)
    def _on_undo(self) -> None:
        if not self.service.undo():
            QMessageBox.information(self, "撤销", "没有可撤销的操作")
        self.refresh_segment_list()
    def _on_redo(self) -> None:
        if not self.service.redo():
            QMessageBox.information(self, "重做", "没有可重做的操作")
        self.refresh_segment_list()
    def _on_delete_selected(self) -> None:
        ids = self.selected_segment_ids()
        if not ids:
            QMessageBox.warning(self, "删除", "请先在列表中选择折痕线段")
            return
        for segment_id in ids:
            self.service.delete_segment(segment_id)
        self.refresh_segment_list()
    def _on_merge_vertices(self) -> None:
        ids = self.selected_segment_ids()
        if len(ids) != 2:
            QMessageBox.warning(self, "合并顶点", "请选择两条相邻折痕线段")
            return
```

```
        first = self.service.pattern.segments.get(ids[0])
        second = self.service.pattern.segments.get(ids[1])
        if first is None or second is None:
            return
        shared = self._shared_vertex(first, second)
        if shared is None:
            QMessageBox.warning(self, "合并顶点", "两条线段没有公共顶点")
            return
        other = first.end_vertex if first.start_vertex == shared else first.start_vertex
        if self.service.merge_vertices(other, shared):
            self.refresh_segment_list()
        else:
            QMessageBox.warning(self, "合并顶点", "顶点距离超出容差，无法合并")
    @staticmethod
    def _shared_vertex(a: CreaseSegment, b: CreaseSegment) -> Optional[int]:
        a_points = {a.start_vertex, a.end_vertex}
        b_points = {b.start_vertex, b.end_vertex}
        common = a_points & b_points
        return next(iter(common)) if common else None
    def refresh_segment_list(self) -> None:
        self.segment_list.blockSignals(True)
        self.segment_list.clear()
        for segment in self.service.pattern.segments.values():
            label = CREASE_TYPE_LABELS.get(segment.crease_type, segment.crease_type)
            length = segment.length(self.service.pattern.vertices)
            item = QListWidgetItem(
                f"
            )
            item.setData(Qt.UserRole, segment.segment_id)
            if segment.color:
                item.setForeground(QBrush(QColor(segment.color)))
            self.segment_list.addItem(item)
        self.segment_list.blockSignals(False)
        self.refresh_statistics()
    def refresh_statistics(self) -> None:
        stats = self.service.statistics()
        parts = [f"{key}: {int(value) if value == int(value) else value}" for key, value in stats.items()]
        self.stats_label.setText("统计信息" + " | ".join(parts))
    def current_crease_type(self) -> str:
        return self._current_type
    def current_layer(self) -> int:
        return self._current_layer
    def snap_radius(self) -> float:
        return self._snap_radius
    def current_color(self) -> str:
        return self._custom_color
    def current_width(self) -> float:
        return self.width_spin.value()
    def draw_pattern(self, painter, transform) -> None:
        pattern = self.service.pattern
        for segment in pattern.segments.values():
            if not segment.visible:
                continue
            a = pattern.vertices.get(segment.start_vertex)
            b = pattern.vertices.get(segment.end_vertex)
            if a is None or b is None:
                continue
            pen = QPen(QColor(segment.color or crease_type_color(segment.crease_type)))
            pen.setWidthF(segment.width)
            pen.setCapStyle(Qt.RoundCap)
            if segment.crease_type == CREASE_TYPE_AUXILIARY:
                pen.setStyle(Qt.DashLine)
            elif segment.crease_type == CREASE_TYPE_CUT:
                pen.setStyle(Qt.DotLine)
            painter.setPen(pen)
            painter.drawLine(transform(a.as_point()), transform(b.as_point()))
    def build_path(self, transform) -> QPainterPath:
        path = QPainterPath()
        pattern = self.service.pattern
        for segment in pattern.segments.values():
            a = pattern.vertices.get(segment.start_vertex)
            b = pattern.vertices.get(segment.end_vertex)
            if a is None or b is None:
                continue
            path.moveTo(transform(a.as_point()))
            path.lineTo(transform(b.as_point()))
        return path
    def hit_test_vertex(self, x: float, y: float) -> Optional[CreaseVertex]:
        return self.service.auto_snap_vertex(x, y, self._snap_radius)
    def hit_test_segment(self, x: float, y: float) -> Optional[CreaseSegment]:
        return self.service.segment_hit_test(x, y, self._snap_radius)
    def create_segment_between(self, start_id: int, end_id: int) -> Optional[CreaseSegment]:
        try:
```

```
            segment = self.service.add_segment(
                start_id, end_id, self._current_type, self._current_layer
            )
        except (ValueError, KeyError) as exc:
            logger.warning("创建折痕线段失败: %s", exc)
            return None
        self.refresh_segment_list()
        return segment
    def create_vertex_at(self, x: float, y: float) -> CreaseVertex:
        vertex = self.service.add_vertex(x, y)
        self.refresh_statistics()
        return vertex
    def move_vertex_to(self, vertex_id: int, x: float, y: float) -> bool:
        moved = self.service.move_vertex(vertex_id, x, y)
        if moved:
            self.refresh_segment_list()
        return moved
    def split_segment_at(self, segment_id: int, x: float, y: float) -> Optional[int]:
        new_id = self.service.split_segment(segment_id, x, y)
        self.refresh_segment_list()
        return new_id
    def toggle_segment_visibility(self, segment_id: int) -> bool:
        segment = self.service.pattern.segments.get(segment_id)
        if segment is None:
            return False
        changed = self.service.set_segment_style(segment_id, visible=not segment.visible)
        self.refresh_segment_list()
        return changed
    def load_pattern(self, pattern: CreasePattern) -> None:
        self.service.pattern = pattern
        self.service.undo_stack.clear()
        self.refresh_segment_list()
    def export_pattern_dict(self) -> dict:
        return self.service.snapshot()
    def validate_geometry(self) -> List[str]:
        issues: List[str] = []
        pattern = self.service.pattern
        for segment in pattern.segments.values():
            length = segment.length(pattern.vertices)
            if length < MIN_SEGMENT_LENGTH:
                issues.append(f"线段
        for vertex in pattern.vertices.values():
            degree = sum(
                1
                for seg in pattern.segments.values()
                if seg.start_vertex == vertex.vertex_id or seg.end_vertex == vertex.vertex_id
            )
            if degree == 1 and not vertex.locked:
                issues.append(f"顶点
        return issues
    def polygon_area(self, vertex_ids: List[int]) -> float:
        points = [
            (self.service.pattern.vertices[vid].x, self.service.pattern.vertices[vid].y)
            for vid in vertex_ids
            if vid in self.service.pattern.vertices
        ]
        if len(points) < 3:
            return 0.0
        area = 0.0
        for index in range(len(points)):
            x1, y1 = points[index]
            x2, y2 = points[(index + 1) % len(points)]
            area += x1 * y2 - x2 * y1
        return abs(area) / 2.0
    def point_inside_face(self, x: float, y: float, vertex_ids: List[int]) -> bool:
        polygon = [
            (self.service.pattern.vertices[vid].x, self.service.pattern.vertices[vid].y)
            for vid in vertex_ids
            if vid in self.service.pattern.vertices
        ]
        if len(polygon) < 3:
            return False
        return point_in_polygon(x, y, polygon)
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/ui/__init__.py
from app.common import logger
__all__ = [
    "canvas_widget",
    "crease_editor_panel",
    "layer_panel",
    "preview_widget",
```

```
    "validation_panel",
    "export_dialog",
    "settings_dialog",
    "about_dialog",
    "main_window",
]
_UI_MODULE_PATHS = {
    "canvas_widget": "app.ui.canvas_widget",
    "crease_editor_panel": "app.ui.crease_editor_panel",
    "layer_panel": "app.ui.layer_panel",
    "preview_widget": "app.ui.preview_widget",
    "validation_panel": "app.ui.validation_panel",
    "export_dialog": "app.ui.export_dialog",
    "settings_dialog": "app.ui.settings_dialog",
    "about_dialog": "app.ui.about_dialog",
    "main_window": "app.ui.main_window",
}
_UI_CLASS_NAMES = {
    "canvas_widget": "CanvasWidget",
    "crease_editor_panel": "CreaseEditorPanel",
    "layer_panel": "LayerPanel",
    "preview_widget": "PreviewWidget",
    "validation_panel": "ValidationPanel",
    "export_dialog": "ExportDialog",
    "settings_dialog": "SettingsDialog",
    "about_dialog": "AboutDialog",
    "main_window": "MainWindow",
}
_loaded_cache: dict = {}
def load_ui_component(component_key: str):
    if component_key in _loaded_cache:
        return _loaded_cache[component_key]
    module_path = _UI_MODULE_PATHS.get(component_key)
    class_name = _UI_CLASS_NAMES.get(component_key)
    if module_path is None or class_name is None:
        raise KeyError(f"未知界面组件: {component_key}")
    try:
        import importlib
        module = importlib.import_module(module_path)
        component = getattr(module, class_name)
    except (ImportError, AttributeError) as exc:
        logger.error("加载界面组件失败 %s: %s", component_key, exc)
        raise
    _loaded_cache[component_key] = component
    return component
def available_components() -> list:
    return list(_UI_MODULE_PATHS.keys())
def is_component_available(component_key: str) -> bool:
    return component_key in _UI_MODULE_PATHS
def clear_component_cache() -> None:
    _loaded_cache.clear()
    logger.info("界面组件缓存已清空")
def describe_ui_package() -> dict:
    return {
        "package": "app.ui",
        "system_name": "折痕图可视化编辑与标准化导出系统",
        "component_count": len(_UI_MODULE_PATHS),
        "components": available_components(),
    }
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/ui/about_dialog.py
from PySide6.QtCore import Qt, QSize
from PySide6.QtGui import QPainter, QColor, QPen, QFont
from PySide6.QtWidgets import QDialog, QVBoxLayout, QHBoxLayout, QLabel, QTabWidget, QWidget, QTextBrowser,
QGroupBox, QFormLayout, QDialogButtonBox, QFrame
from app.common import logger
from app.config import APP_NAME, APP_VERSION, APP_AUTHOR, APP_COPYRIGHT, APP_DESCRIPTION
class LogoWidget(QWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setFixedSize(QSize(96, 96))
    def paintEvent(self, event):
        painter = QPainter(self)
        painter.setRenderHint(QPainter.Antialiasing)
        painter.fillRect(self.rect(), QColor("
        pen_peak = QPen(QColor("
        pen_valley = QPen(QColor("
        pen_edge = QPen(QColor("
        painter.setPen(pen_edge)
        painter.drawRect(8, 8, 80, 80)
        painter.setPen(pen_peak)
```

```
        painter.drawLine(8, 8, 88, 88)
        painter.setPen(pen_valley)
        painter.drawLine(88, 8, 8, 88)
        painter.setPen(pen_edge)
        painter.drawLine(48, 8, 48, 88)
        painter.drawLine(8, 48, 88, 48)
        painter.end()
class AboutDialog(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle(f"关于 {APP_NAME}")
        self.setMinimumSize(560, 460)
        self.setModal(True)
        self._build_ui()
    def _build_ui(self):
        root_layout = QVBoxLayout(self)
        root_layout.setContentsMargins(16, 16, 16, 16)
        root_layout.setSpacing(12)
        root_layout.addWidget(self._build_header())
        line = QFrame()
        line.setFrameShape(QFrame.HLine)
        line.setFrameShadow(QFrame.Sunken)
        root_layout.addWidget(line)
        tabs = QTabWidget(self)
        tabs.addTab(self._build_info_tab(), "基本信息")
        tabs.addTab(self._build_feature_tab(), "功能说明")
        tabs.addTab(self._build_history_tab(), "版本历史")
        tabs.addTab(self._build_license_tab(), "许可声明")
        root_layout.addWidget(tabs, 1)
        button_box = QDialogButtonBox(QDialogButtonBox.Ok, self)
        button_box.button(QDialogButtonBox.Ok).setText("确定")
        button_box.accepted.connect(self.accept)
        root_layout.addWidget(button_box)
    def _build_header(self) -> QWidget:
        header = QWidget(self)
        layout = QHBoxLayout(header)
        layout.setContentsMargins(0, 0, 0, 0)
        layout.setSpacing(16)
        logo = LogoWidget(header)
        layout.addWidget(logo, 0, Qt.AlignTop)
        text_layout = QVBoxLayout()
        text_layout.setSpacing(6)
        title_label = QLabel(APP_NAME, header)
        title_font = QFont()
        title_font.setPointSize(15)
        title_font.setBold(True)
        title_label.setFont(title_font)
        text_layout.addWidget(title_label)
        version_label = QLabel(f"版本 {APP_VERSION}", header)
        version_label.setStyleSheet("color:
        text_layout.addWidget(version_label)
        desc_label = QLabel(APP_DESCRIPTION, header)
        desc_label.setWordWrap(True)
        desc_label.setStyleSheet("color:
        text_layout.addWidget(desc_label)
        text_layout.addStretch(1)
        layout.addLayout(text_layout, 1)
        return header
    def _build_info_tab(self) -> QWidget:
        page = QWidget(self)
        layout = QVBoxLayout(page)
        layout.setContentsMargins(12, 12, 12, 12)
        group = QGroupBox("软件信息", page)
        form = QFormLayout(group)
        form.setLabelAlignment(Qt.AlignRight)
        form.setHorizontalSpacing(16)
        form.setVerticalSpacing(10)
        form.addRow("软件名称：", QLabel(APP_NAME, group))
        form.addRow("当前版本：", QLabel(APP_VERSION, group))
        form.addRow("开发单位：", QLabel(APP_AUTHOR, group))
        form.addRow("运行平台：", QLabel("Windows 11 / Windows 7", group))
        form.addRow("运行环境：", QLabel("Python 3 + SQLite 3.x", group))
        form.addRow("版权声明：", QLabel(APP_COPYRIGHT, group))
        layout.addWidget(group)
        tech_group = QGroupBox("技术栈", page)
        tech_layout = QVBoxLayout(tech_group)
        tech_text = QLabel(
          "界面框架：PySide6\n"
          "数据存储：SQLite 本地嵌入式数据库\n"
          "导出格式：SVG / DXF / FOLD\n"
          "几何计算：内置折痕几何与约束校验引擎",
            tech_group,
```

```
        )
        tech_text.setStyleSheet("color:
        tech_layout.addWidget(tech_text)
        layout.addWidget(tech_group)
        layout.addStretch(1)
        return page
    def _build_feature_tab(self) -> QWidget:
        page = QWidget(self)
        layout = QVBoxLayout(page)
        layout.setContentsMargins(12, 12, 12, 12)
        browser = QTextBrowser(page)
        browser.setOpenExternalLinks(False)
        browser.setHtml(
        )
        layout.addWidget(browser)
        return page
    def _build_history_tab(self) -> QWidget:
        page = QWidget(self)
        layout = QVBoxLayout(page)
        layout.setContentsMargins(12, 12, 12, 12)
        browser = QTextBrowser(page)
        browser.setHtml(
        )
        layout.addWidget(browser)
        return page
    def _build_license_tab(self) -> QWidget:
        page = QWidget(self)
        layout = QVBoxLayout(page)
        layout.setContentsMargins(12, 12, 12, 12)
        browser = QTextBrowser(page)
        browser.setHtml(
            f
        )
        layout.addWidget(browser)
        return page
    def showEvent(self, event):
        super().showEvent(event)
        logger.info("关于窗口已打开")
# 文件: app/ui/canvas_widget.py
import math
from typing import Optional
from PySide6.QtCore import Qt, QPointF, QRectF, Signal
from PySide6.QtGui import (
    QPainter,
    QPen,
    QBrush,
    QColor,
    QMouseEvent,
    QWheelEvent,
    QKeyEvent,
    QPaintEvent,
    QResizeEvent,
)
from PySide6.QtWidgets import QWidget, QMenu, QInputDialog
from app.common import logger
from app.core.geometry import (
    distance_point_to_segment,
    point_in_polygon,
    snap_to_grid,
)
from app.utils.color_utils import crease_type_color
CREASE_TYPE_PEAK = "peak"
CREASE_TYPE_VALLEY = "valley"
CREASE_TYPE_FLAT = "flat"
CREASE_TYPE_CUT = "cut"
CREASE_TYPE_BOUNDARY = "boundary"
CREASE_TYPE_AUXILIARY = "auxiliary"
TOOL_SELECT = "select"
TOOL_DRAW_PEAK = "draw_peak"
TOOL_DRAW_VALLEY = "draw_valley"
TOOL_DRAW_FLAT = "draw_flat"
TOOL_DRAW_CUT = "draw_cut"
TOOL_DRAW_BOUNDARY = "draw_boundary"
TOOL_DRAW_AUXILIARY = "draw_auxiliary"
TOOL_ERASE = "erase"
DRAW_TOOL_TO_TYPE = {
    TOOL_DRAW_PEAK: CREASE_TYPE_PEAK,
    TOOL_DRAW_VALLEY: CREASE_TYPE_VALLEY,
    TOOL_DRAW_FLAT: CREASE_TYPE_FLAT,
    TOOL_DRAW_CUT: CREASE_TYPE_CUT,
    TOOL_DRAW_BOUNDARY: CREASE_TYPE_BOUNDARY,
    TOOL_DRAW_AUXILIARY: CREASE_TYPE_AUXILIARY,
}
```

```
HIT_TOLERANCE_PX = 8.0
VERTEX_HIT_RADIUS_PX = 7.0
MIN_ZOOM = 0.15
MAX_ZOOM = 12.0
class CanvasWidget(QWidget):
    crease_added = Signal(dict)
    crease_removed = Signal(int)
    crease_modified = Signal(int, dict)
    vertex_moved = Signal(int, float, float)
    selection_changed = Signal(object)
    status_message = Signal(str)
    def __init__(self, parent: Optional[QWidget] = None) -> None:
        super().__init__(parent)
        self.setMouseTracking(True)
        self.setFocusPolicy(Qt.StrongFocus)
        self.setMinimumSize(480, 360)
        self._creases: list[dict] = []
        self._vertices: list[dict] = []
        self._layers: list[dict] = []
        self._zoom = 1.0
        self._offset = QPointF(0.0, 0.0)
        self._grid_size = 20.0
        self._show_grid = True
        self._snap_enabled = True
        self._show_vertices = True
        self._current_tool = TOOL_SELECT
        self._selected_crease_id: Optional[int] = None
        self._selected_vertex_id: Optional[int] = None
        self._hover_crease_id: Optional[int] = None
        self._drawing = False
        self._draw_start = QPointF()
        self._draw_current = QPointF()
        self._dragging_vertex = False
        self._panning = False
        self._pan_anchor = QPointF()
        self._next_crease_id = 1
        self._next_vertex_id = 1
    def set_creases(self, creases: list[dict]) -> None:
        self._creases = list(creases)
        max_id = 0
        for item in self._creases:
            cid = int(item.get("id", 0))
            if cid > max_id:
                max_id = cid
        self._next_crease_id = max_id + 1
        self.update()
    def set_vertices(self, vertices: list[dict]) -> None:
        self._vertices = list(vertices)
        max_id = 0
        for item in self._vertices:
            vid = int(item.get("id", 0))
            if vid > max_id:
                max_id = vid
        self._next_vertex_id = max_id + 1
        self.update()
    def set_layers(self, layers: list[dict]) -> None:
        self._layers = list(layers)
        self.update()
    def creases(self) -> list[dict]:
        return list(self._creases)
    def vertices(self) -> list[dict]:
        return list(self._vertices)
    def set_tool(self, tool: str) -> None:
        if tool not in DRAW_TOOL_TO_TYPE and tool not in (TOOL_SELECT, TOOL_ERASE):
            logger.warning("未知工具: %s", tool)
            return
        self._current_tool = tool
        self.setCursor(Qt.CrossCursor if tool != TOOL_SELECT else Qt.ArrowCursor)
        self.status_message.emit(f"当前工具: {tool}")
    def current_tool(self) -> str:
        return self._current_tool
    def set_grid_size(self, size: float) -> None:
        self._grid_size = max(2.0, float(size))
        self.update()
    def set_show_grid(self, enabled: bool) -> None:
        self._show_grid = bool(enabled)
        self.update()
    def set_snap_enabled(self, enabled: bool) -> None:
        self._snap_enabled = bool(enabled)
    def set_show_vertices(self, enabled: bool) -> None:
        self._show_vertices = bool(enabled)
        self.update()
    def selected_crease_id(self) -> Optional[int]:
```

```
        return self._selected_crease_id
    def clear_selection(self) -> None:
        self._selected_crease_id = None
        self._selected_vertex_id = None
        self.selection_changed.emit(None)
        self.update()
    def select_crease(self, crease_id: Optional[int]) -> None:
        self._selected_crease_id = crease_id
        self._selected_vertex_id = None
        self.selection_changed.emit(crease_id)
        self.update()
    def update_crease(self, crease_id: int, changes: dict) -> None:
        for item in self._creases:
            if int(item.get("id", -1)) == crease_id:
                item.update(changes)
                self.update()
                return
    def remove_crease(self, crease_id: int) -> None:
        before = len(self._creases)
        self._creases = [c for c in self._creases if int(c.get("id", -1)) != crease_id]
        if len(self._creases) != before:
            if self._selected_crease_id == crease_id:
                self._selected_crease_id = None
                self.selection_changed.emit(None)
            self.update()
    def add_crease(self, crease: dict) -> int:
        crease_id = int(crease.get("id") or self._next_crease_id)
        self._next_crease_id = max(self._next_crease_id, crease_id) + 1
        crease["id"] = crease_id
        self._creases.append(crease)
        self.update()
        return crease_id
    def add_vertex(self, vertex: dict) -> int:
        vertex_id = int(vertex.get("id") or self._next_vertex_id)
        self._next_vertex_id = max(self._next_vertex_id, vertex_id) + 1
        vertex["id"] = vertex_id
        self._vertices.append(vertex)
        self.update()
        return vertex_id
    def reset_view(self) -> None:
        self._zoom = 1.0
        self._offset = QPointF(0.0, 0.0)
        self.update()
    def zoom_in(self) -> None:
        self._apply_zoom(1.2, self.rect().center())
    def zoom_out(self) -> None:
        self._apply_zoom(1.0 / 1.2, self.rect().center())
    def _to_screen(self, point: QPointF) -> QPointF:
        return QPointF(
            point.x() * self._zoom + self._offset.x(),
            point.y() * self._zoom + self._offset.y(),
        )
    def _to_world(self, point: QPointF) -> QPointF:
        return QPointF(
            (point.x() - self._offset.x()) / self._zoom,
            (point.y() - self._offset.y()) / self._zoom,
        )
    def _apply_zoom(self, factor: float, anchor: QPointF) -> None:
        new_zoom = max(MIN_ZOOM, min(MAX_ZOOM, self._zoom * factor))
        if abs(new_zoom - self._zoom) < 1e-6:
            return
        world_anchor = self._to_world(anchor)
        self._zoom = new_zoom
        self._offset = QPointF(
            anchor.x() - world_anchor.x() * self._zoom,
            anchor.y() - world_anchor.y() * self._zoom,
        )
        self.update()
    def _snap(self, point: QPointF) -> QPointF:
        if not self._snap_enabled:
            return point
        return snap_to_grid(point, self._grid_size)
    def paintEvent(self, event: QPaintEvent) -> None:
        painter = QPainter(self)
        painter.setRenderHint(QPainter.Antialiasing, True)
        painter.fillRect(self.rect(), QColor(250, 250, 250))
        if self._show_grid:
            self._draw_grid(painter)
        self._draw_creases(painter)
        if self._show_vertices:
            self._draw_vertices(painter)
        if self._drawing:
            self._draw_preview_line(painter)
```

```
        painter.end()
    def _draw_grid(self, painter: QPainter) -> None:
        step = self._grid_size * self._zoom
        if step < 4.0:
            return
        pen = QPen(QColor(225, 228, 232))
        pen.setWidth(1)
        painter.setPen(pen)
        start_x = self._offset.x() % step
        x = start_x
        while x < self.width():
            painter.drawLine(int(x), 0, int(x), self.height())
            x += step
        start_y = self._offset.y() % step
        y = start_y
        while y < self.height():
            painter.drawLine(0, int(y), self.width(), int(y))
            y += step
    def _draw_creases(self, painter: QPainter) -> None:
        for crease in self._creases:
            crease_id = int(crease.get("id", -1))
            color = QColor(crease.get("color") or crease_type_color(crease.get("type", CREASE_TYPE_VALLEY)))
            width = float(crease.get("width", 1.6))
            pen = QPen(color)
            pen.setWidthF(width)
            pen.setCapStyle(Qt.RoundCap)
            crease_type = crease.get("type", CREASE_TYPE_VALLEY)
            if crease_type == CREASE_TYPE_PEAK:
                pen.setStyle(Qt.DashDotLine)
            elif crease_type == CREASE_TYPE_VALLEY:
                pen.setStyle(Qt.SolidLine)
            elif crease_type == CREASE_TYPE_FLAT:
                pen.setStyle(Qt.DotLine)
            elif crease_type == CREASE_TYPE_CUT:
                pen.setStyle(Qt.DashLine)
            elif crease_type == CREASE_TYPE_BOUNDARY:
                pen.setStyle(Qt.SolidLine)
                pen.setWidthF(max(width, 2.0))
            elif crease_type == CREASE_TYPE_AUXILIARY:
                pen.setStyle(Qt.DotLine)
                color.setAlpha(140)
                pen.setColor(color)
            if crease_id == self._selected_crease_id:
                pen.setColor(QColor(255, 140, 0))
                pen.setWidthF(width + 1.4)
            elif crease_id == self._hover_crease_id:
                pen.setWidthF(width + 0.8)
            p1 = self._to_screen(QPointF(float(crease.get("x1", 0.0)), float(crease.get("y1", 0.0))))
            p2 = self._to_screen(QPointF(float(crease.get("x2", 0.0)), float(crease.get("y2", 0.0))))
            painter.setPen(pen)
            painter.drawLine(p1, p2)
    def _draw_vertices(self, painter: QPainter) -> None:
        for vertex in self._vertices:
            vid = int(vertex.get("id", -1))
            pos = self._to_screen(QPointF(float(vertex.get("x", 0.0)), float(vertex.get("y", 0.0))))
            radius = VERTEX_HIT_RADIUS_PX
            if vid == self._selected_vertex_id:
                painter.setBrush(QBrush(QColor(255, 140, 0)))
                painter.setPen(QPen(QColor(180, 90, 0), 1.5))
            else:
                painter.setBrush(QBrush(QColor(255, 255, 255)))
                painter.setPen(QPen(QColor(90, 110, 140), 1.2))
            painter.drawEllipse(pos, radius, radius)
    def _draw_preview_line(self, painter: QPainter) -> None:
        pen = QPen(QColor(60, 120, 200))
        pen.setWidthF(1.4)
        pen.setStyle(Qt.DashLine)
        painter.setPen(pen)
        painter.drawLine(self._to_screen(self._draw_start), self._to_screen(self._draw_current))
    def _hit_crease(self, screen_pos: QPointF) -> Optional[int]:
        world = self._to_world(screen_pos)
        tolerance = HIT_TOLERANCE_PX / max(self._zoom, 1e-6)
        best_id: Optional[int] = None
        best_dist = tolerance
        for crease in self._creases:
            p1 = (float(crease.get("x1", 0.0)), float(crease.get("y1", 0.0)))
            p2 = (float(crease.get("x2", 0.0)), float(crease.get("y2", 0.0)))
            dist = distance_point_to_segment((world.x(), world.y()), p1, p2)
            if dist < best_dist:
                best_dist = dist
                best_id = int(crease.get("id", -1))
        return best_id
    def _hit_vertex(self, screen_pos: QPointF) -> Optional[int]:
```

```
        for vertex in self._vertices:
            pos = self._to_screen(QPointF(float(vertex.get("x", 0.0)), float(vertex.get("y", 0.0))))
            dx = pos.x() - screen_pos.x()
            dy = pos.y() - screen_pos.y()
            if math.hypot(dx, dy) <= VERTEX_HIT_RADIUS_PX:
                return int(vertex.get("id", -1))
        return None
    def _find_vertex_at(self, world_point: QPointF) -> Optional[int]:
        for vertex in self._vertices:
            vx = float(vertex.get("x", 0.0))
            vy = float(vertex.get("y", 0.0))
            if math.hypot(vx - world_point.x(), vy - world_point.y()) <= self._grid_size * 0.5:
                return int(vertex.get("id", -1))
        return None
    def mousePressEvent(self, event: QMouseEvent) -> None:
        pos = event.position()
        if event.button() == Qt.MiddleButton:
            self._panning = True
            self._pan_anchor = pos
            self.setCursor(Qt.ClosedHandCursor)
            return
        if event.button() != Qt.LeftButton:
            return
        if self._current_tool == TOOL_SELECT:
            vertex_id = self._hit_vertex(pos)
            if vertex_id is not None:
                self._selected_vertex_id = vertex_id
                self._selected_crease_id = None
                self._dragging_vertex = True
                self.selection_changed.emit(vertex_id)
                self.update()
                return
            crease_id = self._hit_crease(pos)
            self._selected_crease_id = crease_id
            self._selected_vertex_id = None
            self.selection_changed.emit(crease_id)
            self.update()
            return
        if self._current_tool == TOOL_ERASE:
            crease_id = self._hit_crease(pos)
            if crease_id is not None:
                self.remove_crease(crease_id)
                self.crease_removed.emit(crease_id)
                self.status_message.emit(f"已删除折痕
            return
        if self._current_tool in DRAW_TOOL_TO_TYPE:
            world = self._snap(self._to_world(pos))
            self._drawing = True
            self._draw_start = world
            self._draw_current = world
            self.update()
    def mouseMoveEvent(self, event: QMouseEvent) -> None:
        pos = event.position()
        if self._panning:
            delta = pos - self._pan_anchor
            self._offset += delta
            self._pan_anchor = pos
            self.update()
            return
        if self._dragging_vertex and self._selected_vertex_id is not None:
            world = self._snap(self._to_world(pos))
            for vertex in self._vertices:
                if int(vertex.get("id", -1)) == self._selected_vertex_id:
                    vertex["x"] = world.x()
                    vertex["y"] = world.y()
                    break
            self.vertex_moved.emit(self._selected_vertex_id, world.x(), world.y())
            self.update()
            return
        if self._drawing:
            self._draw_current = self._snap(self._to_world(pos))
            self.update()
            return
        hover_id = self._hit_crease(pos)
        if hover_id != self._hover_crease_id:
            self._hover_crease_id = hover_id
            self.update()
        world = self._to_world(pos)
        self.status_message.emit(f"坐标: ({world.x():.1f}, {world.y():.1f})")
    def mouseReleaseEvent(self, event: QMouseEvent) -> None:
        if event.button() == Qt.MiddleButton and self._panning:
            self._panning = False
            self.setCursor(Qt.CrossCursor if self._current_tool != TOOL_SELECT else Qt.ArrowCursor)
```

```
            return
        if event.button() != Qt.LeftButton:
            return
        if self._dragging_vertex:
            self._dragging_vertex = False
            return
        if self._drawing:
            self._drawing = False
            end_point = self._snap(self._to_world(event.position()))
            start_point = self._draw_start
            if math.hypot(end_point.x() - start_point.x(), end_point.y() - start_point.y()) < 1e-3:
                self.update()
                return
            crease_type = DRAW_TOOL_TO_TYPE.get(self._current_tool, CREASE_TYPE_VALLEY)
            crease = {
                "id": self._next_crease_id,
                "type": crease_type,
                "x1": start_point.x(),
                "y1": start_point.y(),
                "x2": end_point.x(),
                "y2": end_point.y(),
                "width": 1.6,
                "layer_id": self._active_layer_id(),
            }
            crease_id = self.add_crease(crease)
            self.crease_added.emit(dict(crease))
            self.status_message.emit(f"新增折痕
            self.update()
    def mouseDoubleClickEvent(self, event: QMouseEvent) -> None:
        if event.button() != Qt.LeftButton:
            return
        crease_id = self._hit_crease(event.position())
        if crease_id is None:
            return
        crease = self._find_crease(crease_id)
        if crease is None:
            return
        self._show_crease_menu(crease, event.globalPosition().toPoint())
    def wheelEvent(self, event: QWheelEvent) -> None:
        delta = event.angleDelta().y()
        if delta == 0:
            return
        factor = 1.15 if delta > 0 else 1.0 / 1.15
        self._apply_zoom(factor, event.position())
    def keyPressEvent(self, event: QKeyEvent) -> None:
        key = event.key()
        if key == Qt.Key_Delete and self._selected_crease_id is not None:
            crease_id = self._selected_crease_id
            self.remove_crease(crease_id)
            self.crease_removed.emit(crease_id)
            return
        if key == Qt.Key_Escape:
            self._drawing = False
            self.clear_selection()
            return
        if key == Qt.Key_Plus or key == Qt.Key_Equal:
            self.zoom_in()
            return
        if key == Qt.Key_Minus:
            self.zoom_out()
            return
        if key == Qt.Key_0:
            self.reset_view()
            return
        super().keyPressEvent(event)
    def resizeEvent(self, event: QResizeEvent) -> None:
        super().resizeEvent(event)
        self.update()
    def contextMenuEvent(self, event) -> None:
        crease_id = self._hit_crease(QPointF(event.pos()))
        if crease_id is None:
            return
        crease = self._find_crease(crease_id)
        if crease is None:
            return
        self._show_crease_menu(crease, event.globalPos())
    def _find_crease(self, crease_id: int) -> Optional[dict]:
        for crease in self._creases:
            if int(crease.get("id", -1)) == crease_id:
                return crease
        return None
    def _active_layer_id(self) -> Optional[int]:
        for layer in self._layers:
```

```
            if layer.get("visible", True):
                return int(layer.get("id", 0))
        return None
    def _show_crease_menu(self, crease: dict, global_pos) -> None:
        menu = QMenu(self)
        type_menu = menu.addMenu("切换折痕类型")
        type_actions = {
            CREASE_TYPE_PEAK: "峰折线",
            CREASE_TYPE_VALLEY: "谷折线",
            CREASE_TYPE_FLAT: "段折线",
            CREASE_TYPE_CUT: "剪裁线",
            CREASE_TYPE_BOUNDARY: "边界线",
            CREASE_TYPE_AUXILIARY: "辅助线",
        }
        for crease_type, label in type_actions.items():
            action = type_menu.addAction(label)
            action.setData(crease_type)
        width_action = menu.addAction("设置线宽…")
        delete_action = menu.addAction("删除折痕")
        chosen = menu.exec(global_pos)
        if chosen is None:
            return
        crease_id = int(crease.get("id", -1))
        if chosen is width_action:
            value, ok = QInputDialog.getDouble(self, "线宽", "请输入线宽:", float(crease.get("width", 1.6)), 0.5,
12.0, 1)
            if ok:
                self.update_crease(crease_id, {"width": value})
                self.crease_modified.emit(crease_id, {"width": value})
        elif chosen is delete_action:
            self.remove_crease(crease_id)
            self.crease_removed.emit(crease_id)
        else:
            crease_type = chosen.data()
            if crease_type:
                self.update_crease(crease_id, {"type": crease_type})
                self.crease_modified.emit(crease_id, {"type": crease_type})
                self.status_message.emit(f"折痕
    def export_snapshot(self) -> dict:
        return {
            "zoom": self._zoom,
            "offset": (self._offset.x(), self._offset.y()),
            "grid_size": self._grid_size,
            "show_grid": self._show_grid,
            "snap_enabled": self._snap_enabled,
            "show_vertices": self._show_vertices,
        }
    def apply_snapshot(self, snapshot: dict) -> None:
        if not snapshot:
            return
        self._zoom = float(snapshot.get("zoom", 1.0))
        offset = snapshot.get("offset", (0.0, 0.0))
        self._offset = QPointF(float(offset[0]), float(offset[1]))
        self._grid_size = float(snapshot.get("grid_size", 20.0))
        self._show_grid = bool(snapshot.get("show_grid", True))
        self._snap_enabled = bool(snapshot.get("snap_enabled", True))
        self._show_vertices = bool(snapshot.get("show_vertices", True))
        self.update()
    def crease_at_point(self, world_x: float, world_y: float) -> Optional[int]:
        return self._hit_crease(self._to_screen(QPointF(world_x, world_y)))
    def point_inside_region(self, world_x: float, world_y: float, polygon: list[tuple[float, float]]) -> bool:
        return point_in_polygon((world_x, world_y), polygon)
    def bounding_rect(self) -> QRectF:
        if not self._creases:
            return QRectF()
        xs: list[float] = []
        ys: list[float] = []
        for crease in self._creases:
            xs.extend([float(crease.get("x1", 0.0)), float(crease.get("x2", 0.0))])
            ys.extend([float(crease.get("y1", 0.0)), float(crease.get("y2", 0.0))])
        return QRectF(min(xs), min(ys), max(xs) - min(xs), max(ys) - min(ys))
    def fit_to_view(self) -> None:
        rect = self.bounding_rect()
        if rect.isEmpty():
            self.reset_view()
            return
        margin = 40.0
        scale_x = (self.width() - margin * 2) / max(rect.width(), 1e-6)
        scale_y = (self.height() - margin * 2) / max(rect.height(), 1e-6)
        self._zoom = max(MIN_ZOOM, min(MAX_ZOOM, min(scale_x, scale_y)))
        center = rect.center()
        self._offset = QPointF(
```

```
            self.width() / 2.0 - center.x() * self._zoom,
            self.height() / 2.0 - center.y() * self._zoom,
        )
        self.update()
# 文件: app/ui/crease_editor_panel.py
from PySide6.QtCore import Qt, Signal
from PySide6.QtGui import QColor
from PySide6.QtWidgets import (
    QWidget,
    QVBoxLayout,
    QHBoxLayout,
    QFormLayout,
    QGroupBox,
    QLabel,
    QLineEdit,
    QComboBox,
    QDoubleSpinBox,
    QSpinBox,
    QPushButton,
    QColorDialog,
    QListWidget,
    QListWidgetItem,
    QMessageBox,
    QCheckBox,
)
from app.common import logger
from app.core.settings import settings
from app.models.crease import CreaseType
CREASE_TYPE_LABELS = {
    CreaseType.MOUNTAIN: "峰折线",
    CreaseType.VALLEY: "谷折线",
    CreaseType.CUT: "剪裁线",
    CreaseType.BORDER: "边界线",
    CreaseType.AUXILIARY: "辅助参考线",
    CreaseType.FLAT: "段折线",
}
class CreaseEditorPanel(QWidget):
    crease_changed = Signal(dict)
    crease_deleted = Signal(str)
    crease_selected = Signal(str)
    def __init__(self, parent=None):
        super().__init__(parent)
        self._current_crease_id = None
        self._current_color = "
        self._build_ui()
        self._load_defaults()
    def _build_ui(self):
        root = QVBoxLayout(self)
        root.setContentsMargins(8, 8, 8, 8)
        root.setSpacing(8)
        self.crease_list = QListWidget()
        self.crease_list.setSelectionMode(QListWidget.SingleSelection)
        self.crease_list.currentItemChanged.connect(self._on_list_selection_changed)
        list_box = QGroupBox("折痕列表")
        list_layout = QVBoxLayout(list_box)
        list_layout.addWidget(self.crease_list)
        root.addWidget(list_box, 1)
        form_box = QGroupBox("折痕属性")
        form = QFormLayout(form_box)
        form.setLabelAlignment(Qt.AlignRight)
        self.id_label = QLabel("-")
        form.addRow("折痕编号", self.id_label)
        self.type_combo = QComboBox()
        for crease_type, label in CREASE_TYPE_LABELS.items():
            self.type_combo.addItem(label, crease_type)
        self.type_combo.currentIndexChanged.connect(self._on_type_changed)
        form.addRow("折痕类型", self.type_combo)
        self.layer_combo = QComboBox()
        self.layer_combo.currentIndexChanged.connect(self._on_layer_changed)
        form.addRow("所属图层", self.layer_combo)
        self.start_x_spin = self._make_coord_spin()
        self.start_y_spin = self._make_coord_spin()
        self.end_x_spin = self._make_coord_spin()
        self.end_y_spin = self._make_coord_spin()
        form.addRow("起点 X", self.start_x_spin)
        form.addRow("起点 Y", self.start_y_spin)
        form.addRow("终点 X", self.end_x_spin)
        form.addRow("终点 Y", self.end_y_spin)
        self.width_spin = QDoubleSpinBox()
        self.width_spin.setRange(0.1, 20.0)
        self.width_spin.setSingleStep(0.1)
        self.width_spin.setDecimals(2)
```

```
        self.width_spin.valueChanged.connect(self._on_width_changed)
        form.addRow("线宽 (px)", self.width_spin)
        self.color_button = QPushButton("选择颜色")
        self.color_button.clicked.connect(self._choose_color)
        self.color_preview = QLabel()
        self.color_preview.setFixedSize(48, 20)
        color_row = QHBoxLayout()
        color_row.addWidget(self.color_preview)
        color_row.addWidget(self.color_button)
        color_row.addStretch(1)
        color_wrap = QWidget()
        color_wrap.setLayout(color_row)
        form.addRow("线条颜色", color_wrap)
        self.dash_spin = QSpinBox()
        self.dash_spin.setRange(0, 20)
        self.dash_spin.setSuffix(" px")
        self.dash_spin.valueChanged.connect(self._on_dash_changed)
        form.addRow("虚线间隔", self.dash_spin)
        self.locked_check = QCheckBox("锁定该折痕")
        self.locked_check.stateChanged.connect(self._on_locked_changed)
        form.addRow("编辑状态", self.locked_check)
        self.note_edit = QLineEdit()
        self.note_edit.setPlaceholderText("折痕备注信息")
        self.note_edit.editingFinished.connect(self._on_note_changed)
        form.addRow("备注", self.note_edit)
        root.addWidget(form_box, 2)
        button_row = QHBoxLayout()
        self.apply_button = QPushButton("应用修改")
        self.apply_button.clicked.connect(self._apply_changes)
        self.reset_button = QPushButton("重置")
        self.reset_button.clicked.connect(self._reset_fields)
        self.delete_button = QPushButton("删除折痕")
        self.delete_button.clicked.connect(self._delete_current)
        button_row.addWidget(self.apply_button)
        button_row.addWidget(self.reset_button)
        button_row.addWidget(self.delete_button)
        root.addLayout(button_row)
        self._set_editing_enabled(False)
    def _make_coord_spin(self):
        spin = QDoubleSpinBox()
        spin.setRange(-100000.0, 100000.0)
        spin.setDecimals(3)
        spin.setSingleStep(1.0)
        return spin
    def _load_defaults(self):
        self.width_spin.setValue(float(settings.get("crease.default_width", 1.0)))
        self._current_color = settings.get("crease.default_color", "
        self._update_color_preview()
        self.dash_spin.setValue(int(settings.get("crease.default_dash", 0)))
    def _update_color_preview(self):
        self.color_preview.setStyleSheet(
            f"background-color: {self._current_color}; border: 1px solid
        )
    def _set_editing_enabled(self, enabled):
        for widget in (
            self.type_combo,
            self.layer_combo,
            self.start_x_spin,
            self.start_y_spin,
            self.end_x_spin,
            self.end_y_spin,
            self.width_spin,
            self.color_button,
            self.dash_spin,
            self.locked_check,
            self.note_edit,
            self.apply_button,
            self.reset_button,
            self.delete_button,
        ):
            widget.setEnabled(enabled)
    def set_layers(self, layers):
        self.layer_combo.blockSignals(True)
        self.layer_combo.clear()
        for layer in layers:
            self.layer_combo.addItem(layer.get("name", "未命名图层"), layer.get("id"))
        self.layer_combo.blockSignals(False)
    def load_creases(self, creases):
        self.crease_list.blockSignals(True)
        self.crease_list.clear()
        for crease in creases:
            item = QListWidgetItem(self._format_crease_label(crease))
```

```
            item.setData(Qt.UserRole, crease.get("id"))
            self.crease_list.addItem(item)
        self.crease_list.blockSignals(False)
        if self.crease_list.count() == 0:
            self._current_crease_id = None
            self._set_editing_enabled(False)
    def _format_crease_label(self, crease):
        type_label = CREASE_TYPE_LABELS.get(crease.get("crease_type"), "未知折痕")
        return f"{crease.get('id', '?')} | {type_label}"
    def load_crease(self, crease):
        if not crease:
            self._current_crease_id = None
            self._set_editing_enabled(False)
            return
        self._current_crease_id = crease.get("id")
        self.id_label.setText(str(self._current_crease_id))
        self._set_editing_enabled(True)
        index = self.type_combo.findData(crease.get("crease_type"))
        if index >= 0:
            self.type_combo.blockSignals(True)
            self.type_combo.setCurrentIndex(index)
            self.type_combo.blockSignals(False)
        layer_index = self.layer_combo.findData(crease.get("layer_id"))
        if layer_index >= 0:
            self.layer_combo.blockSignals(True)
            self.layer_combo.setCurrentIndex(layer_index)
            self.layer_combo.blockSignals(False)
        self.start_x_spin.setValue(float(crease.get("start_x", 0.0)))
        self.start_y_spin.setValue(float(crease.get("start_y", 0.0)))
        self.end_x_spin.setValue(float(crease.get("end_x", 0.0)))
        self.end_y_spin.setValue(float(crease.get("end_y", 0.0)))
        self.width_spin.setValue(float(crease.get("line_width", 1.0)))
        self._current_color = crease.get("color", "
        self._update_color_preview()
        self.dash_spin.setValue(int(crease.get("dash_gap", 0)))
        self.locked_check.setChecked(bool(crease.get("locked", False)))
        self.note_edit.setText(crease.get("note", ""))
    def _on_list_selection_changed(self, current, previous):
        if current is None:
            return
        crease_id = current.data(Qt.UserRole)
        self.crease_selected.emit(str(crease_id))
    def _on_type_changed(self, index):
        if self._current_crease_id is None:
            return
        crease_type = self.type_combo.itemData(index)
        self._emit_change({"crease_type": crease_type})
    def _on_layer_changed(self, index):
        if self._current_crease_id is None:
            return
        layer_id = self.layer_combo.itemData(index)
        self._emit_change({"layer_id": layer_id})
    def _on_width_changed(self, value):
        if self._current_crease_id is None:
            return
        self._emit_change({"line_width": float(value)})
    def _on_dash_changed(self, value):
        if self._current_crease_id is None:
            return
        self._emit_change({"dash_gap": int(value)})
    def _on_locked_changed(self, state):
        if self._current_crease_id is None:
            return
        self._emit_change({"locked": state == Qt.Checked.value})
    def _on_note_changed(self):
        if self._current_crease_id is None:
            return
        self._emit_change({"note": self.note_edit.text()})
    def _choose_color(self):
        color = QColorDialog.getColor(QColor(self._current_color), self, "选择折痕颜色")
        if not color.isValid():
            return
        self._current_color = color.name()
        self._update_color_preview()
        if self._current_crease_id is not None:
            self._emit_change({"color": self._current_color})
    def _collect_changes(self):
        return {
            "crease_type": self.type_combo.currentData(),
            "layer_id": self.layer_combo.currentData(),
            "start_x": self.start_x_spin.value(),
            "start_y": self.start_y_spin.value(),
            "end_x": self.end_x_spin.value(),
```

```
            "end_y": self.end_y_spin.value(),
            "line_width": self.width_spin.value(),
            "color": self._current_color,
            "dash_gap": self.dash_spin.value(),
            "locked": self.locked_check.isChecked(),
            "note": self.note_edit.text(),
        }
    def _emit_change(self, changes):
        payload = {"id": self._current_crease_id}
        payload.update(changes)
        self.crease_changed.emit(payload)
    def _apply_changes(self):
        if self._current_crease_id is None:
            QMessageBox.information(self, "提示", "请先选择一条折痕。")
            return
        try:
            self._emit_change(self._collect_changes())
            logger.info("折痕 %s 属性已更新", self._current_crease_id)
        except Exception as exc:
            logger.exception("应用折痕属性失败")
            QMessageBox.warning(self, "错误", f"应用折痕属性失败：{exc}")
    def _reset_fields(self):
        if self._current_crease_id is None:
            return
        self._load_defaults()
    def _delete_current(self):
        if self._current_crease_id is None:
            return
        answer = QMessageBox.question(
            self,
          "删除确认",
            f"确定删除折痕 {self._current_crease_id} 吗？",
            QMessageBox.Yes | QMessageBox.No,
        )
        if answer != QMessageBox.Yes:
            return
        crease_id = self._current_crease_id
        self.crease_deleted.emit(str(crease_id))
        logger.info("折痕 %s 已删除", crease_id)
        self._current_crease_id = None
        self._set_editing_enabled(False)
    def current_crease_id(self):
        return self._current_crease_id
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/ui/export_dialog.py
from PySide6.QtCore import Qt
from PySide6.QtWidgets import QDialog, QVBoxLayout, QHBoxLayout, QFormLayout, QGroupBox, QLabel, QLineEdit,
QComboBox, QCheckBox, QSpinBox, QDoubleSpinBox, QPushButton, QFileDialog, QDialogButtonBox, QMessageBox,
QTabWidget, QWidget, QTableWidget, QTableWidgetItem, QHeaderView, QAbstractItemView, QProgressBar
from app.common import logger
from app.config import config
from app.exceptions import ExportError
from app.services.export_service import ExportService
from app.services.project_service import ProjectService
class ExportDialog(QDialog):
    def __init__(self, project_id: int, parent=None):
        super().__init__(parent)
        self.project_id = project_id
        self.export_service = ExportService()
        self.project_service = ProjectService()
        self._export_result = None
        self.setWindowTitle("标准化导出 – 折痕图可视化编辑与标准化导出系统")
        self.setMinimumSize(680, 560)
        self._build_ui()
        self._load_project_info()
    def _build_ui(self):
        root = QVBoxLayout(self)
        root.setSpacing(10)
        info_box = QGroupBox("工程信息")
        info_layout = QFormLayout(info_box)
        self.project_name_label = QLabel("-")
        self.crease_count_label = QLabel("0")
        self.layer_count_label = QLabel("0")
        info_layout.addRow("工程名称：", self.project_name_label)
        info_layout.addRow("折痕数量：", self.crease_count_label)
        info_layout.addRow("图层数量：", self.layer_count_label)
        root.addWidget(info_box)
        self.tab_widget = QTabWidget()
        self.tab_widget.addTab(self._build_format_tab(), "导出格式")
        self.tab_widget.addTab(self._build_options_tab(), "导出选项")
```

```
        self.tab_widget.addTab(self._build_preview_tab(), "内容预览")
        root.addWidget(self.tab_widget, 1)
        self.progress_bar = QProgressBar()
        self.progress_bar.setRange(0, 100)
        self.progress_bar.setValue(0)
        self.progress_bar.setVisible(False)
        root.addWidget(self.progress_bar)
        button_box = QDialogButtonBox()
        self.export_button = button_box.addButton("开始导出", QDialogButtonBox.AcceptRole)
        self.close_button = button_box.addButton("关闭", QDialogButtonBox.RejectRole)
        self.export_button.clicked.connect(self._on_export)
        self.close_button.clicked.connect(self.reject)
        root.addWidget(button_box)
    def _build_format_tab(self) -> QWidget:
        page = QWidget()
        layout = QVBoxLayout(page)
        form_box = QGroupBox("输出格式")
        form_layout = QFormLayout(form_box)
        self.format_combo = QComboBox()
        self.format_combo.addItem("SVG 矢量图 (.svg)", "svg")
        self.format_combo.addItem("DXF 工程图 (.dxf)", "dxf")
        self.format_combo.addItem("FOLD 折痕数据 (.fold)", "fold")
        self.format_combo.currentIndexChanged.connect(self._on_format_changed)
        form_layout.addRow("格式", self.format_combo)
        path_row = QHBoxLayout()
        self.path_edit = QLineEdit()
        self.path_edit.setPlaceholderText("请选择导出文件保存路径")
        browse_btn = QPushButton("浏览...")
        browse_btn.clicked.connect(self._on_browse_path)
        path_row.addWidget(self.path_edit, 1)
        path_row.addWidget(browse_btn)
        form_layout.addRow("保存路径", path_row)
        self.unit_combo = QComboBox()
        self.unit_combo.addItem("毫米 (mm)", "mm")
        self.unit_combo.addItem("厘米 (cm)", "cm")
        self.unit_combo.addItem("英寸 (in)", "in")
        form_layout.addRow("单位", self.unit_combo)
        self.scale_spin = QDoubleSpinBox()
        self.scale_spin.setRange(0.01, 1000.0)
        self.scale_spin.setValue(1.0)
        self.scale_spin.setSingleStep(0.1)
        self.scale_spin.setDecimals(3)
        form_layout.addRow("缩放比例", self.scale_spin)
        layout.addWidget(form_box)
        desc_box = QGroupBox("格式说明")
        desc_layout = QVBoxLayout(desc_box)
        self.format_desc_label = QLabel()
        self.format_desc_label.setWordWrap(True)
        desc_layout.addWidget(self.format_desc_label)
        layout.addWidget(desc_box)
        layout.addStretch(1)
        self._on_format_changed(0)
        return page
    def _build_options_tab(self) -> QWidget:
        page = QWidget()
        layout = QVBoxLayout(page)
        crease_box = QGroupBox("折痕类型过滤")
        crease_layout = QVBoxLayout(crease_box)
        self.include_mountain = QCheckBox("包含峰折线 (Mountain)")
        self.include_mountain.setChecked(True)
        self.include_valley = QCheckBox("包含谷折线 (Valley)")
        self.include_valley.setChecked(True)
        self.include_flat = QCheckBox("包含段折线 (Flat)")
        self.include_flat.setChecked(True)
        self.include_cut = QCheckBox("包含剪裁线 (Cut)")
        self.include_cut.setChecked(True)
        self.include_boundary = QCheckBox("包含边界线 (Boundary)")
        self.include_boundary.setChecked(True)
        self.include_auxiliary = QCheckBox("包含辅助参考线 (Auxiliary)")
        self.include_auxiliary.setChecked(False)
        for cb in (self.include_mountain, self.include_valley, self.include_flat,
                   self.include_cut, self.include_boundary, self.include_auxiliary):
            crease_layout.addWidget(cb)
        layout.addWidget(crease_box)
        style_box = QGroupBox("样式选项")
        style_layout = QFormLayout(style_box)
        self.line_width_spin = QDoubleSpinBox()
        self.line_width_spin.setRange(0.1, 20.0)
        self.line_width_spin.setValue(1.0)
        self.line_width_spin.setSingleStep(0.1)
        style_layout.addRow("线宽", self.line_width_spin)
        self.vertex_check = QCheckBox("导出顶点标记")
```

```
        self.vertex_check.setChecked(True)
        style_layout.addRow("", self.vertex_check)
        self.metadata_check = QCheckBox("包含元数据信息")
        self.metadata_check.setChecked(True)
        style_layout.addRow("", self.metadata_check)
        self.background_check = QCheckBox("包含白色背景")
        self.background_check.setChecked(False)
        style_layout.addRow("", self.background_check)
        layout.addWidget(style_box)
        precision_box = QGroupBox("数值精度")
        precision_layout = QFormLayout(precision_box)
        self.precision_spin = QSpinBox()
        self.precision_spin.setRange(0, 8)
        self.precision_spin.setValue(4)
        precision_layout.addRow("小数位数", self.precision_spin)
        layout.addWidget(precision_box)
        layout.addStretch(1)
        return page
    def _build_preview_tab(self) -> QWidget:
        page = QWidget()
        layout = QVBoxLayout(page)
        self.preview_table = QTableWidget(0, 4)
        self.preview_table.setHorizontalHeaderLabels(["折痕类型", "起点", "终点", "图层"])
        self.preview_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.preview_table.setEditTriggers(QAbstractItemView.NoEditTriggers)
        self.preview_table.setSelectionBehavior(QAbstractItemView.SelectRows)
        layout.addWidget(self.preview_table, 1)
        refresh_btn = QPushButton("刷新预览")
        refresh_btn.clicked.connect(self._refresh_preview)
        layout.addWidget(refresh_btn, 0, Qt.AlignRight)
        return page
    def _load_project_info(self):
        try:
            project = self.project_service.get_project(self.project_id)
            if project is None:
                raise ExportError("工程不存在，无法导出")
            self.project_name_label.setText(project.name)
            creases = self.export_service.list_creases(self.project_id)
            layers = self.export_service.list_layers(self.project_id)
            self.crease_count_label.setText(str(len(creases)))
            self.layer_count_label.setText(str(len(layers)))
            self._refresh_preview()
        except ExportError as exc:
            logger.error("加载工程信息失败: %s", exc)
            QMessageBox.warning(self, "提示", str(exc))
        except Exception as exc:
            logger.exception("加载工程信息异常")
            QMessageBox.critical(self, "错误", f"加载工程信息失败：{exc}")
    def _refresh_preview(self):
        try:
            creases = self.export_service.list_creases(self.project_id)
        except Exception as exc:
            logger.exception("刷新预览失败")
            QMessageBox.warning(self, "提示", f"刷新预览失败：{exc}")
            return
        self.preview_table.setRowCount(0)
        for crease in creases:
            row = self.preview_table.rowCount()
            self.preview_table.insertRow(row)
            self.preview_table.setItem(row, 0, QTableWidgetItem(str(crease.crease_type)))
            self.preview_table.setItem(row, 1, QTableWidgetItem(f"({crease.x1:.2f}, {crease.y1:.2f})"))
            self.preview_table.setItem(row, 2, QTableWidgetItem(f"({crease.x2:.2f}, {crease.y2:.2f})"))
            self.preview_table.setItem(row, 3, QTableWidgetItem(str(crease.layer_id)))
    def _on_format_changed(self, index: int):
        fmt = self.format_combo.itemData(index)
        descriptions = {
            "svg": "SVG 为通用矢量图形格式，适合在浏览器与矢量软件中查看，保留折痕颜色与线型。",
            "dxf": "DXF 为 CAD 工程交换格式，适合对接数控加工与结构仿真流程。",
            "fold": "FOLD 为折纸数据标准格式，包含顶点、边与面片拓扑信息，可被主流折纸软件读取。",
        }
        self.format_desc_label.setText(descriptions.get(fmt, ""))
        if not self.path_edit.text().strip():
            self.path_edit.setText(self._default_path(fmt))
    def _default_path(self, fmt: str) -> str:
        base = config.get("export.default_dir", "")
        name = self.project_name_label.text() if self.project_name_label.text() != "-" else "crease_pattern"
        return f"{base}/{name}.{fmt}" if base else f"{name}.{fmt}"
    def _on_browse_path(self):
        fmt = self.format_combo.currentData()
        filters = {
            "svg": "SVG 文件 (*.svg)",
            "dxf": "DXF 文件 (*.dxf)",
```

```
            "fold": "FOLD 文件 (*.fold)",
        }
        path, _ = QFileDialog.getSaveFileName(
            self, "选择导出路径", self.path_edit.text(), filters.get(fmt, "所有文件 (*.*)")
        )
        if path:
            self.path_edit.setText(path)
    def _collect_options(self) -> dict:
        return {
            "format": self.format_combo.currentData(),
            "path": self.path_edit.text().strip(),
            "unit": self.unit_combo.currentData(),
            "scale": self.scale_spin.value(),
            "line_width": self.line_width_spin.value(),
            "precision": self.precision_spin.value(),
            "include_vertices": self.vertex_check.isChecked(),
            "include_metadata": self.metadata_check.isChecked(),
            "include_background": self.background_check.isChecked(),
            "crease_types": self._selected_crease_types(),
        }
    def _selected_crease_types(self) -> list:
        mapping = [
            (self.include_mountain, "mountain"),
            (self.include_valley, "valley"),
            (self.include_flat, "flat"),
            (self.include_cut, "cut"),
            (self.include_boundary, "boundary"),
            (self.include_auxiliary, "auxiliary"),
        ]
        return [name for checkbox, name in mapping if checkbox.isChecked()]
    def _validate_options(self, options: dict) -> bool:
        if not options["path"]:
            QMessageBox.warning(self, "提示", "请先选择导出文件保存路径")
            return False
        if not options["crease_types"]:
            QMessageBox.warning(self, "提示", "至少需要选择一种折痕类型")
            return False
        return True
    def _on_export(self):
        options = self._collect_options()
        if not self._validate_options(options):
            return
        self.progress_bar.setVisible(True)
        self.progress_bar.setValue(10)
        self.export_button.setEnabled(False)
        try:
            self.progress_bar.setValue(40)
            result = self.export_service.export_project(self.project_id, options)
            self.progress_bar.setValue(90)
            self._export_result = result
            self.progress_bar.setValue(100)
            QMessageBox.information(
                self, "导出完成",
                f"已成功导出 {result.get('count', 0)} 条折痕到\n{options['path']}"
            )
            logger.info("工程 %s 导出成功: %s", self.project_id, options["path"])
            self.accept()
        except ExportError as exc:
            logger.error("导出失败: %s", exc)
            QMessageBox.warning(self, "导出失败", str(exc))
        except Exception as exc:
            logger.exception("导出过程异常")
            QMessageBox.critical(self, "错误", f"导出过程发生异常{exc}")
        finally:
            self.export_button.setEnabled(True)
            self.progress_bar.setVisible(False)
    def export_result(self) -> dict:
        return self._export_result or {}
</DSML parameter>
</DSML invoke>
</DSML calls>
# 文件: app/ui/layer_panel.py
from typing import Optional, List
from PySide6.QtCore import Qt, Signal
from PySide6.QtGui import QColor, QPixmap
from PySide6.QtWidgets import (
    QWidget,
    QVBoxLayout,
    QHBoxLayout,
    QListWidget,
    QListWidgetItem,
    QPushButton,
```

```
    QInputDialog,
    QMessageBox,
    QColorDialog,
    QDoubleSpinBox,
    QLabel,
    QFormLayout,
    QGroupBox,
    QCheckBox,
    QAbstractItemView,
)
from app.models.layer import Layer
from app.utils.color_utils import normalize_hex_color
class LayerPanel(QWidget):
    layer_selected = Signal(int)
    layer_visibility_changed = Signal(int, bool)
    layer_locked_changed = Signal(int, bool)
    layer_created = Signal(str, str)
    layer_removed = Signal(int)
    layer_renamed = Signal(int, str)
    layer_style_changed = Signal(int, dict)
    def __init__(self, parent: Optional[QWidget] = None) -> None:
        super().__init__(parent)
        self._layers: List[Layer] = []
        self._current_layer_id: Optional[int] = None
        self._build_ui()
    def _build_ui(self) -> None:
        root = QVBoxLayout(self)
        root.setContentsMargins(6, 6, 6, 6)
        root.setSpacing(6)
        self.layer_list = QListWidget(self)
        self.layer_list.setSelectionMode(QAbstractItemView.SelectionMode.SingleSelection)
        self.layer_list.currentItemChanged.connect(self._on_current_item_changed)
        self.layer_list.itemChanged.connect(self._on_item_changed)
        root.addWidget(self.layer_list, 1)
        button_row = QHBoxLayout()
        self.btn_add = QPushButton("新建图层", self)
        self.btn_remove = QPushButton("删除图层", self)
        self.btn_rename = QPushButton("重命名", self)
        self.btn_up = QPushButton("上移", self)
        self.btn_down = QPushButton("下移", self)
        for btn in (self.btn_add, self.btn_remove, self.btn_rename, self.btn_up, self.btn_down):
            button_row.addWidget(btn)
        root.addLayout(button_row)
        self.btn_add.clicked.connect(self._on_add_layer)
        self.btn_remove.clicked.connect(self._on_remove_layer)
        self.btn_rename.clicked.connect(self._on_rename_layer)
        self.btn_up.clicked.connect(lambda: self._move_layer(-1))
        self.btn_down.clicked.connect(lambda: self._move_layer(1))
        style_box = QGroupBox("图层样式", self)
        form = QFormLayout(style_box)
        self.color_label = QLabel("
        self.color_label.setAutoFillBackground(True)
        self._apply_color_preview("
        self.btn_color = QPushButton("选择颜色", style_box)
        self.btn_color.clicked.connect(self._on_pick_color)
        self.width_spin = QDoubleSpinBox(style_box)
        self.width_spin.setRange(0.1, 20.0)
        self.width_spin.setSingleStep(0.1)
        self.width_spin.setValue(1.0)
        self.width_spin.valueChanged.connect(self._on_style_changed)
        self.chk_visible = QCheckBox("可见", style_box)
        self.chk_visible.setChecked(True)
        self.chk_visible.toggled.connect(self._on_visible_toggled)
        self.chk_locked = QCheckBox("锁定", style_box)
        self.chk_locked.toggled.connect(self._on_locked_toggled)
        form.addRow("颜色", self.color_label)
        form.addRow("", self.btn_color)
        form.addRow("线宽", self.width_spin)
        form.addRow("", self.chk_visible)
        form.addRow("", self.chk_locked)
        root.addWidget(style_box)
    def set_layers(self, layers: List[Layer]) -> None:
        self._layers = list(layers)
        self._refresh_list()
    def current_layer_id(self) -> Optional[int]:
        return self._current_layer_id
    def _refresh_list(self) -> None:
        self.layer_list.blockSignals(True)
        self.layer_list.clear()
        for layer in self._layers:
            item = QListWidgetItem(layer.name)
            item.setData(Qt.ItemDataRole.UserRole, layer.id)
```

```
            item.setFlags(
                item.flags()
                | Qt.ItemFlag.ItemIsUserCheckable
                | Qt.ItemFlag.ItemIsEditable
            )
            item.setCheckState(
                Qt.CheckState.Checked if layer.visible else Qt.CheckState.Unchecked
            )
            item.setForeground(QColor(normalize_hex_color(layer.color)))
            self.layer_list.addItem(item)
        self.layer_list.blockSignals(False)
        self._sync_selection()
    def _sync_selection(self) -> None:
        if not self._layers:
            self._current_layer_id = None
            self._update_style_controls(None)
            return
        target_row = 0
        if self._current_layer_id is not None:
            for idx, layer in enumerate(self._layers):
                if layer.id == self._current_layer_id:
                    target_row = idx
                    break
        self.layer_list.setCurrentRow(target_row)
        self._current_layer_id = self._layers[target_row].id
        self._update_style_controls(self._layers[target_row])
    def _update_style_controls(self, layer: Optional[Layer]) -> None:
        enabled = layer is not None
        self.btn_color.setEnabled(enabled)
        self.width_spin.setEnabled(enabled)
        self.chk_visible.setEnabled(enabled)
        self.chk_locked.setEnabled(enabled)
        self.btn_remove.setEnabled(enabled)
        self.btn_rename.setEnabled(enabled)
        if layer is None:
            return
        self.chk_visible.blockSignals(True)
        self.chk_locked.blockSignals(True)
        self.width_spin.blockSignals(True)
        self.chk_visible.setChecked(layer.visible)
        self.chk_locked.setChecked(layer.locked)
        self.width_spin.setValue(layer.line_width)
        self.chk_visible.blockSignals(False)
        self.chk_locked.blockSignals(False)
        self.width_spin.blockSignals(False)
        self._apply_color_preview(layer.color)
    def _apply_color_preview(self, color: str) -> None:
        hex_color = normalize_hex_color(color)
        self.color_label.setText(hex_color)
        pix = QPixmap(16, 16)
        pix.fill(QColor(hex_color))
        self.color_label.setPixmap(pix)
    def _find_layer(self, layer_id: Optional[int]) -> Optional[Layer]:
        if layer_id is None:
            return None
        for layer in self._layers:
            if layer.id == layer_id:
                return layer
        return None
    def _on_current_item_changed(
        self, current: Optional[QListWidgetItem], _previous: Optional[QListWidgetItem]
    ) -> None:
        if current is None:
            return
        layer_id = current.data(Qt.ItemDataRole.UserRole)
        self._current_layer_id = layer_id
        self._update_style_controls(self._find_layer(layer_id))
        if layer_id is not None:
            self.layer_selected.emit(int(layer_id))
    def _on_item_changed(self, item: QListWidgetItem) -> None:
        layer_id = item.data(Qt.ItemDataRole.UserRole)
        layer = self._find_layer(layer_id)
        if layer is None:
            return
        checked = item.checkState() == Qt.CheckState.Checked
        if checked != layer.visible:
            layer.visible = checked
            if layer.id == self._current_layer_id:
                self.chk_visible.blockSignals(True)
                self.chk_visible.setChecked(checked)
                self.chk_visible.blockSignals(False)
            self.layer_visibility_changed.emit(int(layer.id), checked)
        new_name = item.text().strip()
```

```
        if new_name and new_name != layer.name:
            layer.name = new_name
            self.layer_renamed.emit(int(layer.id), new_name)
    def _on_add_layer(self) -> None:
        name, ok = QInputDialog.getText(self, "新建图层", "图层名称：")
        if not ok:
            return
        name = name.strip() or f"图层 {len(self._layers) + 1}"
        color = "
        self.layer_created.emit(name, color)
    def _on_remove_layer(self) -> None:
        layer = self._find_layer(self._current_layer_id)
        if layer is None:
            return
        if len(self._layers) <= 1:
            QMessageBox.warning(self, "删除图层", "至少需要保留一个图层。")
            return
        reply = QMessageBox.question(
            self,
          "删除图层",
            f"确定删除图层“{layer.name}”及其包含的折痕吗？",
            QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No,
        )
        if reply == QMessageBox.StandardButton.Yes:
            self.layer_removed.emit(int(layer.id))
    def _on_rename_layer(self) -> None:
        layer = self._find_layer(self._current_layer_id)
        if layer is None:
            return
        name, ok = QInputDialog.getText(self, "重命名图层", "图层名称", text=layer.name)
        if not ok:
            return
        name = name.strip()
        if not name or name == layer.name:
            return
        layer.name = name
        self._refresh_list()
        self.layer_renamed.emit(int(layer.id), name)
    def _move_layer(self, offset: int) -> None:
        layer = self._find_layer(self._current_layer_id)
        if layer is None:
            return
        index = self._layers.index(layer)
        new_index = index + offset
        if new_index < 0 or new_index >= len(self._layers):
            return
        self._layers[index], self._layers[new_index] = (
            self._layers[new_index],
            self._layers[index],
        )
        self._refresh_list()
    def _on_pick_color(self) -> None:
        layer = self._find_layer(self._current_layer_id)
        if layer is None:
            return
        color = QColorDialog.getColor(QColor(normalize_hex_color(layer.color)), self, "选择图层颜色")
        if not color.isValid():
            return
        layer.color = color.name()
        self._apply_color_preview(layer.color)
        self._refresh_list()
        self.layer_style_changed.emit(int(layer.id), {"color": layer.color})
    def _on_style_changed(self, value: float) -> None:
        layer = self._find_layer(self._current_layer_id)
        if layer is None:
            return
        layer.line_width = float(value)
        self.layer_style_changed.emit(int(layer.id), {"line_width": layer.line_width})
    def _on_visible_toggled(self, checked: bool) -> None:
        layer = self._find_layer(self._current_layer_id)
        if layer is None:
            return
        layer.visible = checked
        self._refresh_list()
        self.layer_visibility_changed.emit(int(layer.id), checked)
    def _on_locked_toggled(self, checked: bool) -> None:
        layer = self._find_layer(self._current_layer_id)
        if layer is None:
            return
        layer.locked = checked
        self.layer_locked_changed.emit(int(layer.id), checked)
    def select_layer(self, layer_id: int) -> None:
        self._current_layer_id = layer_id
```

```
        self._sync_selection()
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/ui/main_window.py
from PySide6.QtCore import Qt, QSize
from PySide6.QtGui import QAction, QKeySequence
from PySide6.QtWidgets import QMainWindow, QWidget, QVBoxLayout, QSplitter, QTabWidget, QToolBar, QStatusBar,
QMessageBox, QFileDialog, QLabel, QDockWidget, QSizePolicy
from app.common import logger
from app.ui.canvas_widget import CanvasWidget
from app.ui.crease_editor_panel import CreaseEditorPanel
from app.ui.layer_panel import LayerPanel
from app.ui.validation_panel import ValidationPanel
from app.ui.preview_widget import PreviewWidget
from app.ui.export_dialog import ExportDialog
from app.ui.settings_dialog import SettingsDialog
from app.ui.about_dialog import AboutDialog
from app.services.project_service import ProjectService
from app.services.crease_service import CreaseService
from app.services.validation_service import ValidationService
from app.services.export_service import ExportService
from app.services.preview_service import PreviewService
class MainWindow(QMainWindow):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle("折痕图可视化编辑与标准化导出系统")
        self.resize(1360, 860)
        self.setMinimumSize(QSize(1024, 680))
        self.project_service = ProjectService()
        self.crease_service = CreaseService()
        self.validation_service = ValidationService()
        self.export_service = ExportService()
        self.preview_service = PreviewService()
        self.current_project_id = None
        self._build_actions()
        self._build_menu_bar()
        self._build_tool_bar()
        self._build_central_area()
        self._build_dock_panels()
        self._build_status_bar()
        self._connect_signals()
        self._update_actions_state()
    def _build_actions(self):
        self.action_new_project = QAction("新建工程", self)
        self.action_new_project.setShortcut(QKeySequence.New)
        self.action_new_project.setStatusTip("创建一个新的折痕图工程")
        self.action_open_project = QAction("打开工程", self)
        self.action_open_project.setShortcut(QKeySequence.Open)
        self.action_open_project.setStatusTip("从本地加载已有折痕图工程")
        self.action_save_project = QAction("保存工程", self)
        self.action_save_project.setShortcut(QKeySequence.Save)
        self.action_save_project.setStatusTip("保存当前折痕图工程")
        self.action_save_as = QAction("另存为", self)
        self.action_save_as.setShortcut(QKeySequence.SaveAs)
        self.action_save_as.setStatusTip("将当前工程另存为新文件")
        self.action_export = QAction("标准化导出", self)
        self.action_export.setShortcut("Ctrl+E")
        self.action_export.setStatusTip("导出为 SVG / DXF / FOLD 等标准格式")
        self.action_undo = QAction("撤销", self)
        self.action_undo.setShortcut(QKeySequence.Undo)
        self.action_redo = QAction("重做", self)
        self.action_redo.setShortcut(QKeySequence.Redo)
        self.action_delete = QAction("删除选中", self)
        self.action_delete.setShortcut(QKeySequence.Delete)
        self.action_validate = QAction("几何校验", self)
        self.action_validate.setShortcut("F5")
        self.action_validate.setStatusTip("检测顶点角度、折痕交叉与面片重叠等结构问题")
        self.action_preview = QAction("三维预览", self)
        self.action_preview.setShortcut("F6")
        self.action_preview.setStatusTip("实时呈现折叠后的三维形态")
        self.action_zoom_in = QAction("放大", self)
        self.action_zoom_in.setShortcut(QKeySequence.ZoomIn)
        self.action_zoom_out = QAction("缩小", self)
        self.action_zoom_out.setShortcut(QKeySequence.ZoomOut)
        self.action_fit_view = QAction("适应窗口", self)
        self.action_fit_view.setShortcut("Ctrl+0")
        self.action_settings = QAction("系统设置", self)
        self.action_settings.setStatusTip("配置网格吸附、单位与渲染参数")
        self.action_about = QAction("关于", self)
        self.action_about.setStatusTip("查看软件版本与版权信息")
        self.action_exit = QAction("退出", self)
```

```
        self.action_exit.setShortcut(QKeySequence.Quit)
    def _build_menu_bar(self):
        menu_bar = self.menuBar()
        file_menu = menu_bar.addMenu("文件")
        file_menu.addAction(self.action_new_project)
        file_menu.addAction(self.action_open_project)
        file_menu.addSeparator()
        file_menu.addAction(self.action_save_project)
        file_menu.addAction(self.action_save_as)
        file_menu.addSeparator()
        file_menu.addAction(self.action_export)
        file_menu.addSeparator()
        file_menu.addAction(self.action_exit)
        edit_menu = menu_bar.addMenu("编辑")
        edit_menu.addAction(self.action_undo)
        edit_menu.addAction(self.action_redo)
        edit_menu.addSeparator()
        edit_menu.addAction(self.action_delete)
        view_menu = menu_bar.addMenu("视图")
        view_menu.addAction(self.action_zoom_in)
        view_menu.addAction(self.action_zoom_out)
        view_menu.addAction(self.action_fit_view)
        view_menu.addSeparator()
        view_menu.addAction(self.action_preview)
        tools_menu = menu_bar.addMenu("工具")
        tools_menu.addAction(self.action_validate)
        tools_menu.addSeparator()
        tools_menu.addAction(self.action_settings)
        help_menu = menu_bar.addMenu("帮助")
        help_menu.addAction(self.action_about)
    def _build_tool_bar(self):
        tool_bar = QToolBar("主工具栏", self)
        tool_bar.setMovable(False)
        tool_bar.setIconSize(QSize(20, 20))
        tool_bar.addAction(self.action_new_project)
        tool_bar.addAction(self.action_open_project)
        tool_bar.addAction(self.action_save_project)
        tool_bar.addSeparator()
        tool_bar.addAction(self.action_undo)
        tool_bar.addAction(self.action_redo)
        tool_bar.addAction(self.action_delete)
        tool_bar.addSeparator()
        tool_bar.addAction(self.action_validate)
        tool_bar.addAction(self.action_preview)
        tool_bar.addSeparator()
        tool_bar.addAction(self.action_export)
        self.addToolBar(tool_bar)
    def _build_central_area(self):
        self.canvas_widget = CanvasWidget(self)
        self.preview_widget = PreviewWidget(self)
        self.center_tabs = QTabWidget(self)
        self.center_tabs.setDocumentMode(True)
        self.center_tabs.addTab(self.canvas_widget, "二维折痕编辑")
        self.center_tabs.addTab(self.preview_widget, "三维折叠预览")
        splitter = QSplitter(Qt.Horizontal, self)
        splitter.addWidget(self.center_tabs)
        splitter.setStretchFactor(0, 1)
        splitter.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)
        container = QWidget(self)
        layout = QVBoxLayout(container)
        layout.setContentsMargins(4, 4, 4, 4)
        layout.addWidget(splitter)
        self.setCentralWidget(container)
    def _build_dock_panels(self):
        self.crease_editor_panel = CreaseEditorPanel(self)
        self.layer_panel = LayerPanel(self)
        self.validation_panel = ValidationPanel(self)
        dock_editor = QDockWidget("折痕编辑", self)
        dock_editor.setObjectName("dock_editor")
        dock_editor.setWidget(self.crease_editor_panel)
        dock_editor.setAllowedAreas(Qt.LeftDockWidgetArea | Qt.RightDockWidgetArea)
        self.addDockWidget(Qt.LeftDockWidgetArea, dock_editor)
        dock_layer = QDockWidget("图层管理", self)
        dock_layer.setObjectName("dock_layer")
        dock_layer.setWidget(self.layer_panel)
        dock_layer.setAllowedAreas(Qt.LeftDockWidgetArea | Qt.RightDockWidgetArea)
        self.addDockWidget(Qt.LeftDockWidgetArea, dock_layer)
        dock_validation = QDockWidget("几何校验", self)
        dock_validation.setObjectName("dock_validation")
        dock_validation.setWidget(self.validation_panel)
        dock_validation.setAllowedAreas(Qt.RightDockWidgetArea | Qt.BottomDockWidgetArea)
        self.addDockWidget(Qt.RightDockWidgetArea, dock_validation)
```

```
    def _build_status_bar(self):
        self.status_bar = QStatusBar(self)
        self.setStatusBar(self.status_bar)
        self.label_project = QLabel("未打开工程", self)
        self.label_crease_count = QLabel("折痕数：0", self)
        self.label_validation = QLabel("校验：未执行", self)
        self.status_bar.addWidget(self.label_project)
        self.status_bar.addPermanentWidget(self.label_crease_count)
        self.status_bar.addPermanentWidget(self.label_validation)
    def _connect_signals(self):
        self.action_new_project.triggered.connect(self.on_new_project)
        self.action_open_project.triggered.connect(self.on_open_project)
        self.action_save_project.triggered.connect(self.on_save_project)
        self.action_save_as.triggered.connect(self.on_save_as)
        self.action_export.triggered.connect(self.on_export)
        self.action_undo.triggered.connect(self.on_undo)
        self.action_redo.triggered.connect(self.on_redo)
        self.action_delete.triggered.connect(self.on_delete_selected)
        self.action_validate.triggered.connect(self.on_validate)
        self.action_preview.triggered.connect(self.on_preview)
        self.action_zoom_in.triggered.connect(self.on_zoom_in)
        self.action_zoom_out.triggered.connect(self.on_zoom_out)
        self.action_fit_view.triggered.connect(self.on_fit_view)
        self.action_settings.triggered.connect(self.on_settings)
        self.action_about.triggered.connect(self.on_about)
        self.action_exit.triggered.connect(self.close)
        self.canvas_widget.crease_count_changed.connect(self.on_crease_count_changed)
        self.crease_editor_panel.crease_updated.connect(self.on_crease_updated)
        self.layer_panel.layer_changed.connect(self.on_layer_changed)
        self.validation_panel.issue_selected.connect(self.on_issue_selected)
    def _update_actions_state(self):
        has_project = self.current_project_id is not None
        self.action_save_project.setEnabled(has_project)
        self.action_save_as.setEnabled(has_project)
        self.action_export.setEnabled(has_project)
        self.action_validate.setEnabled(has_project)
        self.action_preview.setEnabled(has_project)
    def on_new_project(self):
        try:
            name, ok = self._ask_project_name()
            if not ok or not name:
                return
            project = self.project_service.create_project(name)
            self.current_project_id = project.id
            self.canvas_widget.load_project(project.id)
            self.layer_panel.load_project(project.id)
            self.validation_panel.clear()
            self.label_project.setText(f"工程：{project.name}")
            self.label_validation.setText("校验：未执行")
            self._update_actions_state()
            logger.info("新建工程成功: %s", project.name)
        except Exception as exc:
            logger.exception("新建工程失败")
            QMessageBox.critical(self, "错误", f"新建工程失败：{exc}")
    def _ask_project_name(self):
        from PySide6.QtWidgets import QInputDialog
        return QInputDialog.getText(self, "新建工程", "请输入工程名称：")
    def on_open_project(self):
        file_path, _ = QFileDialog.getOpenFileName(
            self, "打开工程", "", "折痕图工程 (*.foldproj *.json);;所有文件 (*.*)"
        )
        if not file_path:
            return
        try:
            project = self.project_service.open_project(file_path)
            self.current_project_id = project.id
            self.canvas_widget.load_project(project.id)
            self.layer_panel.load_project(project.id)
            self.validation_panel.clear()
            self.label_project.setText(f"工程：{project.name}")
            self.label_validation.setText("校验：未执行")
            self._update_actions_state()
            logger.info("打开工程成功: %s", file_path)
        except Exception as exc:
            logger.exception("打开工程失败")
            QMessageBox.critical(self, "错误", f"打开工程失败：{exc}")
    def on_save_project(self):
        if self.current_project_id is None:
            return
        try:
            self.project_service.save_project(self.current_project_id)
            self.status_bar.showMessage("工程已保存", 3000)
```

```
        except Exception as exc:
            logger.exception("保存工程失败")
            QMessageBox.critical(self, "错误", f"保存工程失败：{exc}")
    def on_save_as(self):
        if self.current_project_id is None:
            return
        file_path, _ = QFileDialog.getSaveFileName(
            self, "另存为", "", "折痕图工程 (*.foldproj);;JSON 文件 (*.json)"
        )
        if not file_path:
            return
        try:
            self.project_service.save_project_as(self.current_project_id, file_path)
            self.status_bar.showMessage("工程已另存", 3000)
        except Exception as exc:
            logger.exception("另存工程失败")
            QMessageBox.critical(self, "错误", f"另存工程失败：{exc}")
    def on_export(self):
        if self.current_project_id is None:
            return
        dialog = ExportDialog(self.current_project_id, self.export_service, self)
        if dialog.exec():
            self.status_bar.showMessage("导出完成", 3000)
    def on_undo(self):
        if self.canvas_widget.undo():
            self.status_bar.showMessage("已撤销", 2000)
    def on_redo(self):
        if self.canvas_widget.redo():
            self.status_bar.showMessage("已重做", 2000)
    def on_delete_selected(self):
        removed = self.canvas_widget.delete_selected()
        if removed:
            self.status_bar.showMessage(f"已删除 {removed} 个元素", 2000)
    def on_validate(self):
        if self.current_project_id is None:
            return
        try:
            issues = self.validation_service.validate_project(self.current_project_id)
            self.validation_panel.show_issues(issues)
            self.label_validation.setText(f"校验：{len(issues)} 个问题")
            self.status_bar.showMessage(f"校验完成，共发现 {len(issues)} 个问题", 4000)
        except Exception as exc:
            logger.exception("几何校验失败")
            QMessageBox.critical(self, "错误", f"几何校验失败：{exc}")
    def on_preview(self):
        if self.current_project_id is None:
            return
        try:
            self.preview_widget.refresh(self.current_project_id)
            self.center_tabs.setCurrentWidget(self.preview_widget)
        except Exception as exc:
            logger.exception("三维预览失败")
            QMessageBox.critical(self, "错误", f"三维预览失败：{exc}")
    def on_zoom_in(self):
        self.canvas_widget.zoom_in()
    def on_zoom_out(self):
        self.canvas_widget.zoom_out()
    def on_fit_view(self):
        self.canvas_widget.fit_view()
    def on_settings(self):
        dialog = SettingsDialog(self)
        if dialog.exec():
            self.canvas_widget.apply_settings()
            self.status_bar.showMessage("设置已更新", 3000)
    def on_about(self):
        dialog = AboutDialog(self)
        dialog.exec()
    def on_crease_count_changed(self, count):
        self.label_crease_count.setText(f"折痕数：{count}")
    def on_crease_updated(self, crease_id):
        self.canvas_widget.refresh_crease(crease_id)
    def on_layer_changed(self, layer_id):
        self.canvas_widget.refresh_layer(layer_id)
    def on_issue_selected(self, issue):
        self.canvas_widget.highlight_issue(issue)
    def closeEvent(self, event):
        if self.current_project_id is not None:
            reply = QMessageBox.question(
                self, "退出确认", "是否保存当前工程后退出？",
                QMessageBox.Yes | QMessageBox.No | QMessageBox.Cancel
            )
            if reply == QMessageBox.Cancel:
```

```
                event.ignore()
                return
            if reply == QMessageBox.Yes:
                try:
                    self.project_service.save_project(self.current_project_id)
                except Exception:
                    logger.exception("退出前保存工程失败")
        logger.info("主窗口关闭")
        event.accept()
# 文件: app/ui/preview_widget.py
import math
from typing import List, Optional, Tuple
from PySide6.QtCore import Qt, QPointF, Signal
from PySide6.QtGui import QColor, QPainter, QPen, QFont, QWheelEvent, QMouseEvent
from PySide6.QtWidgets import (
    QWidget,
    QVBoxLayout,
    QHBoxLayout,
    QLabel,
    QPushButton,
    QSlider,
    QComboBox,
    QGroupBox,
    QFormLayout,
    QSizePolicy,
)
from app.common import logger
from app.core.geometry import Point2D, Segment2D
from app.models.crease import CreaseType
class FoldPreviewCanvas(QWidget):
    view_changed = Signal(float, float, float)
    def __init__(self, parent: Optional[QWidget] = None) -> None:
        super().__init__(parent)
        self.setMinimumSize(420, 360)
        self.setSizePolicy(QSizePolicy.Expanding, QSizePolicy.Expanding)
        self.setMouseTracking(True)
        self.setAutoFillBackground(True)
        self._segments: List[Segment2D] = []
        self._fold_angle = 90.0
        self._yaw = 35.0
        self._pitch = 28.0
        self._zoom = 1.0
        self._pan = QPointF(0.0, 0.0)
        self._last_mouse = QPointF(0.0, 0.0)
        self._dragging = False
        self._show_grid = True
        self._show_axes = True
    def set_segments(self, segments: List[Segment2D]) -> None:
        self._segments = list(segments)
        self.update()
    def set_fold_angle(self, angle: float) -> None:
        self._fold_angle = max(0.0, min(180.0, angle))
        self.update()
    def set_view(self, yaw: float, pitch: float, zoom: float) -> None:
        self._yaw = yaw
        self._pitch = max(-89.0, min(89.0, pitch))
        self._zoom = max(0.2, min(6.0, zoom))
        self.update()
    def reset_view(self) -> None:
        self._yaw = 35.0
        self._pitch = 28.0
        self._zoom = 1.0
        self._pan = QPointF(0.0, 0.0)
        self.update()
    def toggle_grid(self, visible: bool) -> None:
        self._show_grid = visible
        self.update()
    def toggle_axes(self, visible: bool) -> None:
        self._show_axes = visible
        self.update()
    def _project(self, x: float, y: float, z: float) -> QPointF:
        yaw = math.radians(self._yaw)
        pitch = math.radians(self._pitch)
        cos_y, sin_y = math.cos(yaw), math.sin(yaw)
        cos_p, sin_p = math.cos(pitch), math.sin(pitch)
        rx = x * cos_y - y * sin_y
        ry = x * sin_y + y * cos_y
        rz = z
        sx = rx
        sy = ry * cos_p - rz * sin_p
        depth = ry * sin_p + rz * cos_p
        scale = 180.0 * self._zoom
        cx = self.width() / 2.0 + self._pan.x()
```

```
        cy = self.height() / 2.0 + self._pan.y()
        return QPointF(cx + sx * scale, cy - sy * scale + depth * 0.0)
    def _folded_point(self, point: Point2D) -> Tuple[float, float, float]:
        angle = math.radians(self._fold_angle)
        fold_factor = math.sin(angle)
        z = point.y * fold_factor * 0.6
        return point.x, point.y * math.cos(angle * 0.5), z
    def paintEvent(self, event) -> None:
        painter = QPainter(self)
        painter.setRenderHint(QPainter.Antialiasing, True)
        painter.fillRect(self.rect(), QColor(28, 30, 36))
        if self._show_grid:
            self._draw_grid(painter)
        if self._show_axes:
            self._draw_axes(painter)
        if not self._segments:
            painter.setPen(QColor(150, 155, 165))
            painter.setFont(QFont("Microsoft YaHei", 11))
            painter.drawText(self.rect(), Qt.AlignCenter, "暂无折痕数据，请先在编辑区绘制折痕图")
            painter.end()
            return
        for segment in self._segments:
            self._draw_segment(painter, segment)
        painter.end()
    def _draw_grid(self, painter: QPainter) -> None:
        pen = QPen(QColor(48, 52, 60))
        pen.setWidth(1)
        painter.setPen(pen)
        extent = 4
        for i in range(-extent, extent + 1):
            p1 = self._project(float(i), -float(extent), 0.0)
            p2 = self._project(float(i), float(extent), 0.0)
            painter.drawLine(p1, p2)
            p3 = self._project(-float(extent), float(i), 0.0)
            p4 = self._project(float(extent), float(i), 0.0)
            painter.drawLine(p3, p4)
    def _draw_axes(self, painter: QPainter) -> None:
        origin = self._project(0.0, 0.0, 0.0)
        axes = [
            (self._project(1.6, 0.0, 0.0), QColor(220, 80, 80), "X"),
            (self._project(0.0, 1.6, 0.0), QColor(80, 200, 100), "Y"),
            (self._project(0.0, 0.0, 1.6), QColor(90, 140, 240), "Z"),
        ]
        painter.setFont(QFont("Microsoft YaHei", 9))
        for end, color, label in axes:
            pen = QPen(color)
            pen.setWidth(2)
            painter.setPen(pen)
            painter.drawLine(origin, end)
            painter.drawText(end + QPointF(4.0, -4.0), label)
    def _draw_segment(self, painter: QPainter, segment: Segment2D) -> None:
        x1, y1, z1 = self._folded_point(segment.start)
        x2, y2, z2 = self._folded_point(segment.end)
        p1 = self._project(x1, y1, z1)
        p2 = self._project(x2, y2, z2)
        color, width, style = self._style_for(segment.crease_type)
        pen = QPen(color)
        pen.setWidthF(width)
        pen.setStyle(style)
        pen.setCapStyle(Qt.RoundCap)
        painter.setPen(pen)
        painter.drawLine(p1, p2)
        if segment.crease_type in (CreaseType.MOUNTAIN, CreaseType.VALLEY):
            self._draw_fold_marker(painter, p1, p2, segment.crease_type)
    def _style_for(self, crease_type: CreaseType) -> Tuple[QColor, float, Qt.PenStyle]:
        if crease_type == CreaseType.MOUNTAIN:
            return QColor(235, 90, 90), 2.2, Qt.SolidLine
        if crease_type == CreaseType.VALLEY:
            return QColor(90, 170, 240), 2.2, Qt.SolidLine
        if crease_type == CreaseType.CUT:
            return QColor(230, 230, 230), 1.6, Qt.DashLine
        if crease_type == CreaseType.BOUNDARY:
            return QColor(160, 165, 175), 1.8, Qt.SolidLine
        return QColor(140, 145, 155), 1.2, Qt.DotLine
    def _draw_fold_marker(self, painter: QPainter, p1: QPointF, p2: QPointF, crease_type: CreaseType) -> None:
        dx = p2.x() - p1.x()
        dy = p2.y() - p1.y()
        length = math.hypot(dx, dy)
        if length < 1.0:
            return
        ux, uy = dx / length, dy / length
        nx, ny = -uy, ux
        marker_len = 7.0
```

```
        marker_gap = 22.0
        count = max(1, int(length // marker_gap))
        color = QColor(235, 90, 90) if crease_type == CreaseType.MOUNTAIN else QColor(90, 170, 240)
        pen = QPen(color)
        pen.setWidthF(1.6)
        painter.setPen(pen)
        for i in range(count):
            t = (i + 0.5) / count
            cx = p1.x() + dx * t
            cy = p1.y() + dy * t
            tip = QPointF(cx + nx * marker_len, cy + ny * marker_len)
            base_a = QPointF(cx - ux * 4.0, cy - uy * 4.0)
            base_b = QPointF(cx + ux * 4.0, cy + uy * 4.0)
            painter.drawLine(base_a, tip)
            painter.drawLine(base_b, tip)
    def mousePressEvent(self, event: QMouseEvent) -> None:
        if event.button() == Qt.LeftButton:
            self._dragging = True
            self._last_mouse = event.position()
            self.setCursor(Qt.ClosedHandCursor)
    def mouseMoveEvent(self, event: QMouseEvent) -> None:
        if not self._dragging:
            return
        pos = event.position()
        delta = pos - self._last_mouse
        self._last_mouse = pos
        if event.modifiers() & Qt.ShiftModifier:
            self._yaw += delta.x() * 0.4
            self._pitch = max(-89.0, min(89.0, self._pitch + delta.y() * 0.4))
        else:
            self._pan += delta
        self.view_changed.emit(self._yaw, self._pitch, self._zoom)
        self.update()
    def mouseReleaseEvent(self, event: QMouseEvent) -> None:
        if event.button() == Qt.LeftButton:
            self._dragging = False
            self.setCursor(Qt.ArrowCursor)
    def wheelEvent(self, event: QWheelEvent) -> None:
        steps = event.angleDelta().y() / 120.0
        self._zoom = max(0.2, min(6.0, self._zoom * (1.0 + steps * 0.12)))
        self.view_changed.emit(self._yaw, self._pitch, self._zoom)
        self.update()
class PreviewWidget(QWidget):
    def __init__(self, parent: Optional[QWidget] = None) -> None:
        super().__init__(parent)
        self._segments: List[Segment2D] = []
        self._build_ui()
    def _build_ui(self) -> None:
        root = QVBoxLayout(self)
        root.setContentsMargins(8, 8, 8, 8)
        root.setSpacing(8)
        self.canvas = FoldPreviewCanvas(self)
        self.canvas.view_changed.connect(self._on_view_changed)
        root.addWidget(self.canvas, 1)
        control_box = QGroupBox("折叠预览控制")
        form = QFormLayout(control_box)
        form.setLabelAlignment(Qt.AlignRight)
        self.angle_slider = QSlider(Qt.Horizontal)
        self.angle_slider.setRange(0, 180)
        self.angle_slider.setValue(90)
        self.angle_slider.valueChanged.connect(self._on_angle_changed)
        self.angle_label = QLabel("90°")
        angle_row = QHBoxLayout()
        angle_row.addWidget(self.angle_slider, 1)
        angle_row.addWidget(self.angle_label)
        form.addRow("折叠角度", angle_row)
        self.projection_combo = QComboBox()
        self.projection_combo.addItems(["等轴测投影", "俯视投影", "正视投影"])
        self.projection_combo.currentIndexChanged.connect(self._on_projection_changed)
        form.addRow("投影方式", self.projection_combo)
        self.grid_combo = QComboBox()
        self.grid_combo.addItems(["显示网格", "隐藏网格"])
        self.grid_combo.currentIndexChanged.connect(self._on_grid_changed)
        form.addRow("参考网格", self.grid_combo)
        self.axes_combo = QComboBox()
        self.axes_combo.addItems(["显示坐标轴", "隐藏坐标轴"])
        self.axes_combo.currentIndexChanged.connect(self._on_axes_changed)
        form.addRow("坐标轴", self.axes_combo)
        root.addWidget(control_box)
        button_row = QHBoxLayout()
        self.reset_button = QPushButton("重置视角")
        self.reset_button.clicked.connect(self.canvas.reset_view)
```

```
        self.refresh_button = QPushButton("刷新预览")
        self.refresh_button.clicked.connect(self.refresh)
        button_row.addWidget(self.reset_button)
        button_row.addWidget(self.refresh_button)
        button_row.addStretch(1)
        root.addLayout(button_row)
        self.status_label = QLabel("折痕线段：0")
        self.status_label.setAlignment(Qt.AlignLeft)
        root.addWidget(self.status_label)
    def set_segments(self, segments: List[Segment2D]) -> None:
        self._segments = list(segments)
        self.canvas.set_segments(self._segments)
        self.status_label.setText(f"折痕线段：{len(self._segments)}")
    def refresh(self) -> None:
        self.canvas.set_segments(self._segments)
        logger.debug("三维预览已刷新，线段数量=%d", len(self._segments))
    def set_fold_angle(self, angle: float) -> None:
        self.angle_slider.setValue(int(angle))
    def _on_angle_changed(self, value: int) -> None:
        self.angle_label.setText(f"{value}°")
        self.canvas.set_fold_angle(float(value))
    def _on_projection_changed(self, index: int) -> None:
        if index == 0:
            self.canvas.set_view(35.0, 28.0, self.canvas._zoom)
        elif index == 1:
            self.canvas.set_view(0.0, 89.0, self.canvas._zoom)
        else:
            self.canvas.set_view(0.0, 0.0, self.canvas._zoom)
    def _on_grid_changed(self, index: int) -> None:
        self.canvas.toggle_grid(index == 0)
    def _on_axes_changed(self, index: int) -> None:
        self.canvas.toggle_axes(index == 0)
    def _on_view_changed(self, yaw: float, pitch: float, zoom: float) -> None:
        logger.debug("预览视角变化 yaw=%.1f pitch=%.1f zoom=%.2f", yaw, pitch, zoom)
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/ui/settings_dialog.py
from PySide6.QtCore import Signal
from PySide6.QtWidgets import QDialog, QVBoxLayout, QHBoxLayout, QFormLayout, QSpinBox, QDoubleSpinBox,
QComboBox, QCheckBox, QPushButton, QColorDialog, QTabWidget, QWidget, QLineEdit, QMessageBox, QFileDialog
from app.common import logger
from app.core.settings import settings
from app.utils.color_utils import color_to_hex, hex_to_color
class ColorButton(QPushButton):
    color_changed = Signal(str)
    def __init__(self, color_hex: str, parent=None):
        super().__init__(parent)
        self._color_hex = color_hex
        self.setFixedSize(64, 26)
        self.clicked.connect(self._choose_color)
        self._refresh_style()
    def _refresh_style(self):
        self.setStyleSheet(
            f"background-color: {self._color_hex}; border: 1px solid
        )
        self.setText(self._color_hex.upper())
    def color_hex(self) -> str:
        return self._color_hex
    def set_color_hex(self, value: str):
        self._color_hex = value
        self._refresh_style()
    def _choose_color(self):
        initial = hex_to_color(self._color_hex)
        chosen = QColorDialog.getColor(initial, self, "选择颜色")
        if chosen.isValid():
            self._color_hex = color_to_hex(chosen)
            self._refresh_style()
            self.color_changed.emit(self._color_hex)
class SettingsDialog(QDialog):
    settings_applied = Signal()
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle("系统设置 – 折痕图可视化编辑与标准化导出系统")
        self.setMinimumWidth(560)
        self.setMinimumHeight(480)
        self._build_ui()
        self._load_settings()
    def _build_ui(self):
        root = QVBoxLayout(self)
        self.tabs = QTabWidget(self)
        self.tabs.addTab(self._build_canvas_tab(), "画布与网格")
```

```
        self.tabs.addTab(self._build_crease_tab(), "折痕样式")
        self.tabs.addTab(self._build_validation_tab(), "校验规则")
        self.tabs.addTab(self._build_export_tab(), "导出设置")
        root.addWidget(self.tabs)
        button_row = QHBoxLayout()
        button_row.addStretch(1)
        self.btn_reset = QPushButton("恢复默认")
        self.btn_cancel = QPushButton("取消")
        self.btn_apply = QPushButton("应用")
        self.btn_ok = QPushButton("确定")
        self.btn_reset.clicked.connect(self._reset_defaults)
        self.btn_cancel.clicked.connect(self.reject)
        self.btn_apply.clicked.connect(self._apply_settings)
        self.btn_ok.clicked.connect(self._accept_settings)
        button_row.addWidget(self.btn_reset)
        button_row.addWidget(self.btn_cancel)
        button_row.addWidget(self.btn_apply)
        button_row.addWidget(self.btn_ok)
        root.addLayout(button_row)
    def _build_canvas_tab(self) -> QWidget:
        page = QWidget()
        form = QFormLayout(page)
        self.spin_grid_size = QDoubleSpinBox()
        self.spin_grid_size.setRange(1.0, 200.0)
        self.spin_grid_size.setSingleStep(1.0)
        self.spin_grid_size.setSuffix(" mm")
        form.addRow("网格间距", self.spin_grid_size)
        self.spin_snap_tolerance = QDoubleSpinBox()
        self.spin_snap_tolerance.setRange(0.0, 20.0)
        self.spin_snap_tolerance.setSingleStep(0.5)
        self.spin_snap_tolerance.setSuffix(" px")
        form.addRow("吸附容差", self.spin_snap_tolerance)
        self.chk_show_grid = QCheckBox("显示网格")
        self.chk_show_axes = QCheckBox("显示坐标轴")
        self.chk_snap_grid = QCheckBox("吸附到网格")
        self.chk_snap_vertex = QCheckBox("吸附到顶点")
        form.addRow(self.chk_show_grid)
        form.addRow(self.chk_show_axes)
        form.addRow(self.chk_snap_grid)
        form.addRow(self.chk_snap_vertex)
        self.combo_render_mode = QComboBox()
        self.combo_render_mode.addItems(["抗锯齿", "快速", "高质量"])
        form.addRow("渲染模式", self.combo_render_mode)
        self.spin_zoom_step = QDoubleSpinBox()
        self.spin_zoom_step.setRange(1.05, 3.0)
        self.spin_zoom_step.setSingleStep(0.05)
        form.addRow("缩放步进", self.spin_zoom_step)
        return page
    def _build_crease_tab(self) -> QWidget:
        page = QWidget()
        form = QFormLayout(page)
        self.color_mountain = ColorButton("
        self.color_valley = ColorButton("
        self.color_flat = ColorButton("
        self.color_cut = ColorButton("
        self.color_boundary = ColorButton("
        self.color_auxiliary = ColorButton("
        form.addRow("峰折线颜色", self.color_mountain)
        form.addRow("谷折线颜色", self.color_valley)
        form.addRow("段折线颜色", self.color_flat)
        form.addRow("剪裁线颜色", self.color_cut)
        form.addRow("边界线颜色", self.color_boundary)
        form.addRow("辅助线颜色", self.color_auxiliary)
        self.spin_line_width = QDoubleSpinBox()
        self.spin_line_width.setRange(0.5, 10.0)
        self.spin_line_width.setSingleStep(0.5)
        self.spin_line_width.setSuffix(" px")
        form.addRow("线宽", self.spin_line_width)
        self.combo_line_style = QComboBox()
        self.combo_line_style.addItems(["实线", "虚线", "点线", "点划线"])
        form.addRow("线型", self.combo_line_style)
        self.spin_vertex_radius = QDoubleSpinBox()
        self.spin_vertex_radius.setRange(1.0, 12.0)
        self.spin_vertex_radius.setSingleStep(0.5)
        self.spin_vertex_radius.setSuffix(" px")
        form.addRow("顶点半径", self.spin_vertex_radius)
        return page
    def _build_validation_tab(self) -> QWidget:
        page = QWidget()
        form = QFormLayout(page)
        self.chk_auto_validate = QCheckBox("编辑时自动校验")
        self.chk_check_angle = QCheckBox("检测顶点角度异常")
```

```
        self.chk_check_cross = QCheckBox("检测折痕交叉冲突")
        self.chk_check_overlap = QCheckBox("检测面片重叠")
        self.chk_check_foldable = QCheckBox("检测可折叠性")
        form.addRow(self.chk_auto_validate)
        form.addRow(self.chk_check_angle)
        form.addRow(self.chk_check_cross)
        form.addRow(self.chk_check_overlap)
        form.addRow(self.chk_check_foldable)
        self.spin_angle_tolerance = QDoubleSpinBox()
        self.spin_angle_tolerance.setRange(0.1, 30.0)
        self.spin_angle_tolerance.setSingleStep(0.5)
        self.spin_angle_tolerance.setSuffix(" °")
        form.addRow("角度容差", self.spin_angle_tolerance)
        self.spin_min_segment = QDoubleSpinBox()
        self.spin_min_segment.setRange(0.0, 50.0)
        self.spin_min_segment.setSingleStep(0.5)
        self.spin_min_segment.setSuffix(" mm")
        form.addRow("最短线段", self.spin_min_segment)
        self.combo_severity = QComboBox()
        self.combo_severity.addItems(["提示", "警告", "错误"])
        form.addRow("最低报告级别", self.combo_severity)
        return page
    def _build_export_tab(self) -> QWidget:
        page = QWidget()
        form = QFormLayout(page)
        self.edit_default_dir = QLineEdit()
        browse = QPushButton("浏览…")
        browse.clicked.connect(self._choose_export_dir)
        dir_row = QHBoxLayout()
        dir_row.addWidget(self.edit_default_dir, 1)
        dir_row.addWidget(browse)
        dir_widget = QWidget()
        dir_widget.setLayout(dir_row)
        form.addRow("默认导出目录", dir_widget)
        self.combo_default_format = QComboBox()
        self.combo_default_format.addItems(["SVG", "DXF", "FOLD"])
        form.addRow("默认导出格式", self.combo_default_format)
        self.combo_unit = QComboBox()
        self.combo_unit.addItems(["毫米", "厘米", "英寸"])
        form.addRow("导出单位", self.combo_unit)
        self.spin_precision = QSpinBox()
        self.spin_precision.setRange(0, 8)
        form.addRow("坐标精度", self.spin_precision)
        self.chk_export_meta = QCheckBox("导出时包含元数据")
        self.chk_export_layer = QCheckBox("导出时按图层分组")
        self.chk_export_aux = QCheckBox("导出包含辅助线")
        form.addRow(self.chk_export_meta)
        form.addRow(self.chk_export_layer)
        form.addRow(self.chk_export_aux)
        self.spin_auto_save = QSpinBox()
        self.spin_auto_save.setRange(0, 120)
        self.spin_auto_save.setSuffix(" 分钟")
        form.addRow("自动保存间隔", self.spin_auto_save)
        return page
    def _choose_export_dir(self):
        current = self.edit_default_dir.text().strip()
        chosen = QFileDialog.getExistingDirectory(self, "选择默认导出目录", current)
        if chosen:
            self.edit_default_dir.setText(chosen)
    def _load_settings(self):
        try:
            self.spin_grid_size.setValue(float(settings.get("canvas.grid_size", 10.0)))
            self.spin_snap_tolerance.setValue(float(settings.get("canvas.snap_tolerance", 6.0)))
            self.chk_show_grid.setChecked(bool(settings.get("canvas.show_grid", True)))
            self.chk_show_axes.setChecked(bool(settings.get("canvas.show_axes", True)))
            self.chk_snap_grid.setChecked(bool(settings.get("canvas.snap_grid", True)))
            self.chk_snap_vertex.setChecked(bool(settings.get("canvas.snap_vertex", True)))
            self.combo_render_mode.setCurrentText(str(settings.get("canvas.render_mode", "抗锯齿")))
            self.spin_zoom_step.setValue(float(settings.get("canvas.zoom_step", 1.2)))
            self.color_mountain.set_color_hex(str(settings.get("crease.color_mountain", "
            self.color_valley.set_color_hex(str(settings.get("crease.color_valley", "
            self.color_flat.set_color_hex(str(settings.get("crease.color_flat", "
            self.color_cut.set_color_hex(str(settings.get("crease.color_cut", "
            self.color_boundary.set_color_hex(str(settings.get("crease.color_boundary", "
            self.color_auxiliary.set_color_hex(str(settings.get("crease.color_auxiliary", "
            self.spin_line_width.setValue(float(settings.get("crease.line_width", 1.5)))
            self.combo_line_style.setCurrentText(str(settings.get("crease.line_style", "实线")))
            self.spin_vertex_radius.setValue(float(settings.get("crease.vertex_radius", 3.0)))
            self.chk_auto_validate.setChecked(bool(settings.get("validation.auto", True)))
            self.chk_check_angle.setChecked(bool(settings.get("validation.check_angle", True)))
            self.chk_check_cross.setChecked(bool(settings.get("validation.check_cross", True)))
            self.chk_check_overlap.setChecked(bool(settings.get("validation.check_overlap", True)))
```

```
            self.chk_check_foldable.setChecked(bool(settings.get("validation.check_foldable", True)))
            self.spin_angle_tolerance.setValue(float(settings.get("validation.angle_tolerance", 1.0)))
            self.spin_min_segment.setValue(float(settings.get("validation.min_segment", 0.5)))
            self.combo_severity.setCurrentText(str(settings.get("validation.severity", "警告")))
            self.edit_default_dir.setText(str(settings.get("export.default_dir", "")))
            self.combo_default_format.setCurrentText(str(settings.get("export.default_format", "SVG")))
            self.combo_unit.setCurrentText(str(settings.get("export.unit", "毫米")))
            self.spin_precision.setValue(int(settings.get("export.precision", 4)))
            self.chk_export_meta.setChecked(bool(settings.get("export.include_meta", True)))
            self.chk_export_layer.setChecked(bool(settings.get("export.group_layer", True)))
            self.chk_export_aux.setChecked(bool(settings.get("export.include_aux", False)))
            self.spin_auto_save.setValue(int(settings.get("export.auto_save_minutes", 10)))
        except Exception as exc:
            logger.error("加载系统设置失败: %s", exc)
    def _collect_settings(self) -> dict:
        return {
            "canvas.grid_size": self.spin_grid_size.value(),
            "canvas.snap_tolerance": self.spin_snap_tolerance.value(),
            "canvas.show_grid": self.chk_show_grid.isChecked(),
            "canvas.show_axes": self.chk_show_axes.isChecked(),
            "canvas.snap_grid": self.chk_snap_grid.isChecked(),
            "canvas.snap_vertex": self.chk_snap_vertex.isChecked(),
            "canvas.render_mode": self.combo_render_mode.currentText(),
            "canvas.zoom_step": self.spin_zoom_step.value(),
            "crease.color_mountain": self.color_mountain.color_hex(),
            "crease.color_valley": self.color_valley.color_hex(),
            "crease.color_flat": self.color_flat.color_hex(),
            "crease.color_cut": self.color_cut.color_hex(),
            "crease.color_boundary": self.color_boundary.color_hex(),
            "crease.color_auxiliary": self.color_auxiliary.color_hex(),
            "crease.line_width": self.spin_line_width.value(),
            "crease.line_style": self.combo_line_style.currentText(),
            "crease.vertex_radius": self.spin_vertex_radius.value(),
            "validation.auto": self.chk_auto_validate.isChecked(),
            "validation.check_angle": self.chk_check_angle.isChecked(),
            "validation.check_cross": self.chk_check_cross.isChecked(),
            "validation.check_overlap": self.chk_check_overlap.isChecked(),
            "validation.check_foldable": self.chk_check_foldable.isChecked(),
            "validation.angle_tolerance": self.spin_angle_tolerance.value(),
            "validation.min_segment": self.spin_min_segment.value(),
            "validation.severity": self.combo_severity.currentText(),
            "export.default_dir": self.edit_default_dir.text().strip(),
            "export.default_format": self.combo_default_format.currentText(),
            "export.unit": self.combo_unit.currentText(),
            "export.precision": self.spin_precision.value(),
            "export.include_meta": self.chk_export_meta.isChecked(),
            "export.group_layer": self.chk_export_layer.isChecked(),
            "export.include_aux": self.chk_export_aux.isChecked(),
            "export.auto_save_minutes": self.spin_auto_save.value(),
        }
    def _apply_settings(self) -> bool:
        try:
            data = self._collect_settings()
            for key, value in data.items():
                settings.set(key, value)
            settings.save()
            self.settings_applied.emit()
            logger.info("系统设置已更新，共 %d 项", len(data))
            return True
        except Exception as exc:
            logger.error("保存系统设置失败: %s", exc)
            QMessageBox.critical(self, "错误", f"保存设置失败：{exc}")
            return False
    def _accept_settings(self):
        if self._apply_settings():
            self.accept()
    def _reset_defaults(self):
        confirm = QMessageBox.question(
            self,
            "恢复默认",
            "确定要将所有设置恢复为默认值吗？",
            QMessageBox.Yes | QMessageBox.No,
        )
        if confirm != QMessageBox.Yes:
            return
        try:
            settings.reset_defaults()
            self._load_settings()
            logger.info("系统设置已恢复默认值")
        except Exception as exc:
            logger.error("恢复默认设置失败: %s", exc)
            QMessageBox.warning(self, "警告", f"恢复默认设置失败：{exc}")
```

```
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/ui/validation_panel.py
from PySide6.QtCore import Qt, Signal
from PySide6.QtGui import QColor, QBrush
from PySide6.QtWidgets import (
    QWidget,
    QVBoxLayout,
    QHBoxLayout,
    QPushButton,
    QTableWidget,
    QTableWidgetItem,
    QHeaderView,
    QLabel,
    QComboBox,
    QMessageBox,
    QAbstractItemView,
)
from app.common import logger
from app.core.constraints import SeverityLevel
from app.services.validation_service import ValidationService
SEVERITY_LABELS = {
    SeverityLevel.ERROR: "错误",
    SeverityLevel.WARNING: "警告",
    SeverityLevel.INFO: "提示",
}
SEVERITY_COLORS = {
    SeverityLevel.ERROR: QColor(220, 80, 80),
    SeverityLevel.WARNING: QColor(230, 160, 40),
    SeverityLevel.INFO: QColor(70, 130, 200),
}
class ValidationPanel(QWidget):
    issue_activated = Signal(object)
    def __init__(self, validation_service: ValidationService, parent=None):
        super().__init__(parent)
        self._service = validation_service
        self._issues = []
        self._build_ui()
    def _build_ui(self):
        layout = QVBoxLayout(self)
        layout.setContentsMargins(6, 6, 6, 6)
        layout.setSpacing(6)
        toolbar = QHBoxLayout()
        self._run_button = QPushButton("执行校验")
        self._run_button.clicked.connect(self.run_validation)
        toolbar.addWidget(self._run_button)
        toolbar.addWidget(QLabel("级别"))
        self._filter_combo = QComboBox()
        self._filter_combo.addItem("全部", None)
        self._filter_combo.addItem("错误", SeverityLevel.ERROR)
        self._filter_combo.addItem("警告", SeverityLevel.WARNING)
        self._filter_combo.addItem("提示", SeverityLevel.INFO)
        self._filter_combo.currentIndexChanged.connect(self._apply_filter)
        toolbar.addWidget(self._filter_combo)
        self._clear_button = QPushButton("清空")
        self._clear_button.clicked.connect(self.clear_issues)
        toolbar.addWidget(self._clear_button)
        toolbar.addStretch(1)
        layout.addLayout(toolbar)
        self._summary_label = QLabel("尚未执行校验")
        layout.addWidget(self._summary_label)
        self._table = QTableWidget(0, 4)
        self._table.setHorizontalHeaderLabels(["级别", "位置", "描述", "建议"])
        self._table.horizontalHeader().setSectionResizeMode(0, QHeaderView.ResizeToContents)
        self._table.horizontalHeader().setSectionResizeMode(1, QHeaderView.ResizeToContents)
        self._table.horizontalHeader().setSectionResizeMode(2, QHeaderView.Stretch)
        self._table.horizontalHeader().setSectionResizeMode(3, QHeaderView.Stretch)
        self._table.setSelectionBehavior(QAbstractItemView.SelectRows)
        self._table.setSelectionMode(QAbstractItemView.SingleSelection)
        self._table.setEditTriggers(QAbstractItemView.NoEditTriggers)
        self._table.verticalHeader().setVisible(False)
        self._table.itemDoubleClicked.connect(self._on_item_double_clicked)
        layout.addWidget(self._table, 1)
    def run_validation(self):
        try:
            report = self._service.validate_current_project()
        except Exception as exc:
            logger.exception("折痕图校验执行失败")
            QMessageBox.critical(self, "校验失败", f"校验过程中出现异常□{exc}")
            return
        self._issues = list(report.issues)
```

```
        self._refresh_summary(report)
        self._apply_filter()
    def set_issues(self, issues):
        self._issues = list(issues)
        self._summary_label.setText(f"共 {len(self._issues)} 条校验结果")
        self._apply_filter()
    def clear_issues(self):
        self._issues = []
        self._table.setRowCount(0)
        self._summary_label.setText("尚未执行校验")
    def _refresh_summary(self, report):
        error_count = sum(1 for i in self._issues if i.severity == SeverityLevel.ERROR)
        warning_count = sum(1 for i in self._issues if i.severity == SeverityLevel.WARNING)
        info_count = sum(1 for i in self._issues if i.severity == SeverityLevel.INFO)
        self._summary_label.setText(
            f"错误 {error_count} · 警告 {warning_count} · 提示 {info_count} · 合计 {len(self._issues)}"
        )
    def _apply_filter(self):
        selected = self._filter_combo.currentData()
        if selected is None:
            visible = list(self._issues)
        else:
            visible = [i for i in self._issues if i.severity == selected]
        self._populate_table(visible)
    def _populate_table(self, issues):
        self._table.setRowCount(0)
        for issue in issues:
            row = self._table.rowCount()
            self._table.insertRow(row)
            severity_item = QTableWidgetItem(SEVERITY_LABELS.get(issue.severity, "未知"))
            color = SEVERITY_COLORS.get(issue.severity)
            if color is not None:
                severity_item.setForeground(QBrush(color))
            severity_item.setData(Qt.UserRole, issue)
            self._table.setItem(row, 0, severity_item)
            self._table.setItem(row, 1, QTableWidgetItem(issue.location_text()))
            desc_item = QTableWidgetItem(issue.message)
            desc_item.setToolTip(issue.message)
            self._table.setItem(row, 2, desc_item)
            suggestion_item = QTableWidgetItem(issue.suggestion or "-")
            suggestion_item.setToolTip(issue.suggestion or "")
            self._table.setItem(row, 3, suggestion_item)
    def _on_item_double_clicked(self, item):
        issue = self._table.item(item.row(), 0).data(Qt.UserRole)
        if issue is not None:
            self.issue_activated.emit(issue)
    def selected_issue(self):
        row = self._table.currentRow()
        if row < 0:
            return None
        item = self._table.item(row, 0)
        if item is None:
            return None
        return item.data(Qt.UserRole)
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/io/__init__.py
import os
from typing import Any, Callable, Dict, List, Optional, Tuple
from app.common import logger
from app.exceptions import ExportError, ImportError as CreaseImportError
SUPPORTED_IMPORT_FORMATS: Dict[str, str] = {
    ".fold": "FOLD 折痕图交换格式",
    ".json": "JSON 折痕数据",
    ".cp": "折痕图文本格式",
}
SUPPORTED_EXPORT_FORMATS: Dict[str, str] = {
    ".svg": "SVG 矢量折痕图",
    ".dxf": "DXF 工程制图交换格式",
    ".fold": "FOLD 折痕图交换格式",
}
FORMAT_ALIASES: Dict[str, str] = {
    "fold": ".fold",
    "json": ".json",
    "cp": ".cp",
    "svg": ".svg",
    "dxf": ".dxf",
}
EXPORT_FORMAT_KEYS: Tuple[str, ...] = ("svg", "dxf", "fold")
IMPORT_FORMAT_KEYS: Tuple[str, ...] = ("fold", "json", "cp")
def normalize_format(fmt: str) -> str:
```

```
    if not fmt:
        raise ValueError("格式标识不能为空")
    key = fmt.strip().lower()
    if key.startswith("."):
        key = key[1:]
    if key not in FORMAT_ALIASES:
        raise ValueError(f"不支持的格式标识: {fmt}")
    return FORMAT_ALIASES[key]
def is_import_supported(fmt: str) -> bool:
    try:
        return normalize_format(fmt) in SUPPORTED_IMPORT_FORMATS
    except ValueError:
        return False
def is_export_supported(fmt: str) -> bool:
    try:
        return normalize_format(fmt) in SUPPORTED_EXPORT_FORMATS
    except ValueError:
        return False
def guess_format_from_path(path: str) -> Optional[str]:
    if not path:
        return None
    ext = os.path.splitext(path)[1].lower()
    if ext in SUPPORTED_IMPORT_FORMATS or ext in SUPPORTED_EXPORT_FORMATS:
        return ext
    return None
def build_import_filter() -> str:
    parts = [f"*{ext}" for ext in SUPPORTED_IMPORT_FORMATS]
    return "折痕图文件 (" + " ".join(parts) + ");;所有文件 (*.*)"
def build_export_filter() -> str:
    parts = [f"*{ext}" for ext in SUPPORTED_EXPORT_FORMATS]
    return "折痕图导出 (" + " ".join(parts) + ");;所有文件 (*.*)"
def resolve_importer(fmt: str) -> Callable[..., Any]:
    target = normalize_format(fmt)
    if target == ".fold":
        from app.io.fold_importer import load_fold_file
        return load_fold_file
    if target == ".json":
        from app.io.fold_importer import load_json_file
        return load_json_file
    if target == ".cp":
        from app.io.fold_importer import load_cp_file
        return load_cp_file
    raise CreaseImportError(f"没有可用的导入器处理格式: {fmt}")
def resolve_exporter(fmt: str) -> Callable[..., Any]:
    target = normalize_format(fmt)
    if target == ".svg":
        from app.io.svg_exporter import export_svg
        return export_svg
    if target == ".dxf":
        from app.io.dxf_exporter import export_dxf
        return export_dxf
    if target == ".fold":
        from app.io.fold_exporter import export_fold
        return export_fold
    raise ExportError(f"没有可用的导出器处理格式: {fmt}")
def import_crease_file(path: str, fmt: Optional[str] = None) -> Dict[str, Any]:
    if not path:
        raise CreaseImportError("导入路径不能为空")
    if not os.path.isfile(path):
        raise CreaseImportError(f"文件不存在: {path}")
    target = fmt or guess_format_from_path(path)
    if target is None:
        raise CreaseImportError(f"无法从路径推断格式: {path}")
    importer = resolve_importer(target)
    try:
        data = importer(path)
    except CreaseImportError:
        raise
    except Exception as exc:
        logger.exception("导入折痕文件失败: %s", path)
        raise CreaseImportError(f"导入失败: {exc}") from exc
    if not isinstance(data, dict):
        raise CreaseImportError("导入器返回的数据结构非法")
    data.setdefault("source_path", path)
    data.setdefault("source_format", normalize_format(target))
    logger.info("已导入折痕文件 %s，格式 %s", path, target)
    return data
def export_crease_file(
    path: str,
    crease_data: Dict[str, Any],
    fmt: Optional[str] = None,
    options: Optional[Dict[str, Any]] = None,
```

```
) -> str:
    if not path:
        raise ExportError("导出路径不能为空")
    if not isinstance(crease_data, dict):
        raise ExportError("折痕数据必须为字典结构")
    target = fmt or guess_format_from_path(path)
    if target is None:
        raise ExportError(f"无法从路径推断导出格式: {path}")
    exporter = resolve_exporter(target)
    opts = options or {}
    try:
        result_path = exporter(path, crease_data, **opts)
    except ExportError:
        raise
    except Exception as exc:
        logger.exception("导出折痕文件失败: %s", path)
        raise ExportError(f"导出失败: {exc}") from exc
    logger.info("已导出折痕文件 %s□格式 %s", result_path, target)
    return result_path
def validate_import_payload(data: Dict[str, Any]) -> List[str]:
    issues: List[str] = []
    if not isinstance(data, dict):
        return ["导入数据不是字典结构"]
    vertices = data.get("vertices")
    edges = data.get("edges")
    if vertices is None:
        issues.append("缺少 vertices 顶点集合")
    elif not isinstance(vertices, list):
        issues.append("vertices 必须为列表")
    elif len(vertices) == 0:
        issues.append("vertices 顶点集合为空")
    if edges is None:
        issues.append("缺少 edges 折痕集合")
    elif not isinstance(edges, list):
        issues.append("edges 必须为列表")
    if isinstance(vertices, list) and isinstance(edges, list):
        total = len(vertices)
        for idx, edge in enumerate(edges):
            if not isinstance(edge, (list, tuple)) or len(edge) < 2:
                issues.append(f"第 {idx} 条折痕结构非法")
                continue
            for vid in edge[:2]:
                if not isinstance(vid, int) or vid < 0 or vid >= total:
                    issues.append(f"第 {idx} 条折痕引用了越界顶点 {vid}")
    return issues
def summarize_import(data: Dict[str, Any]) -> Dict[str, Any]:
    vertices = data.get("vertices") or []
    edges = data.get("edges") or []
    assignments = data.get("assignment") or data.get("edge_types") or {}
    peak = valley = flat = boundary = 0
    if isinstance(assignments, dict):
        for value in assignments.values():
            tag = str(value).lower()
            if tag in ("m", "mountain", "peak"):
                peak += 1
            elif tag in ("v", "valley"):
                valley += 1
            elif tag in ("f", "flat"):
                flat += 1
            elif tag in ("b", "boundary"):
                boundary += 1
    return {
        "vertex_count": len(vertices),
        "edge_count": len(edges),
        "peak_count": peak,
        "valley_count": valley,
        "flat_count": flat,
        "boundary_count": boundary,
        "source_format": data.get("source_format"),
    }
__all__ = [
    "SUPPORTED_IMPORT_FORMATS",
    "SUPPORTED_EXPORT_FORMATS",
    "EXPORT_FORMAT_KEYS",
    "IMPORT_FORMAT_KEYS",
    "normalize_format",
    "is_import_supported",
    "is_export_supported",
    "guess_format_from_path",
    "build_import_filter",
    "build_export_filter",
    "resolve_importer",
```

```
    "resolve_exporter",
    "import_crease_file",
    "export_crease_file",
    "validate_import_payload",
    "summarize_import",
]
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/io/dxf_exporter.py
import math
from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple
from app.common import logger
from app.exceptions import ExportError
from app.models.crease import CreaseType
DXF_HEADER = "0\nSECTION\n2\nHEADER\n0\nENDSEC\n"
DXF_TABLES = (
    "0\nSECTION\n2\nTABLES\n"
    "0\nTABLE\n2\nLAYER\n70\n{layer_count}\n"
    "{layer_defs}"
    "0\nENDTAB\n0\nENDSEC\n"
)
DXF_ENTITIES_OPEN = "0\nSECTION\n2\nENTITIES\n"
DXF_ENTITIES_CLOSE = "0\nENDSEC\n0\nEOF\n"
LAYER_SPECS: Dict[str, Tuple[int, str]] = {
    "MOUNTAIN": (1, "Mountain crease"),
    "VALLEY": (3, "Valley crease"),
    "CUT": (5, "Cut line"),
    "BOUNDARY": (7, "Boundary"),
    "AUXILIARY": (8, "Auxiliary reference"),
}
CREASE_LAYER_MAP: Dict[CreaseType, str] = {
    CreaseType.MOUNTAIN: "MOUNTAIN",
    CreaseType.VALLEY: "VALLEY",
    CreaseType.CUT: "CUT",
    CreaseType.BOUNDARY: "BOUNDARY",
    CreaseType.AUXILIARY: "AUXILIARY",
}
@dataclass
class DxfLine:
    x1: float
    y1: float
    x2: float
    y2: float
    layer: str = "MOUNTAIN"
    color_index: int = 1
    line_type: str = "CONTINUOUS"
@dataclass
class DxfExportOptions:
    unit_scale: float = 1.0
    include_auxiliary: bool = True
    merge_duplicate: bool = True
    precision: int = 6
    layer_overrides: Dict[str, int] = field(default_factory=dict)
class DxfExporter:
    def __init__(self, options: Optional[DxfExportOptions] = None) -> None:
        self.options = options or DxfExportOptions()
        self._lines: List[DxfLine] = []
    def add_line(
        self,
        x1: float,
        y1: float,
        x2: float,
        y2: float,
        crease_type: CreaseType,
    ) -> None:
        layer = CREASE_LAYER_MAP.get(crease_type, "AUXILIARY")
        if layer == "AUXILIARY" and not self.options.include_auxiliary:
            return
        color_index = self.options.layer_overrides.get(layer, LAYER_SPECS[layer][0])
        self._lines.append(
            DxfLine(
                x1=float(x1),
                y1=float(y1),
                x2=float(x2),
                y2=float(y2),
                layer=layer,
                color_index=color_index,
            )
        )
    def add_segments(self, segments: List[Dict]) -> int:
        added = 0
```

```
        for seg in segments:
            try:
                crease_type = seg["crease_type"]
                if isinstance(crease_type, str):
                    crease_type = CreaseType(crease_type)
                self.add_line(
                    seg["x1"], seg["y1"], seg["x2"], seg["y2"], crease_type
                )
                added += 1
            except (KeyError, ValueError, TypeError) as exc:
                logger.warning("跳过无效折痕线段: %s (%s)", seg, exc)
        return added
    def _dedupe(self) -> List[DxfLine]:
        if not self.options.merge_duplicate:
            return list(self._lines)
        seen = set()
        result: List[DxfLine] = []
        for line in self._lines:
            key = (
                round(line.x1, 4),
                round(line.y1, 4),
                round(line.x2, 4),
                round(line.y2, 4),
                line.layer,
            )
            reverse = (
                round(line.x2, 4),
                round(line.y2, 4),
                round(line.x1, 4),
                round(line.y1, 4),
                line.layer,
            )
            if key in seen or reverse in seen:
                continue
            seen.add(key)
            result.append(line)
        return result
    def _format_number(self, value: float) -> str:
        text = f"{value:.{self.options.precision}f}"
        if "." in text:
            text = text.rstrip("0").rstrip(".")
        return text or "0"
    def _build_layer_table(self) -> str:
        used_layers = {line.layer for line in self._lines}
        defs = []
        for name, (color, desc) in LAYER_SPECS.items():
            if name not in used_layers and name != "BOUNDARY":
                continue
            color_index = self.options.layer_overrides.get(name, color)
            defs.append(
                "0\nLAYER\n2\n{name}\n70\n0\n62\n{color}\n6\nCONTINUOUS\n".format(
                    name=name, color=color_index
                )
            )
        return DXF_TABLES.format(layer_count=len(defs), layer_defs="".join(defs))
    def _build_entity(self, line: DxfLine) -> str:
        return (
            "0\nLINE\n8\n{layer}\n62\n{color}\n6\n{line_type}\n"
            "10\n{x1}\n20\n{y1}\n30\n0.0\n"
            "11\n{x2}\n21\n{y2}\n31\n0.0\n"
        ).format(
            layer=line.layer,
            color=line.color_index,
            line_type=line.line_type,
            x1=self._format_number(line.x1 * self.options.unit_scale),
            y1=self._format_number(line.y1 * self.options.unit_scale),
            x2=self._format_number(line.x2 * self.options.unit_scale),
            y2=self._format_number(line.y2 * self.options.unit_scale),
        )
    def build_content(self) -> str:
        if not self._lines:
            raise ExportError("没有可导出的折痕线段")
        lines = self._dedupe()
        self._lines = lines
        parts = [DXF_HEADER, self._build_layer_table(), DXF_ENTITIES_OPEN]
        for line in lines:
            parts.append(self._build_entity(line))
        parts.append(DXF_ENTITIES_CLOSE)
        return "".join(parts)
    def export(self, file_path: str) -> int:
        content = self.build_content()
        try:
            with open(file_path, "w", encoding="utf-8", newline="\n") as fp:
```

```
                fp.write(content)
        except OSError as exc:
            logger.error("DXF 导出失败: %s", exc)
            raise ExportError(f"无法写入 DXF 文件: {exc}") from exc
        logger.info("DXF 导出完成: %s (%d 条线段)", file_path, len(self._lines))
        return len(self._lines)
    @staticmethod
    def validate_content(content: str) -> bool:
        if not content or "SECTION" not in content or "EOF" not in content:
            return False
        return content.count("0\nSECTION") == content.count("0\nENDSEC")
    @staticmethod
    def estimate_bounds(lines: List[DxfLine]) -> Tuple[float, float, float, float]:
        if not lines:
            return (0.0, 0.0, 0.0, 0.0)
        xs = [line.x1 for line in lines] + [line.x2 for line in lines]
        ys = [line.y1 for line in lines] + [line.y2 for line in lines]
        return (min(xs), min(ys), max(xs), max(ys))
    @staticmethod
    def line_length(line: DxfLine) -> float:
        return math.hypot(line.x2 - line.x1, line.y2 - line.y1)
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/io/fold_exporter.py
import json
import math
from pathlib import Path
from typing import Dict, List, Tuple
from app.common import logger
from app.exceptions import ExportError
from app.models.crease import Crease, CreaseType
from app.models.project import Project
FOLD_VERSION = "1.2"
FOLD_FILE_SPEC = "fold"
FOLD_FILE_MIME = "application/octet-stream"
CREASE_TYPE_TO_FOLD = {
    CreaseType.MOUNTAIN: "M",
    CreaseType.VALLEY: "V",
    CreaseType.FLAT: "F",
    CreaseType.CUT: "C",
    CreaseType.BOUNDARY: "B",
    CreaseType.AUXILIARY: "U",
}
class FoldExporter:
    def __init__(self, project: Project):
        if project is None:
            raise ExportError("工程对象为空，无法导出 FOLD 文件")
        self.project = project
        self._warnings: List[str] = []
    @property
    def warnings(self) -> List[str]:
        return list(self._warnings)
    def _collect_creases(self) -> List[Crease]:
        creases: List[Crease] = []
        for layer in self.project.layers:
            if not layer.visible:
                continue
            creases.extend(layer.creases)
        return creases
    def _build_vertex_index(self, creases: List[Crease]) -> Tuple[List[List[float]], Dict[int, int]]:
        vertices: List[List[float]] = []
        index_map: Dict[int, int] = {}
        for crease in creases:
            for vertex in (crease.start, crease.end):
                if vertex.id in index_map:
                    continue
                index_map[vertex.id] = len(vertices)
                vertices.append([round(float(vertex.x), 6), round(float(vertex.y), 6)])
        return vertices, index_map
    def _build_edges(self, creases: List[Crease], index_map: Dict[int, int]) -> Tuple[List[List[int]], List[str],
List[Dict]]:
        edges: List[List[int]] = []
        assignments: List[str] = []
        metadata: List[Dict] = []
        for crease in creases:
            start_index = index_map.get(crease.start.id)
            end_index = index_map.get(crease.end.id)
            if start_index is None or end_index is None:
                self._warnings.append(f"折痕 {crease.id} 端点缺失，已跳过")
                continue
            if start_index == end_index:
```

```
                self._warnings.append(f"折痕 {crease.id} 起止点重合，已跳过")
                continue
            edges.append([start_index, end_index])
            assignments.append(CREASE_TYPE_TO_FOLD.get(crease.crease_type, "F"))
            metadata.append({
                "id": crease.id,
                "layer": crease.layer_name,
                "color": crease.color,
                "width": round(float(crease.width), 4),
                "label": crease.label or "",
            })
        return edges, assignments, metadata
    def _build_faces(self, creases: List[Crease], index_map: Dict[int, int]) -> List[List[int]]:
        faces: List[List[int]] = []
        for face in getattr(self.project, "faces", []) or []:
            indices = [index_map.get(v.id) for v in face.vertices]
            if any(i is None for i in indices):
                self._warnings.append(f"面片 {face.id} 存在未引用顶点，已跳过")
                continue
            if len(indices) < 3:
                self._warnings.append(f"面片 {face.id} 顶点数不足，已跳过")
                continue
            faces.append([int(i) for i in indices])
        return faces
    def _build_metadata(self) -> Dict:
        return {
            "title": self.project.name,
            "creator": "折痕图可视化编辑与标准化导出系统",
            "description": self.project.description or "",
            "unit": self.project.unit or "mm",
            "grid_size": round(float(self.project.grid_size), 4),
            "author": self.project.author or "",
            "created_at": self.project.created_at.isoformat() if self.project.created_at else "",
            "modified_at": self.project.modified_at.isoformat() if self.project.modified_at else "",
        }
    def build_document(self) -> Dict:
        creases = self._collect_creases()
        if not creases:
            raise ExportError("当前工程没有可导出的折痕，请先绘制折痕图")
        vertices, index_map = self._build_vertex_index(creases)
        edges, assignments, edge_meta = self._build_edges(creases, index_map)
        if not edges:
            raise ExportError("折痕数据无效，未生成任何有效边")
        faces = self._build_faces(creases, index_map)
        document = {
            "file_spec": FOLD_FILE_SPEC,
            "file_creator": "折痕图可视化编辑与标准化导出系统",
            "file_version": FOLD_VERSION,
            "file_mime": FOLD_FILE_MIME,
            "file_classes": ["singleModel"],
            "frame_classes": ["creasePattern"],
            "frame_attributes": ["2D"],
            "frame_unit": self.project.unit or "mm",
            "vertices_coords": vertices,
            "edges_vertices": edges,
            "edges_assignment": assignments,
            "edges_metadata": edge_meta,
            "faces_vertices": faces,
            "metadata": self._build_metadata(),
        }
        return document
    def validate_document(self, document: Dict) -> bool:
        coords = document.get("vertices_coords", [])
        edges = document.get("edges_vertices", [])
        assignments = document.get("edges_assignment", [])
        if len(edges) != len(assignments):
            raise ExportError("边数量与折痕类型数量不一致")
        for edge in edges:
            if len(edge) != 2:
                raise ExportError("存在非法边定义")
            for index in edge:
                if index < 0 or index >= len(coords):
                    raise ExportError(f"边引用了越界顶点索引 {index}")
        for coord in coords:
            if len(coord) != 2:
                raise ExportError("顶点坐标维度必须为二维")
            if any(math.isnan(float(v)) or math.isinf(float(v)) for v in coord):
                raise ExportError("顶点坐标包含非法数值")
        return True
    def export_to_file(self, file_path: str, indent: int = 2) -> str:
        target = Path(file_path)
        if target.suffix.lower() != ".fold":
```

```
            target = target.with_suffix(".fold")
        try:
            document = self.build_document()
            self.validate_document(document)
            target.parent.mkdir(parents=True, exist_ok=True)
            with target.open("w", encoding="utf-8") as handle:
                json.dump(document, handle, ensure_ascii=False, indent=indent)
        except ExportError:
            raise
        except OSError as exc:
            logger.error("写入 FOLD 文件失败: %s", exc)
            raise ExportError(f"写入 FOLD 文件失败: {exc}") from exc
        except (TypeError, ValueError) as exc:
            logger.error("序列化 FOLD 数据失败: %s", exc)
            raise ExportError(f"序列化 FOLD 数据失败: {exc}") from exc
        logger.info("已导出 FOLD 文件: %s", target)
        return str(target)
    def export_to_string(self, indent: int = 2) -> str:
        document = self.build_document()
        self.validate_document(document)
        return json.dumps(document, ensure_ascii=False, indent=indent)
    def summary(self) -> Dict:
        creases = self._collect_creases()
        counts: Dict[str, int] = {}
        for crease in creases:
            key = crease.crease_type.value if hasattr(crease.crease_type, "value") else str(crease.crease_type)
            counts[key] = counts.get(key, 0) + 1
        return {
            "project": self.project.name,
            "crease_total": len(creases),
            "type_counts": counts,
            "layer_total": len(self.project.layers),
        }
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/io/fold_importer.py
import json
from typing import Any, Dict, List, Optional, Tuple
from app.common import logger
from app.exceptions import ImportError as CreaseImportError
from app.models.crease import Crease, CreaseType
from app.utils.file_utils import read_text_file
FOLD_VERSION = "1.1"
VALID_CREASE_ASSIGNMENTS = {"M", "V", "F", "B", "C", "U"}
class FoldImporter:
    def __init__(self) -> None:
        self._raw_data: Optional[Dict[str, Any]] = None
        self._vertices: List[Tuple[float, float]] = []
        self._edges: List[Tuple[int, int]] = []
        self._assignments: List[str] = []
        self._warnings: List[str] = []
    @property
    def warnings(self) -> List[str]:
        return list(self._warnings)
    def load_file(self, file_path: str) -> Dict[str, Any]:
        text = read_text_file(file_path)
        if not text:
            raise CreaseImportError(f"无法读取文件内容: {file_path}")
        try:
            data = json.loads(text)
        except json.JSONDecodeError as exc:
            raise CreaseImportError(f"FOLD 文件 JSON 解析失败: {exc}") from exc
        if not isinstance(data, dict):
            raise CreaseImportError("FOLD 文件根节点必须为对象")
        self._raw_data = data
        logger.info("已加载 FOLD 文件: %s", file_path)
        return data
    def parse(self, data: Optional[Dict[str, Any]] = None) -> List[Crease]:
        source = data if data is not None else self._raw_data
        if source is None:
            raise CreaseImportError("尚未加载任何 FOLD 数据")
        self._warnings.clear()
        self._parse_vertices(source)
        self._parse_edges(source)
        self._parse_assignments(source)
        return self._build_creases()
    def _parse_vertices(self, data: Dict[str, Any]) -> None:
        raw_vertices = data.get("vertices_coords")
        if raw_vertices is None:
            raise CreaseImportError("FOLD 文件缺少 vertices_coords 字段")
        if not isinstance(raw_vertices, list):
```

```
            raise CreaseImportError("vertices_coords 必须为数组")
        vertices: List[Tuple[float, float]] = []
        for index, item in enumerate(raw_vertices):
            if not isinstance(item, (list, tuple)) or len(item) < 2:
                raise CreaseImportError(f"顶点 {index} 坐标格式非法")
            try:
                x = float(item[0])
                y = float(item[1])
            except (TypeError, ValueError) as exc:
                raise CreaseImportError(f"顶点 {index} 坐标无法转换为数值") from exc
            vertices.append((x, y))
        self._vertices = vertices
    def _parse_edges(self, data: Dict[str, Any]) -> None:
        raw_edges = data.get("edges_vertices")
        if raw_edges is None:
            raise CreaseImportError("FOLD 文件缺少 edges_vertices 字段")
        if not isinstance(raw_edges, list):
            raise CreaseImportError("edges_vertices 必须为数组")
        vertex_count = len(self._vertices)
        edges: List[Tuple[int, int]] = []
        for index, item in enumerate(raw_edges):
            if not isinstance(item, (list, tuple)) or len(item) != 2:
                raise CreaseImportError(f"边 {index} 顶点索引格式非法")
            try:
                start = int(item[0])
                end = int(item[1])
            except (TypeError, ValueError) as exc:
                raise CreaseImportError(f"边 {index} 顶点索引无法转换为整数") from exc
            if not (0 <= start < vertex_count) or not (0 <= end < vertex_count):
                raise CreaseImportError(f"边 {index} 顶点索引越界")
            if start == end:
                self._warnings.append(f"边 {index} 起止顶点相同，已忽略")
                continue
            edges.append((start, end))
        self._edges = edges
    def _parse_assignments(self, data: Dict[str, Any]) -> None:
        raw_assignments = data.get("edges_assignment")
        edge_count = len(self._edges)
        if raw_assignments is None:
            self._warnings.append("缺少 edges_assignment 字段，全部按辅助线处理")
            self._assignments = ["U"] * edge_count
            return
        if not isinstance(raw_assignments, list):
            raise CreaseImportError("edges_assignment 必须为数组")
        if len(raw_assignments) != edge_count:
            self._warnings.append("edges_assignment 长度与边数量不一致，缺失部分按辅助线处理")
        assignments: List[str] = []
        for index in range(edge_count):
            value = raw_assignments[index] if index < len(raw_assignments) else "U"
            text = str(value).strip().upper()
            if text not in VALID_CREASE_ASSIGNMENTS:
                self._warnings.append(f"边 {index} 折痕类型 '{value}' 非法，按辅助线处理")
                text = "U"
            assignments.append(text)
        self._assignments = assignments
    def _build_creases(self) -> List[Crease]:
        creases: List[Crease] = []
        for index, (start, end) in enumerate(self._edges):
            assignment = self._assignments[index]
            crease_type = self._map_crease_type(assignment)
            start_point = self._vertices[start]
            end_point = self._vertices[end]
            crease = Crease(
                start_x=start_point[0],
                start_y=start_point[1],
                end_x=end_point[0],
                end_y=end_point[1],
                crease_type=crease_type,
            )
            creases.append(crease)
        logger.info("FOLD 解析完成，共 %d 条折痕", len(creases))
        return creases
    @staticmethod
    def _map_crease_type(assignment: str) -> CreaseType:
        mapping = {
            "M": CreaseType.MOUNTAIN,
            "V": CreaseType.VALLEY,
            "F": CreaseType.FLAT,
            "B": CreaseType.BOUNDARY,
            "C": CreaseType.CUT,
            "U": CreaseType.AUXILIARY,
        }
```

```
        return mapping.get(assignment, CreaseType.AUXILIARY)
    def extract_metadata(self) -> Dict[str, Any]:
        if self._raw_data is None:
            return {}
        metadata: Dict[str, Any] = {}
        for key in ("file_creator", "file_title", "file_description", "file_classes"):
            if key in self._raw_data:
                metadata[key] = self._raw_data[key]
        metadata["fold_version"] = self._raw_data.get("file_spec", FOLD_VERSION)
        return metadata
    def validate_structure(self) -> Tuple[bool, List[str]]:
        issues: List[str] = []
        if self._raw_data is None:
            return False, ["尚未加载 FOLD 数据"]
        if not self._vertices:
            issues.append("折痕图不包含任何顶点")
        if not self._edges:
            issues.append("折痕图不包含任何折痕边")
        if len(self._assignments) != len(self._edges):
            issues.append("折痕类型数量与边数量不匹配")
        return len(issues) == 0, issues
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
# 文件: app/io/svg_exporter.py
import math
from typing import List, Dict, Optional, Tuple
from xml.sax.saxutils import escape
from app.common import logger
from app.exceptions import ExportError
from app.models.crease import Crease, CreaseType
from app.models.layer import Layer
from app.utils.color_utils import normalize_hex_color
from app.utils.unit_utils import mm_to_px
CREASE_STYLE: Dict[CreaseType, Dict[str, str]] = {
    CreaseType.MOUNTAIN: {"stroke": "
    CreaseType.VALLEY: {"stroke": "
    CreaseType.FLAT: {"stroke": "
    CreaseType.CUT: {"stroke": "
    CreaseType.BOUNDARY: {"stroke": "
    CreaseType.AUXILIARY: {"stroke": "
}
class SvgExporter:
    def __init__(self, scale: float = 1.0, margin: float = 20.0):
        self.scale = scale if scale > 0 else 1.0
        self.margin = margin if margin >= 0 else 0.0
        self._warnings: List[str] = []
    @property
    def warnings(self) -> List[str]:
        return list(self._warnings)
    def _resolve_style(self, crease: Crease) -> Dict[str, str]:
        base = CREASE_STYLE.get(crease.crease_type, CREASE_STYLE[CreaseType.FLAT])
        style = dict(base)
        if crease.color:
            style["stroke"] = normalize_hex_color(crease.color, fallback=base["stroke"])
        if crease.line_width and crease.line_width > 0:
            style["width"] = f"{crease.line_width:.2f}"
        return style
    def _compute_bounds(self, creases: List[Crease]) -> Tuple[float, float, float, float]:
        xs: List[float] = []
        ys: List[float] = []
        for crease in creases:
            xs.extend([crease.start_x, crease.end_x])
            ys.extend([crease.start_y, crease.end_y])
        if not xs or not ys:
            return 0.0, 0.0, 100.0, 100.0
        min_x, max_x = min(xs), max(xs)
        min_y, max_y = min(ys), max(ys)
        if math.isclose(min_x, max_x):
            max_x = min_x + 1.0
        if math.isclose(min_y, max_y):
            max_y = min_y + 1.0
        return min_x, min_y, max_x, max_y
    def _validate(self, creases: List[Crease]) -> None:
        self._warnings.clear()
        for index, crease in enumerate(creases):
            if not isinstance(crease, Crease):
                raise ExportError(f"第 {index + 1} 条记录不是有效的折痕对象")
            if math.isclose(crease.start_x, crease.end_x) and math.isclose(crease.start_y, crease.end_y):
                self._warnings.append(f"折痕 {crease.crease_id} 起止点重合，已跳过")
            if crease.crease_type not in CREASE_STYLE:
                self._warnings.append(f"折痕 {crease.crease_id} 类型未知，按平折处理")
```

```
    def _build_defs(self, layers: Optional[List[Layer]]) -> str:
        parts = ["<defs>"]
        parts.append(
            '<marker id="arrow-end" markerWidth="8" markerHeight="8" refX="6" refY="3" '
            'orient="auto" markerUnits="strokeWidth">'
            '<path d="M0,0 L6,3 L0,6 z" fill="'
        )
        if layers:
            for layer in layers:
                color = normalize_hex_color(getattr(layer, "color", ""), fallback="
                parts.append(
                    f'<pattern id="layer-{layer.layer_id}" width="8" height="8" '
                    f'patternUnits="userSpaceOnUse">'
                    f'<rect width="8" height="8" fill="{color}" fill-opacity="0.08"/></pattern>'
                )
        parts.append("</defs>")
        return "".join(parts)
    def _render_crease(self, crease: Crease) -> str:
        style = self._resolve_style(crease)
        x1 = mm_to_px(crease.start_x, self.scale)
        y1 = mm_to_px(crease.start_y, self.scale)
        x2 = mm_to_px(crease.end_x, self.scale)
        y2 = mm_to_px(crease.end_y, self.scale)
        dash_attr = "" if style["dash"] == "none" else f' stroke-dasharray="{style["dash"]}"'
        opacity = getattr(crease, "opacity", 1.0)
        opacity = 1.0 if opacity is None else max(0.0, min(1.0, float(opacity)))
        return (
            f'<line id="crease-{crease.crease_id}" class="crease {crease.crease_type.value}" '
            f'x1="{x1:.3f}" y1="{y1:.3f}" x2="{x2:.3f}" y2="{y2:.3f}" '
            f'stroke="{style["stroke"]}" stroke-width="{style["width"]}"'
            f'{dash_attr} stroke-linecap="round" stroke-opacity="{opacity:.2f}"/>'
        )
    def _render_vertex(self, crease: Crease) -> str:
        radius = 1.6
        points = [
            (mm_to_px(crease.start_x, self.scale), mm_to_px(crease.start_y, self.scale)),
            (mm_to_px(crease.end_x, self.scale), mm_to_px(crease.end_y, self.scale)),
        ]
        circles = []
        for idx, (cx, cy) in enumerate(points):
            circles.append(
                f'<circle class="vertex" data-crease="{crease.crease_id}" data-end="{idx}" '
                f'cx="{cx:.3f}" cy="{cy:.3f}" r="{radius}" fill="'
            )
        return "".join(circles)
    def build_svg(self, creases: List[Crease], layers: Optional[List[Layer]] = None,
                  title: str = "折痕图") -> str:
        self._validate(creases)
        min_x, min_y, max_x, max_y = self._compute_bounds(creases)
        width = mm_to_px(max_x - min_x, self.scale) + self.margin * 2
        height = mm_to_px(max_y - min_y, self.scale) + self.margin * 2
        offset_x = self.margin - mm_to_px(min_x, self.scale)
        offset_y = self.margin - mm_to_px(min_y, self.scale)
        body: List[str] = []
        body.append('<?xml version="1.0" encoding="UTF-8"?>')
        body.append(
            f'<svg xmlns="http://www.w3.org/2000/svg" version="1.1" '
            f'width="{width:.3f}" height="{height:.3f}" '
            f'viewBox="0 0 {width:.3f} {height:.3f}">'
        )
        body.append(f"<title>{escape(title)}</title>")
        body.append(self._build_defs(layers))
        body.append(
            '<style type="text/css">'
            '.crease{fill:none;}'
            '.crease.mountain{stroke-linecap:round;}'
            '.crease.valley{stroke-linecap:round;}'
            '.vertex{pointer-events:none;}'
            '</style>'
        )
        body.append(f'<g id="crease-layer" transform="translate({offset_x:.3f},{offset_y:.3f})">')
        for crease in creases:
            if math.isclose(crease.start_x, crease.end_x) and math.isclose(crease.start_y, crease.end_y):
                continue
            body.append(self._render_crease(crease))
        body.append("</g>")
        body.append('<g id="vertex-layer">')
        for crease in creases:
            body.append(self._render_vertex(crease))
        body.append("</g>")
        body.append("</svg>")
        return "\n".join(body)
    def export(self, file_path: str, creases: List[Crease],
```

```
                layers: Optional[List[Layer]] = None, title: str = "折痕图") -> str:
        if not file_path:
            raise ExportError("导出路径不能为空")
        if not file_path.lower().endswith(".svg"):
            file_path = file_path + ".svg"
        try:
            content = self.build_svg(creases, layers, title)
            with open(file_path, "w", encoding="utf-8") as fp:
                fp.write(content)
        except ExportError:
            raise
        except OSError as exc:
            logger.error("SVG 导出失败: %s", exc)
            raise ExportError(f"无法写入文件: {exc}") from exc
        for warning in self._warnings:
            logger.warning("SVG 导出提示: %s", warning)
        logger.info("已导出 SVG 折痕图: %s (%d 条折痕)", file_path, len(creases))
        return file_path
</□□DSML□□ parameter>
</□□DSML□□ invoke>
</□□DSML□□ calls>
```